Appunti di Calcolatori

Alessio Novel

Dove studiare....3
Introduzione....3
  Prestazioni....5
Aritmetica calcolatori....6
  Numeri interi....6
    Matematica dell’orologio....10
  Numeri reali....10
  Codifica del testo....12
Reti logiche....12
  Algebra di Boole....13
    Regole base....13
    Regole di De Morgan....14
  Reti sequenziali....16
    Latch S-R....17
    Latch D....18
    Flip-flop Master-Slave....18
  Registri....19
Assembly....19
    RISC....21
    CISC....21
RISC-V....23
  Principio 1: semplicità....23
  Principio 2: velocità....24
  Principio 3: buon progetto richiede buoni compromessi....25
  Operazioni logiche....26
  Istruzioni condizionali....29
  Procedure e funzioni....31
  Uso dei registri in RISC-V....35
Intel....38
    Registri....38
    Istruzioni....39
ARM....45
    Registri....46
    Istruzioni....47
Toolchain....53
  GCC....55
    Compilatore....56
    Assembler....56
    Linker....57

Librerie..... 57
Il processore..... 58
Add..... 71
Load..... 72
Beq..... 73
Pipeline..... 74
Vantaggi RISC..... 76
Hazard..... 77
La gerarchia di memoria..... 80
La RAM..... 81
SRAM..... 82
DRAM..... 83
Gestione della memoria..... 87
Cache..... 89
Mappatura diretta..... 89
Associative..... 93
Set-associative..... 93
Dispositivi I/O..... 96
Bus sincrono..... 98
Bus asincrono..... 98
Interrupt..... 103
Eccezioni..... 106

Dove studiare

Dove studiare cosa

Indicativamente:

- Lez. 1: Sez. 1.1-1.5, 1.7, 1.8, cenni 1.6 + Slide/materiale su Moodle
- Lez. 2-4: Sez. 2.4, 3.1-3.3, 3.5 (no HW e RISC-V) + Slide /materiale su Moodle
- Lez. 5: Sez. 2.6 + Slide/materiale su Moodle
- Lez. 6-10: Sez. 2.1-2.11 + Slide/materiale su Moodle
- Lez. 11: Sez. 2.17 + Slide/materiale su Moodle
- Lez. 12: Slide/materiale su Moodle
- Lez. 13-15: Slide/materiale su Moodle
- Lez. 16: Sez. 2.12-2.15 + Slide/materiale su Moodle
- Lez. 17-20: Cap. 4 (in parte) + Slide/materiale su Moodle
- Lez. 20-22: Cap. 5 (in parte) + Slide/materiale su Moodle
- Lez. 23-24: Slide/materiale su Moodle

Introduzione

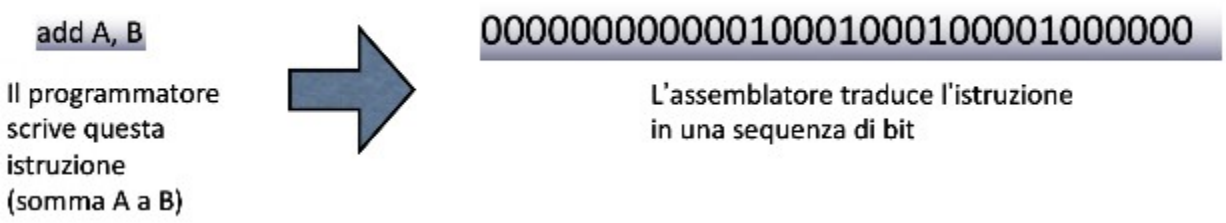
I calcolatori al momento sono importantissimi, negli Stati Uniti ad esempio producono il 10% del Prodotto Interno Lordo.
Nel 1946 venne messa in funzione ENIAC (commissionata per il calcolo delle traiettorie dei proiettili) e generò un blackout, la macchina occupava una stanza
Nel 1969 Margaret Hamilton progetto IBM che guidò le esplorazioni lunari, oggi gli iPhone sono grandi come una mano e sono molto più potenti.
I primi personal computer vennero fuori negli anni 50
Ovviamente bisogna distinguere vari tipi di calcolatori in base ad utilizzo e tipo
Esistono 3 categorie di calcolatori:

- Personal computer (desktop e laptop)
  Unico utente alla volta
  Buone prestazioni a costo ridotto
  Pensati per eseguire software di terze parti
- Server
  Più utenti alla volta (in rete per esempio)
  Eseguono grandi carichi di lavoro
    - Poche applicazioni ma molto complesse

    - Tante applicazioni ma molto semplici
- Embedded (IoT)
  Microprocessori a stretto contatto con l'hardware, che è la parte principale
  Applicazioni dedicate

Studiare i calcolatori e il loro funzionamento serve per poter capire meglio come ottimizzare il codice sapendo cosa succede ai bassi livelli.
Ci sono vari elementi che influenzano le prestazioni di un programma come algoritmi, linguaggi, compilatori, architetture, processore, sistema di memoria, sistema operativo, hardware

Il **sistema operativo** è il programma principale che gira su un calcolatore, il quale alloca la memoria, esegue le operazioni di i/o e il multitasking (in realtà la simultaneità in un unico processore è solo un illusione)
Il **compilatore** è un programma che traduce da linguaggio ad alto livello a linguaggio macchina
Il componente base di un calcolatore sono le **porte logiche**, che corrispondono ad **interruttori** elettrici. Infatti l'unita di base di informazione è il bit (0 o 1)
All'inizio i programmatori scrivevano il codice in binario, poi venne inventato un linguaggio più intuitivo, il quale veniva tradotto dall'**assembler** in binario, chiamato **assembly**
L'assembly è un linguaggio che viene tradotto, non compilato, e traduce istruzioni da "stringhe" in bit.
Il linguaggio assembly richiede una linea di codice per ogni istruzione (non vale sempre ma per gli assembly più base) che il calcolatore eseguirà a livello hardware.



Di solito per far girare un programma scritto in un linguaggio di alto livello, si traduce prima in assembly e poi in codice macchina (cioè bit), ma molti compilatori saltano la fase dell'assembly e traducono direttamente in linguaggio macchina.
Il processore è composto dalla **datapath** (cioè la ALU), che esegue le operazioni aritmetiche, e dalla **parte di controllo**.
La memoria cache è molto vicina al processore e permette di salvare i dati utilizzati più frequentemente per ottimizzare il loro accesso.

Nel mondo dell'informatica si usano spesso le astrazioni, che permettono di gestire un progetto di grande complessità nascondendone i dettagli.
ISA: (Instruction Set Architecture) è un'interfaccia che raggruppa tutte le funzioni che il processore offre a livello hardware (i/o, registre etc) così da poterle usare dai programmi senza preoccuparsi di come le istruzioni vengono implementate
ABI: (Application Binary Interface) unendo l'ISA con l'interfaccia del sistema operativo (per dispositivi I/O, memoria etc), otteniamo l'ABI, la quale definisce anche come utilizzare i registri a livello software

L’importanza dell’astrazione e delle architetture è che i programmatori possono scrivere il programma indipendentemente dall’hardware su cui sarà eseguito
La memoria si divide in volatile (ram) e non volatile (hard disk, ssd)

I calcolatori esprimono la maggior parte del loro potenziale quando sono interconnessi (LAN, WAN)

Prestazioni

In un calcolatore si possono identificare due tipi di prestazioni:

- **Tempo medio di risposta**
  Questo è interesse del singolo utente al quale interessa quanto tempo passa tra avvio e terminazione di un task
- **Throughput**
  Questo è interesse lato server, per esempio il gestore del centro di calcolo vuole sapere quanti task (cioè quanti utenti posso soddisfare) in un’unità di tempo

Ovviamente in una macchina multitask, il tempo di risposta dipende anche dagli altri task attivi, che possono occupare tempo della macchina.
Il **tempo di esecuzione della CPU** è invece il tempo effettivo su cui la CPU lavora per il task (quindi non tiene conto delle “attese” di completamento degli altri task), il tempo di esecuzione è dedicato in parte al programma utente e in parte al sistema operativo.



I processori sono costruiti usando un segnale periodico, chiamato **clock**, che ne sincronizza le operazioni.
Il **ciclo di clock** è l’intervallo di tempo (misurata in secondi) che intercorre tra due colpi di clock.
La **frequenza** di clock è l’inverso del ciclo di clock, ed è misurata in Hertz.

Il modo migliore per misurare le prestazioni di un computer è misurando il tempo necessario per l’esecuzione di un programma (essendo lo scopo del computer).
Il tempo necessario per eseguire un determinato programma è influenzato da:

- Numero di istruzioni
- Numero di cicli di clock medi per istruzione (CPI)
- Frequenza di clock (Oppure l’inverso, il Ciclo di clock)

$$Tempo = Numero\ \ Istruzioni\ \times\ CPI \times\ Ciclo\ \ di\ \ Clock = \frac{Numero\ \ Istruzioni\ \times\ CPI}{Frequenza\ \ Clock}$$

Notiamo che il tempo è misurato in secondi, il numero di istruzioni e il numero di cicli di clock per istruzione sono dei numeri normali, e il ciclo di clock è misurato in secondi, quindi le misure tornano.
Quindi non possiamo dire che un processore è “migliore” di un altro solo perchè aumenta il Ciclo di clock, perchè le prestazioni dipendono anche dagli altri fattori, ad esempio l’algoritmo usato potrebbe aumentare il numero di istruzioni e l’implementazione delle istruzioni aumentare il numero di cicli di clock per un istruzione.
L’ISA (l’interfaccia che la macchina offre al software) è il fattore che influenza di più sul tempo della CPU perchè va a toccare tutti e 3 i fattori
Ad esempio per ottimizzare i tempi si può scegliere un algoritmo che usi meno istruzioni, oppure implementare delle istruzioni più efficienti (ovvero con un basso Clock per Istruzione)
Ovviamente ogni linguaggio di programmazione ottimizza in modo diverso algoritmi e istruzioni
Il problema dello “stop delle prestazioni” è dovuto ad un limite fisico perché nella formula della potenza richiesta da un processore

$$Potenza = capacità \times tensione^2 \times frequenza\ \ di\ \ commutazione$$

la tensione è elevata al quadrato e quindi il trucco è stato quello di abbassare la tensione aumentando la frequenza (e limitando così l’aumento di consumi), solo che se si abbassa la tensione sotto al Volt ci sono dei fenomeni di scarica che danno dei problemi.
Visto questo limite la strada che si è scelta di intraprendere è quella del parallelismo, con processori multi-core, questa strada però porta con sé delle difficoltà per il programmatore in quanto:

- è molto più difficile progettare un programma che opera in maniera parallela. (**correttezza**)
- bisogna garantire che il carico di lavoro sulle CPU si mantenga bilanciato (altrimenti sarebbe inutile il multi-core) (**efficienza**)

Aritmetica calcolatori

Tutto ciò che c’è all’interno di un calcolatore è rappresentato da bit (0 o 1, i circuiti elettronici fanno o passare la corrente (1) o non la fanno passare (0)) e 8 bit formano un byte.
Bit = binary digit
Le sequenze di bit devono poter rappresentare in qualche modo qualsiasi tipo di dato che viene gestito in un calcolatore (numeri, parole, simboli, immagini, suoni, **programmi**, etc.)
Per poter “leggere” e utilizzare le informazioni di questi bit c’è bisogno di definire una codifica (una funzione che associa ad un oggetto una sequenza di bit, una **rappresentazione digitale**).
Con una sequenza di n bit si possono rappresentare 2^n oggetti diversi

Numeri interi

Noi umani usiamo il sistema decimale ma nulla vieta di usare un sistema con più o meno cifre.
Se scegliamo un sistema di numerazione con base B (**base di numerazione**), avremo a disposizione B cifre, cioè da 0 a $B-1$, il valore di ogni cifra in una sequenza di cifre dipende dalla sua posizione:

$$c_i c_{i-1} c_{i-2} \dots c_0 = c_i * B^i + c_{i-1} * B^{i-1} + \dots + c_0 * B^0 = \sum_{k=0}^{i} c_k B^k$$

Ad esempio il sistema binario sembra fatto apposta per i computer (solo due simboli).
Con k cifre binarie, posso rappresentare i numeri interi che vanno da 0 a $2^k - 1$ (di solito si usano $k = 8,16,32,64$, cioè potenze di 2)
Notiamo che ogni byte (formato da 8 bit) può assumere valori da 0 a $2^8 - 1$, cioè da 0 a 255.
Visto che i numeri in base 2 richiedono un gran numero di cifre, spesso si usa la base **esadecimale** per ridurre il numero di cifre.
La conversione da binario a esadecimale (e viceversa) è molto semplice, basta prendere gruppi di 4 bit, di conseguenza ogni cifra esadecimale corrisponde a 4 cifre binarie
Per passare da base B a base 10 abbiamo già visto come fare, per passare da base 10 a base B invece dobbiamo dividere iteramente per B

1. Per convertire x_{10} in base B:
2. Divisione intera fra x e B
3. Il resto e' la cifra da inserire a sinistra nel numero convertito
4. Il quoziente viene assegnato ad x
5. Ritorna al punto 2

$1000_{10} = ?_2$

X	$X/2$	$X\%2$
1000	500	0
500	250	0
250	125	0
125	62	1
62	31	0
31	15	1
15	7	1
7	3	1
3	1	1
1	0	1

$1000_{10} = 1111101000_2$

$1000_{10} = ?_{16}$

X	$X/16$	$X\%16$
1000	62	8
62	3	14 (E)
3	0	3

$1000_{10} = 3E8_{16}$

Somma di bit:

A	B	Carry	C	Carry
0	0	0	0	0
1	0	0	1	0
0	1	0	1	0
1	1	0	0	1
0	0	1	1	0
1	0	1	0	1
0	1	1	0	1
1	1	1	1	1

A + B + Carry = C + Carry

Sottrazione di bit:

A	0	1	1	0
B	0	0	1	1
A-B	0	1	0	1

Notiamo che, come quando moltiplichiamo un numero in base decimale per una potenza di 10, dobbiamo aggiungere tanti zeri quanto è la potenza, la stessa cosa vale per i numeri in base binaria con le potenze di 2 e questa operazione viene chiamata **shifting**.
Questo è comodo nelle moltiplicazioni tra due numeri in base binaria, in quanto basta ricopiare il numero shiftato in base a dove sono gli zeri e gli uni.

Codifiche per numeri interi (quindi anche negativi):

- Codifica con modulo e segno
  - Il bit più significativo (a sinistra) rappresenta il segno
    - 0 = positivo
    - 1 = negativo
  - Gli altri $k-1$ bit rappresentano il valore assoluto
  - Range di numeri da $-2^{k-1}+1$ a $2^{k-1}-1$
  - Doppia codifica dello zero ($+0$ e -0)
- Codifica complemento a 1
  - Il bit più significativo (a sinistra) rappresenta il segno
    - 0 = positivo
    - 1 = negativo
  - In caso di numero negativo si calcola il complemento a 1 per ottenere il valore assoluto del numero e ci si aggiunge il meno davanti
  - Range di numeri da $-2^{k-1}+1$ a $2^{k-1}-1$

- Doppia codifica dello zero
- Il vantaggio rispetto a modulo e segno è che le operazioni di somma e sottrazione sono più facili da eseguire
  - Se nella somma c'è riporto nella cifra più significativa bisogna sommarla al risultato
  - Se i segni degli addendi sono uguali ma il segno del risultato è diverso vuol dire che c'è stato un overflow
-

- Somma di numeri interi codificati in complemento a 1:
  1. Sommare le rappresentazioni dei due numeri
  2. Riporto sul bit più significativo $\rightarrow$ sommarlo al risultato
  3. Se i segni dei due operandi sono uguali ma diversi da quello del risultato di quest'ultima somma, il risultato non è attendibile, ovvero non è rappresentabile su k bit (overflow)
  4. Altrimenti, il risultato è corretto
- Esempio: $6 + -3$ (codifica su 5 bit)
  - $6 = 00110; -3 = \overline{00011} = 11100$

$$\begin{array}{r} \overset{(1)}{} \\ 0\ 0\ 1\ 1\ 0 \\ 1\ 1\ 1\ 0\ 0 \\ \hline 0\ 0\ 0\ 1\ 0 \end{array}$$

  - $00010 + 1 = 00011\ (= 3)$

- Codifica complemento a 2
  - Il bit più significativo (a sinistra) rappresenta il segno
    - 0 = positivo
    - 1 = negativo
  - Il complemento a 2 si trova calcolando il complemento a 1 e sommando 1 (oppure scorrendo da destra e lasciare invariato fino al primo bit uguale ad 1 compreso, da lì in poi invertire tutti i bit)
  - In caso di numero negativo si calcola il complemento a 2 per ottenere il valore assoluto del numero e ci si aggiunge il meno davanti
  - Range di numeri da -2^{k-1} a $2^{k-1}-1$
  - La codifica dello zero è unica
  - Operazioni di somma e sottrazione + semplice

Inoltre, anche la somma è piú semplice...

- $x + (-y) = x + (2^k - y) = 2^k + x - y...$
- ... su k bit, questo è equivalente a $x - y$

    - Le somme tra segni concordi possono causare overflow, questo capita se e solo se i riporti in colonna n e $n+1$ sono diversi
    - Le somme tra segni discordi non causano overflow e il riporto della cifra + significativa si ignora

L'overflow si ha quando la somma va oltre al range disponibile con quei determinati bit (-2^{k-1} a $2^{k-1}-1$), l'overflow si verifica solo se hanno segni concordi

Matematica dell'orologio

Considero solo numeri interi

- Da 0 a $n-1$
- Da $-\frac{n}{2}$ a $\frac{n}{2}-1$

Ogni volta che arrivo al numero più grande, riparto dal più piccolo
La definizione più formale è che viene fatto il modulo

Quindi eseguendo l'operazione $a+b$ in realtà otteniamo $(a+b)\ \%\ 2^k$
Parlando invece di complemento a 2 avremo:

- Se $a+b<-2^{k-1}$, ottengo $a+b+2^k$ (il quale sarà positivo -> overflow)
- Se $a+b>2^{k-1}-1$, ottengo $a+b-2^k$ (il quale sarà negativo -> overflow)

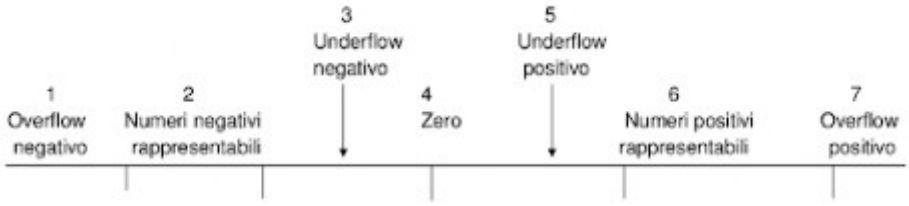
Numeri reali

Certi numeri reali non sono rappresentabili su un numero finito di bit (numeri irrazionali ad es. π), quindi in certi casi possiamo rappresentare solo loro approssimazione

- Virgola fissa (fixed point)
    - Rappresentabili su k bit
        - $k-f$ bit per la parte intera
        - f bit per la parte decimale
    - Converto in base-10 come se fosse non avesse virgola e poi moltiplico per B^{-f}
        - $x_{10} = \sum_{i=0}^{k-1} c_i B^{i-f} = \sum_{i=0}^{k-1} c_i B^i \cdot B^{-f} = \left(\sum_{i=0}^{k-1} c_i B^i\right) \cdot B^{-f}$
        - In altre parole, convertiamo il numero in base 10 come se non avesse virgola e poi moltiplichiamo per B^{-f}.
    - Operazioni in maniera semplice, visto che si possono effettuare come se fossero numeri interi e poi scalare il risultato.
    - Il problema è che con una virgola fissata non riesco a rappresentare numeri con grande differenza di ordine di grandezza (tipo numeri molto piccoli e numeri molto grandi)
- Virgola mobile (floating point)
    - Un numero reale x può essere riscritto come $x = M \cdot B^E$
        - M = Mantissa
        - E = Esponente
    - Rappresentabili su k bit

    - m bit per la Mantissa M (in realtà $m-1$ perchè c'è anche il segno)
    - $e=k-m$ bit per l'esponente E
- La Mantissa e l'Esponente possono essere rappresentati in complemento a 2 (standard) o con altre tecniche di codifica
- Il segno viene rappresentato all'interno della Mantissa
- Nello standard IEEE 754, sull'esponente viene fatta un'operazione di shifting $E'=E+2^{e-1}-1$, cosicché sia sempre non negativo (sarà 0 nel caso dei numeri non normalizzati)
- Lo standard si usa con $k=32\ \ bit, e=8\ \ bit, m=23\ \ bit\ \ (+1\ \ per\ \ il\ \ segno)$
- Se $E'\ >0$, allora si dice numero normalizzato
    - Mantissa = $1.M$ (si omette l'1 per risparmiare bit)
    - Esponente $E=E'-\left(2^{e-1}-1\right)$
    - Nel caso di 32 bit: $\left(-1\right)^{s}\cdot 1.M\cdot 2^{E'-127}$
- Se $E'\ =0$, allora si dice numero denormalizzato
    - Mantissa = $0.M$ (si omette lo 0 per risparmiare bit)
    - Esponente $E=-\left(2^{e-1}-2\right)$
    - Nel caso di 32 bit: $\left(-1\right)^{s}\cdot 0.M\cdot 2^{-126}$
- I numeri denormalizzati vengono inseriti per poter essere più precisi per numeri "piccoli", infatti è per questo che l'esponente è uguale al minimo per i numeri normalizzati (sempre 126), però avendo la mantissa che inizia con 0 permette di rappresentare numeri molto più piccoli
- Vantaggio su ordinamento (prima guardo segno, poi esponente e poi mantissa)
- La seguente tabella mostra le combinazioni esponente-mantissa e quali numeri rappresentano

Categoria	E'	M	s
Numeri normalizzati	1-254	qualunque	0/1
Numeri denormalizzati	0	non zero	0/1
± Zero	0	0	0/1
± Infinito	255	0	0/1
NaN (Not a Number)	255	non zero	0/1



Codifica del testo

Il problema è rappresentare i caratteri come sequenze di 0 e 1, ricordiamoci che con n bit possiamo codificare 2^n simboli diversi.
L'ASCII è uno standard a 7 bit il quale codifica i caratteri in numeri (128 simboli)

- Visto che un byte ha 8 bit, il bit più significativo è sempre 0

Esiste però l'extended ASCII, il quale è creato per i caratteri speciali, il quale usa 8 bit (quindi altri 128 simboli):

- Se il primo bit è 0, la codifica è uguale all'ASCII a 7 bit
- Altrimenti si usa l'ASCII esteso (ce ne sono diversi in base ai paesi e agli alfabeti)

Esempio

- Codifichiamo la parola "Ciao"
  - C è codificata come $67 = 0x43 = 01000011$
  - i è codificata come $105 = 0x69 = 01101001$
  - a è codificata come $97 = 0x61 = 01100001$
  - o è codificata come $111 = 0x6F = 01101111$

01000011	01101001	01100001	01101111
C	i	a	o

Quindi se il byte ha un valore minore di 128 (primo bit = 0) la codifica avviene in modo univoco, altrimenti la codifica non è più univoca perché dipende tutto da quale tipo di Extended ASCII viene usato
Il problema era che nella condivisione di un documento i caratteri fuori dall'ASCII a 7 bit si sfasavano perchè appunto dipendeva dall' Extended ASCII della macchina su cui si leggeva il file.
Viene introdotta la codifica UNICODE la quale da' una codifica univoca a tutti i possibili caratteri, la più usata è UTF-32 con 4 byte, quindi 32 bit (2^{32} possibili simboli)
Esiste anche la UTF-16 e la UTF-8 .
Tutte queste codifiche sono compatibili con ASCII, per quanto riguarda gli ultimi 7 bit.

Reti logiche

I computer sono realizzati tramite circuiti elettronici, che permettono di salvare il valore logico di un bit e scambiarlo.
Essendo che sono elementi digitali, avremo due livelli fondamentali:

- Alto (1), vero
- Basso (0), falso

Tutti i livelli intermedi non hanno importanza per il calcolatore e sono assunti solo in fase transitoria.
In passato si usavano le valvole, ora però si usano i transistor.
Coi transistor, in base alla resistenza, si può far passare la corrente (1) oppure bloccarla (0).

Le reti logiche sono dei circuiti che trasformano alcuni valori logici in ingresso in altri valori logici in uscita.
Le reti logiche sono di due tipi:

- Combinatorie
  Sono statiche e non hanno memoria
  L'uscita dipende solo dal valore di ingresso
- Sequenziali
  L'uscita non dipende solo dall'ingresso ma anche dallo storico degli ingressi passati, quindi in un certo senso hanno memoria (stato)

Tutti gli elementi dei calcolatori sono delle combinazioni di questi due tipi di reti logiche.
Un modo per rappresentare il mapping input/output è usare la tabella della verità, da cui per ogni combinazione di input si legge la combinazione di output

INPUT			OUTPUT		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

Algebra di Boole

Permette di esprimere in maniera più compatta le espressioni logiche tramite espressioni algebriche già conosciute:

- AND
  Rappresentato con il simbolo del prodotto: $\cdot$
  Produce 1 in output solo se tutti e due gli operatori sono 1, produce 0 in tutti gli altri casi
- OR
  Rappresentato con il simbolo della somma: $+$
  Produce 0 in output solo se tutti e due gli operandi sono 0, produce 1 in tutti gli altri casi
- NOT
  Rappresentato con il simbolo della barra: $\underline{A}$
  Inverte il valore logico

Regole base

- Identità
  $A + 0 = A$
  $A \cdot 1 = A$

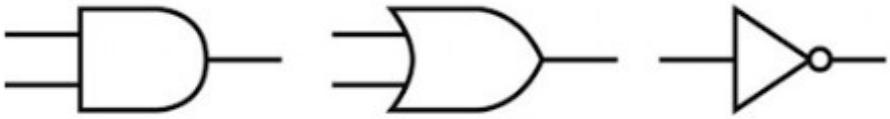
- Regola zero e uno
  $A+1=1$
  $A\cdot 0=0$
- Regole dell'inversa
  $A+\underline{A}=1$
  $A\cdot \underline{A}=0$
- Regola commutativa
  $A+B=B+A$
  $A\cdot B=B\cdot A$
- Regola associativa
  $A+(B+C)=(A+B)+C$
  $A\cdot(B\cdot C)=(A\cdot B)\cdot C$
- Regola distributiva
  $A\cdot(B+C)=(A\cdot B)+(A\cdot C)$
  $A+(B\cdot C)=(A+B)\cdot(A+C)$ (questa non vale "matematicamente" ma vale solo quando si parla di operazioni logiche)

Regole di De Morgan

$$\underline{A\cdot B}=\underline{A}+\underline{B}$$
$$\underline{A+B}=\underline{A}\cdot\underline{B}$$

Queste regole ci dicono che se abbiamo un elemento logico che implementa NAND o NOR, allora possiamo ricavare da esso tutte le operazioni logiche

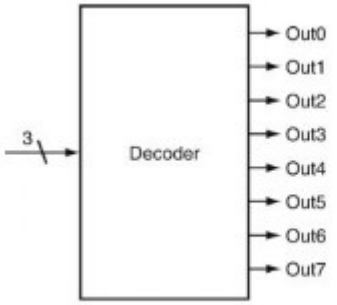
Esistono dei circuiti elettronici (porte logiche) che implementano gli operatori booleani fondamentali e si possono combinare tra loro



Esistono vari tipi di circuito

- Decoder
  Prende in ingresso n bit e da in uscita m bit
  Questo funziona perché per esempio con un input di 3 bit si possono rappresentare 2^3

simboli diversi, e di conseguenza un output di 8 bit

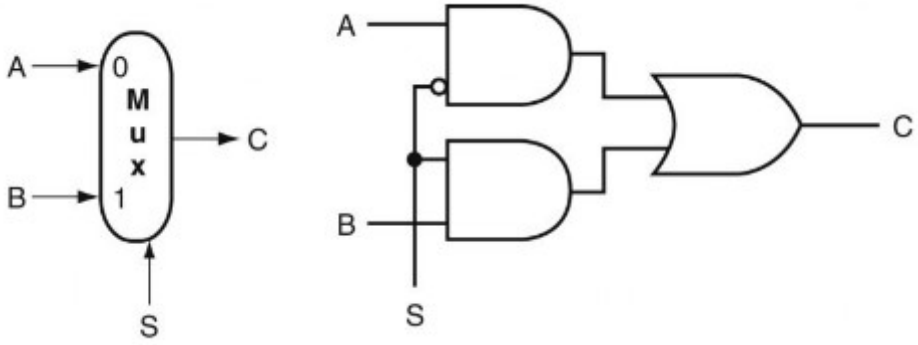


a. A 3-bit decoder

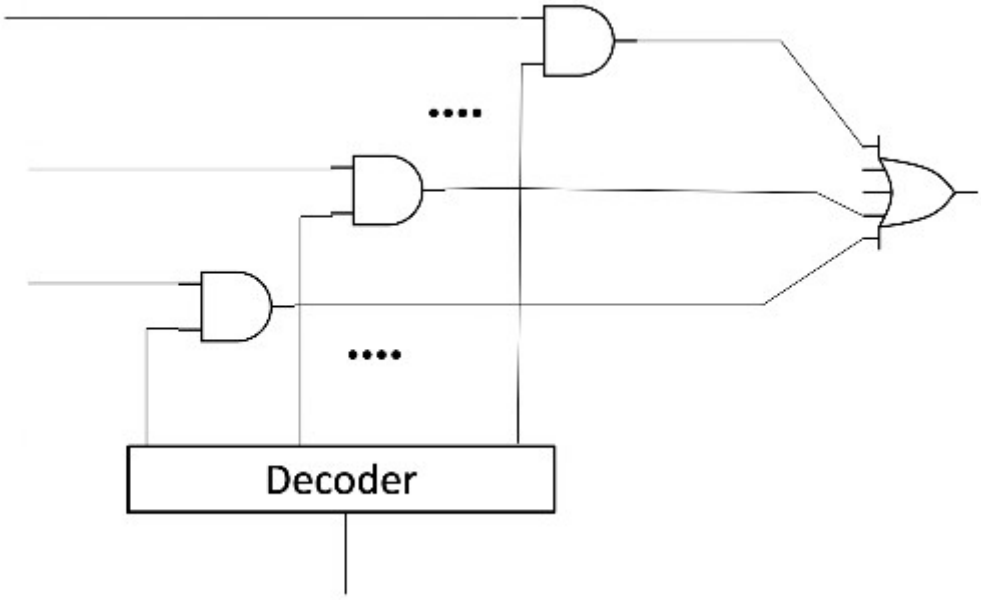
Inputs			Outputs							
In2	In1	In0	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

b. The truth table for a 3-bit decoder

- Multiplexer
  Due ingressi logici e un selettore, quindi il risultato viene “scelto” tra gli input a seconda del selettore

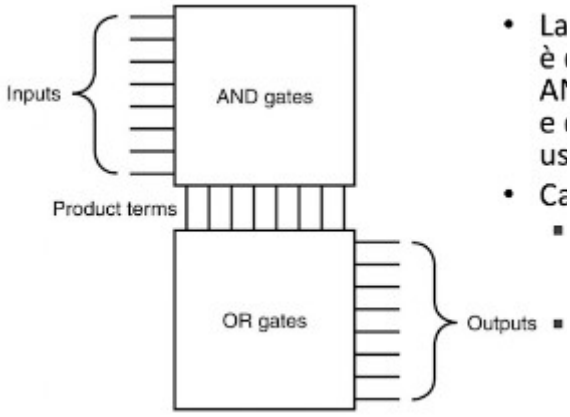


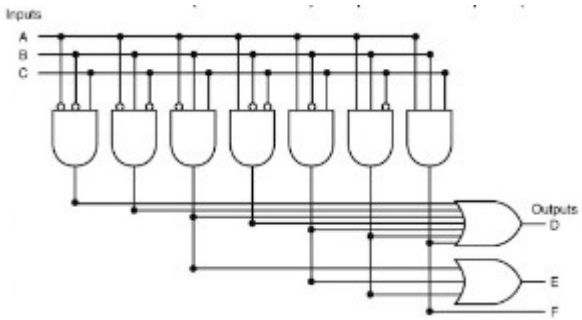
- Multiplexer a n vie
  Tanti multiplexer che prendono il selettore da un decoder



Calcolare l’espressione logica partendo da una tabella della verità è molto semplice facendo combinazioni di AND e OR.

Basta prendere ciascuna riga uguale a 1 e mettere in AND tutti gli ingressi mettendo in NOT quelli equivalenti a 0, successivamente mettere in OR tutte queste righe





La struttura che si forma è una cosiddetta PLA (Programmable Logic Array) che si compone di due stadi, la prima è una barriera di AND (detti anche mintermini) e la seconda una di OR
Caratteristiche PLA:

- La dimensione è data dalla somma del piano AND (numero di mintermini) e del piano OR (numero di uscite)
- Ci sono porte logiche solo per le configurazioni che producono 1 in uscita
- Se un mintermine è condiviso tra varie uscite lo si può riutilizzare

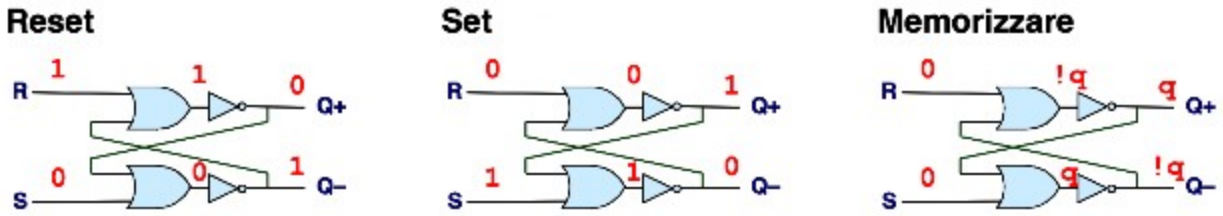
Per costo di una rete logica si intende la somma del numero di porte logiche e del numeri di ingressi, è possibile avere implementazioni (e di conseguenza costi) diverse per ogni rete
Prima di mettersi a implementare una funziona logica è opportuno analizzarla e vedere se si può “semplificare” tramite le regole algebriche (questo argomento si chiama **sintesi logica**)

Un BUS è un insieme di input che viaggiano assieme, praticamente un array di bit.
Ad esempio un multiplexer che prende in input due array da 32 bit in realtà è implementato come 32 multiplexer che prendono lo stesso bit come selettore.

Reti sequenziali

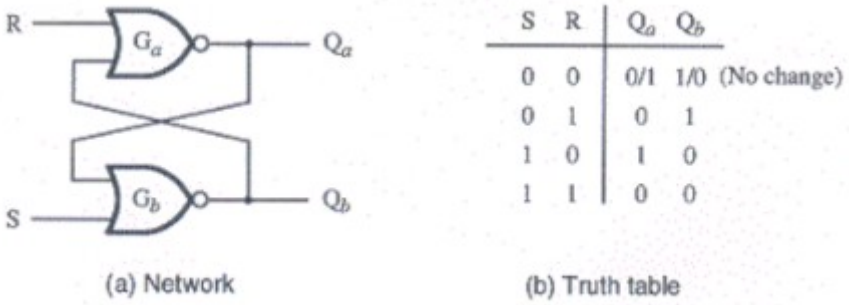
Tutte le funzioni logiche viste finora e le relative reti sono note come reti combinatorie, le quali non hanno una nozione di “tempo” e non hanno memoria del passato, infatti ogni uscita dipende solo dagli ingressi.
Però in certe applicazioni c’è bisogno di introdurre una “memoria” nel sistema.
Per implementare ciò bisogna ridirezionare l’uscita in modo che vadano in ingresso ad altre porte del medesimo stadio, formando un anello (un loop).
Questo evento si chiama reazione, e provoca un aumento della complessità di analisi.
Quindi la memoria è dovuta al fatto che certi ingressi “ricordano” l’uscita passata

Latch S-R

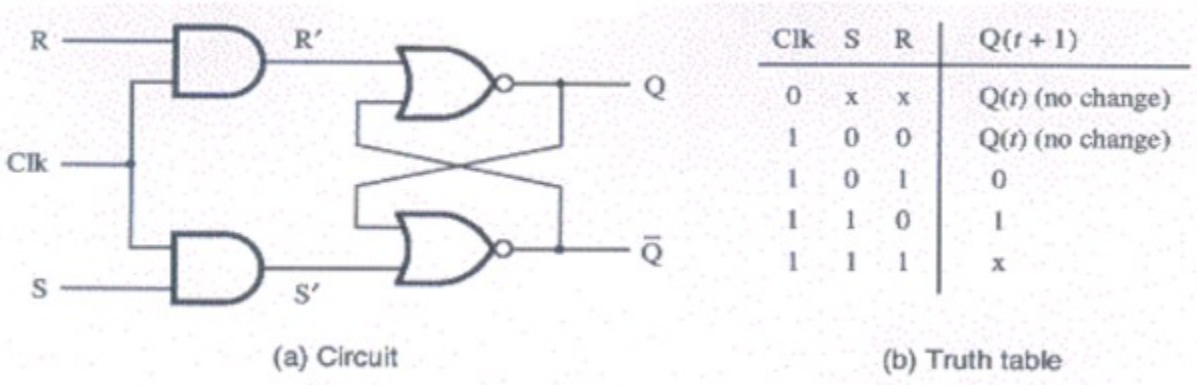


Questo circuito si chiama **Latch S-R** (set-reset)
In questo esempio con le varie combinazioni degli input si può:

- Reset
  Mettere le uscite in 0 e 1
- Set
  Mettere le uscite in 1 e 0
- Memorizzare
  Lasciare le uscite al loro stato



Visto che il tempo della propagazione del segnale non è mai nullo, l'effetto del cambio di un ingresso richiede del tempo per propagarsi sulle uscite.
Infatti gli elementi di memoria sono quindi sempre temporizzati, da un tempo scandito dal **clock**, per evitare problemi di indecibilità



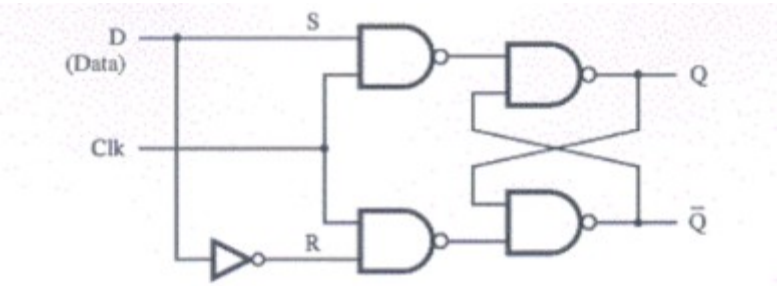
Con questa implementazione quando il clock è a zero, il sistema rimane statico essendo che il 0 del clock domina negli AND, invece quando il clock è a 1, il sistema si aggiorna e funziona normalmente

Il difetto di Latch S-R è che c'è uno stato indecidibile, cioè l'uscita non può essere nota con certezza, quando entrambi gli ingressi sono a 1

Per rimediare a questo problema esistono **Latch-D** (data) e **flip-flop**

Latch D

Gli ingressi del circuito sono ottenuti da un'unica variabile (e non più due distinte) la quale viene presa da un lato e negata dall'altro

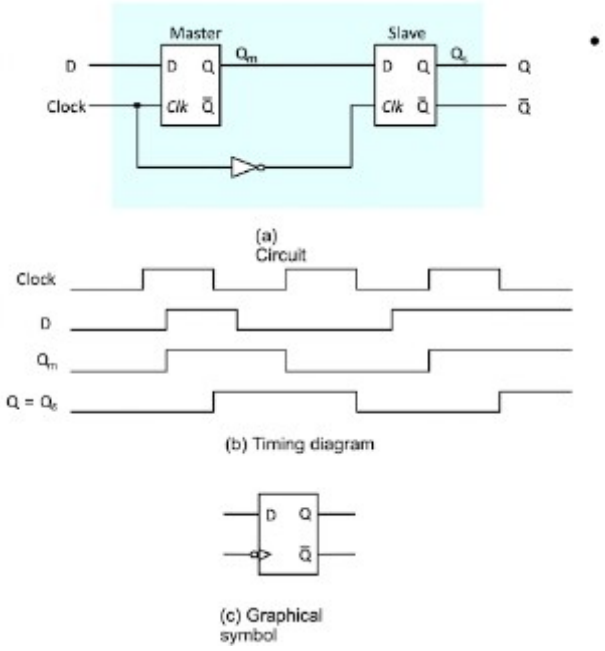


Clk	D	Q(t + 1)
0	x	Q(t)
1	0	0
1	1	1

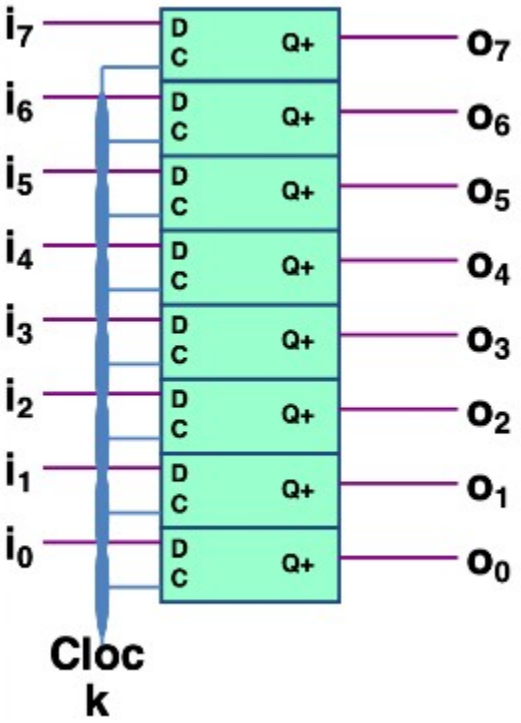
(b) Truth table

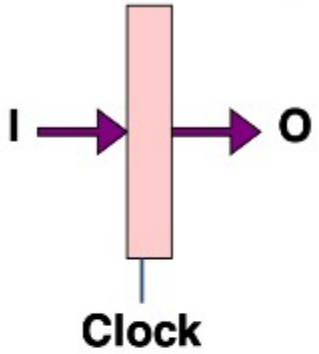
Flip-flop Master-Slave

Configurazioni più complesse come questa permettono di ottenere che l'uscita del circuito commuti esattamente al termine dell'impulso del clock. Caricano gli input durante il fronte di salita del clock

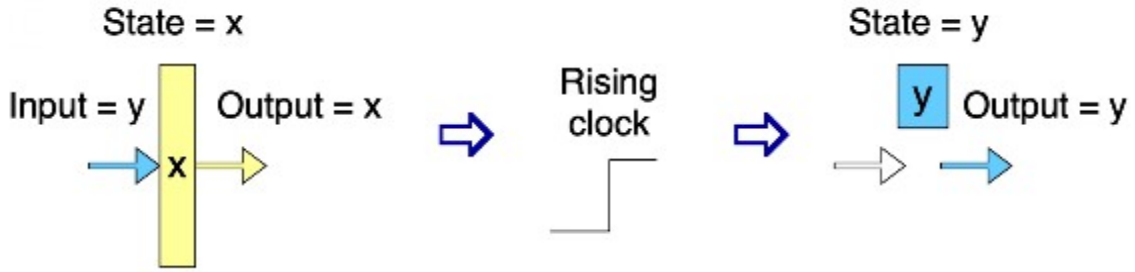


Registri





I registri sono delle reti sequenziali che caricano gli input sul fronte in salita del clock, questi circuiti vengono chiamati latch edge-triggered (catturano l'informazione nel momento di un cambiamento, o da 1 a 0 o da 0 a 1)



Questa metodologia permette di aggiornare lo stato, a partire da quello presente, senza creare delle situazioni di corse.
Esistono due tipi di macchine a stati:

- Macchina di Mealy
  L'output dipende dallo stato presente e dall'input
- Macchina di Moore
  L'output dipende solo dallo stato presente

Assembly

La CPU capisce solo il suo linguaggio macchina, quindi sequenze di 0 e 1.

Il programmatore di solito scrive programmi in linguaggi di alto livello (Java, C++, etc).
Il compromesso è Assembly. Ogni istruzione in linguaggio assembly corrisponde ad una istruzione in linguaggio macchina (salvo rare eccezioni, come macro, label).
Assembly è composto da codici mnemonici (in ASCII) che vengono poi tradotte in bit.
L'assembler è il compilatore da Assembly in linguaggio macchina.
L'assembly non è un linguaggio portabile (perchè dipende strettamente dalla CPU)

L'insieme di istruzioni riconosciute da una CPU è definito dalla ISA, che non è un semplice elenco di istruzioni, visto che ci associa una sintassi, una semantica e definisce le modalità di accesso ai dati e registri.
Per quanto riguarda i dati, solitamente vengono presi e salvati nei registri, ed esiste un sistema di indirizzamento (come accedere alla memoria centrale).
L'ISA quindi definisce il linguaggio Assembly da utilizzare. Un processore supporta più ISA.
Struttura di un programma Assembly:

1. Fetch
   Preleva l'istruzione dalla memoria
   L'indirizzo dell'istruzione si trova nell'apposito registro (PC oppure IP)
2. Decode
   Decodifica l'istruzione e capisce cosa deve fare
3. Execute
   Esegue l'istruzione (che sia aritmetica, accesso alla memoria, etc)

Un programma in Assembly è una lista di istruzioni eseguite prevalentemente in ordine sequenziale (non totalmente perché esistono istruzioni di salto che rompono la sequenzialità).
Non esistono cicli o selezioni, ma solo salti!

Tipi di dato su cui opera un linguaggio assembly:

- Costante (immediato)
- Contenuto in registri
- Contenuto in memoria (esistono varie modalità di indirizzamento)

Un registro a k bit è una batteria di k flip-flop di tipo D
Il numero e il tipo di registri varia in base alla ISA (in genere da 4 a 64)

Alcune ISA ammettono solo istruzioni con operandi che sono dei primi due tipi (immediati o in registri), altre ISA invece permettono di accedere direttamente alla memoria.
Un' altra differenza è il poter decidere la destinazione di un'operazione (talvolta è implicita e fissa)
Due importanti filosofie di progettazione delle architetture sono:

- RISC (Reduced Instruction Set Computer)
  Architettura molto semplice e le istruzioni agiscono solo sui registri
  Minor numero di istruzioni, meno potenti
  Semplifica l'implementazione della CPU
- CISC (Complex Instruction Set Computer)
  Architettura molto complicata ma favorisce la facilità dello sviluppo dei programmi

Possibilità di avere operandi o destinazione in memoria
Maggior numero di istruzioni, più potenti

Un altro elemento molto importante è il poter mettere delle condizioni (per i salti e le selezioni)
Le architetture di tipo RISC usano dei registri generici per eseguire le condizioni. Nelle architetture CISC il risultato delle condizioni verrà salvato in un registro a parte fatto di flag
Tipi di istruzioni:

- Istruzioni aritmetiche e logiche
  Eseguite dalla ALU
- Istruzioni di movimento dati
  Per caricare costanti in memoria oppure per caricare dati contenuti in registri in memoria (o viceversa)
- Istruzioni di salto / controllo del flusso (con invocazione di subroutine e ritorno da subroutine)

RISC

Obiettivo: semplificare al massimo la struttura della CPU

- Istruzioni aritmetiche e logiche
  <opcode> <dst>, <arg1>, <arg2>
  <dst>, <arg1> sono registri
  <arg2> registro o valore immediato
- Istruzioni di movimento dati (solo 2)
  load <reg>, <memory location>
  store <memory location>, <reg>

Ovviamente servono più registri visto che si possono usare solo quelli
Codifica: numero fisso di bit

CISC

Obiettivo: fornire istruzioni più potenti/flessibili/performanti

- Istruzioni aritmetiche e logiche
  Possono avere operandi e/o destinazione in memoria, ma in certi casi ci sono comunque dei limiti

Codifica: numero variabile di bit
Sintassi meno regolare

Quindi meglio RISC o CISC?
Sono 2 tipi di ISA differenti con obiettivi differenti, come in ogni cosa, soluzioni estremiste non sono mai una buona idea, quindi le soluzioni “di successo” sono un mix dei due approcci

Nel corso vedremo tre ISA:

- RISC-V (RISC)
- Intel (CISC)

- ARM (RISC “pragmatico”, ovvero modalità di indirizzamento complesse, istruzioni complesse e ha meno registri)

Nelle RISC esistono solo due istruzioni per accedere alla memoria, in CISC invece lo possono fare tutte.
Come si esprime <memory location>? Esistono varie modalità di indirizzamento

1. Assoluto
   Indirizzo di memoria costante espresso come valore immediato
   Problematico per RISC perché la grandezza in bit è statica
2. Indiretto
   Indirizzo di memoria contenuto in un registro
   Utile per implementare puntatori
3. Base + Spiazzamento (2+1)
   Indirizzo ottenuto shiftando (manipolando) il contenuto di un registro con valori costanti (immediati)
4.
   a. Base + Indice (2+3)
      Sommando a un indirizzo in un registro, un valore di un altro registro (ad esempio per scorrere array)
   b. Base + Indice + Spiazzamento (1+2+3)
      Un mix di tutto, quindi sommando all’indirizzo di un registro, un valore contenuto in un altro registro, e anche un valore immediato

Le architetture di tipo RISC forniscono solo indirizzamenti di tipo 3.
Nelle CISC invece è permesso anche la 4a
ARM è un eccezione perchè permette di shiftare con pre-incremento e post-incremento
L’ISA definisce il numero di registri e il tipo dei registri, i registri si dividono in due tipi:

- General purpose
  Qualsiasi cosa, qualsiasi istruzione
- Registri specializzati
  Un sottoinsieme di registri che possono essere usati solo per alcuni scopi specifici

Quindi per sapere come usare i registri, visto che questo non dipende dall’hardware, esiste un’altra interfaccia, la ABI, la quale è un insieme di convenzioni software che dicono come usare i registri.
La ABI è indipendente dall’architettura perché, appunto, è a livello software
L’ABI definisce:

- Come e dove passare i parametri
- Quali registri preservare
- Quando un programma invoca una subroutine, quali registri conteranno un certo valore al ritorno della subroutine

L’**instruction set** è l’insieme delle istruzioni che il processore mette a disposizione, gli IS sono tutti molto simili, cambia qualche sintassi e qualche particolare.

I tre principi di progettazione stabiliti da Von-Neumann sono:

- Semplicità dei dispositivi richiesti
- Chiarezza delle applicazioni
- Velocità di esecuzione

Le istruzioni e i programmi sono memorizzati nella memoria nello stesso identico modo in cui sono memorizzati i dati
Nel corso vedremo tre IS:

- RISC-V (RISC)
  Moderna e open-source
  Abbastanza diffusa nelle applicazioni specifiche
- Intel (CISC)
  È l'architettura più diffusa nei PC che usiamo
- ARM (RISC "pragmatico", ovvero modalità di indirizzamento complesse, istruzioni complesse e ha meno registri)
  È una via di mezzo tra RISC e CISC
  Utilizzato nella maggior parte dei sistemi embedded

Una base delle IS è che prevedano delle istruzioni per il calcolo delle operazioni aritmetiche fondamentali

RISC-V

Principio 1: semplicità

Visto che il modo più semplice di immaginare una istruzione aritmetica è a tre operandi, RISC-V prevede istruzioni aritmetiche **soltanto** a tre operandi
Ad esempio:

$$a = b + c \Rightarrow add \;\; a, b, c$$

Istruzioni più complesse si ottengono dalla combinazione di istruzioni semplici. Quando in un linguaggio di alto livello abbiamo un'espressione "complessa", dobbiamo sempre ricondurci ad operazioni elementari a basso livello

In RISC-V i commenti iniziano con # e continuano fino alla fine della riga.
In RISC-V l'operando destinatario dell'operazione è sempre il primo
(Queste regole non valgono per tutti gli assemblatori)

In realtà le operazioni viste in precedenza non vanno bene perchè non esistono le variabili in linguaggi a basso livello come assembly, ma possiamo usare solo operandi contenuti nei **registri**
In un ciclo di clock il processore accede a tutti i registri
Il RISC-V contiene 32 registri a 64 bit:

- Gruppi di 64 bit: double word
- Gruppi di 32 bit: word

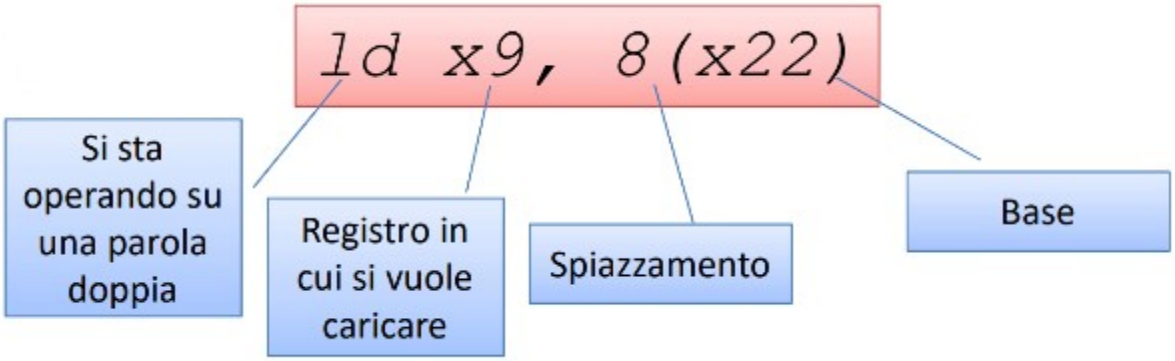
Principio 2: velocità

Avere “solo” 32 registri aumenta la velocità in quanto il processore deve percorrere meno “distanza” e di conseguenza il ciclo di clock sarà più veloce

Quindi le operazione aritmetiche e logiche si effettuano solo tra i registri, ma il problema è che i registri non bastano per poter eseguire un programma con tanti dati, esistono infatti due operazioni di trasferimento da/verso la memoria:

- load
  Preleva i dati da una locazione di memoria e li carica nei registri
- store
  Salva il contenuto del registro nella memoria

La memoria è una sequenza di bit organizzati in gruppi da 8, chiamati byte; ogni byte ha un indirizzo di memoria lineare progressivo
Solitamente però si trasferiscono multipli di 4 byte (word) o di 8 byte (double word)
Per caricare una word, double word, byte è necessario specificare l’indirizzo, in RISC-V questo si fa tramite una **base** (contenuta in un registro) a cui si somma uno **spiazzamento/offset** (costante)



Nei linguaggi di alto livello ci sono tantissime variabili, però i registri del processore abbiamo detto essere pochi, per questo esiste lo **register spilling**, un’operazione del compilatore che trasferisce i valori dei registri che non vengono utilizzati per un po’ di tempo in memoria, e li riprende quando servono, così da liberare i registri per altre variabili.
In pratica lo register spilling sono delle operazioni di store/load fatte implicitamente dal compilatore

Molto spesso nelle operazioni aritmetiche abbiamo a che fare con costanti, quindi esistono delle operazioni che lavorano su costanti (per non dover ogni volta salvare la costante in memoria).
Per esempio per la somma $addi \;\; x22, x22,4$ (la i sta per immediate)

Esistono delle costanti molto utili che semplificano alcune operazioni, ad esempio per trasferire il valore di un registro in un altro si può usare la somma con 0, per questo motivo esiste un registro ad hoc per questa costante ($x0$)

Quando si memorizza una parola bisogna sapere dove va il byte più significativo e dove quello meno significativo:

- Little Endian (Intel e RISC-V)



- Big Endian (Motorola e protocolli internet)



Come i dati, anche i programmi devono essere tradotti in bit per essere salvati in memoria ed essere capiti dal calcolatore
Quindi per l'operazione di somma $add\ \ a,b,c$ ci sarà una sequenza di bit con alcuni bit che indicano che questa è una somma, e altri bit per indicare operandi e destinazione.
In RISC-V ogni operazione viene codificata con una word (32 bit)

In RISC-V i registri vanno da 0 a 31, quindi possono essere rappresentati in 5 bit



- *codop:* codice operativo dell'istruzione
- *funz7* e *funz3*: codici operativi aggiuntivi
- *rs1:* primo operando sorgente
- *rs2:* secondo operando sorgente
- *rd:* operando destinazione

Principio 3: buon progetto richiede buoni compromessi

Nel nostro caso scegliere istruzioni a 32 bit costa molto in termini di:

- Limite sul numero di istruzioni
- Limite sul numero di registri
- Limite alle modalità di indirizzamento

Però viene guadagnato molto a livello di efficienza
Esistono vari formati per le istruzioni:

- R (registro)
- I (immediato)
- Logiche

- Scorrimento
- Salti condizionati
- Salti incondizionati

Istruzione (R)	funz7	rs2	rs1	funz3	rd	codop	Esempio
add	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
sub (sottrazione)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3
Istruzione (I)	**immediato**		**rs1**	**funz3**	**rd**	**codop**	**Esempio**
addi (addizione immediata)	001111101000		00010	000	00001	0010011	addi x1,x2,1000
ld (caricamento di parola doppia)	001111101000		00010	011	00001	0000011	ld x1, 1000 (x2)
Istruzione (S)	**Immediato**	**rs2**	**rs1**	**funz3**	**immediato**	**codop**	**Esempio**
sd (memorizzazione di parola doppia)	0011111	00001	00010	011	01000	0100011	sd x1, 1000 (x2)

Un programma dunque consiste in una sequenza di bit, che viene scritta nella RAM.
In momenti diversi si possono rappresentare programmi diversi nella stessa RAM

Operazioni logiche

All'inizio i calcolatori operavano su word intere, però ci fu la necessità di operare su porzioni di word o addirittura sul singolo bit

Lo shift logico (a destra o a sinistra) ad esempio consiste nel traslare tutto di una posizione.
Si inseriscono gli 0 (o a destra o a sinistra) e si perdono gli ultimi bit in caso di overflow
Shift a sinistra

slli regArr , regPart , nShift

```
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00001001

slli x11, x19, 4

00000000 00000000 00000000 00000000 00000000 00000000 00000000 10010000
```

Quindi nell'indirizzo $x11$ troverò il valore di $x19$ spostato a sinistra di 4 posizioni (moltiplicato per 2^4)



In RISC-V i numeri vengono codificati in complemento a 2.
Se effettuiamo la precedente operazione di shifting nel caso in cui in $x19$ ci sia un numero negativo "grande", otterremo un numero positivo (perché il primo bit a 1 viene "perso"), questo è un caso di overflow
L'overflow dipende tutto dal range di disponibilità
Shift a destra

srli regArr , regPart , nShift

```
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00011000
```

- Se effettuiamo:

```
srli x11, x19, 4
```

- Otteniamo:

```
00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000001
```

Ovviamente bisogna vederla come una divisione tra interi, quindi appunto 24 diviso 16 fa 1 con il resto di 8, e quindi lo shift da come risultato 1

Nel caso del shift a destra di un numero negativo, verrà sempre fuori un numero positivo, anche nel caso in cui il numero è rappresentabile perfettamente
Questo si risolve con lo shift aritmetico (solo a destra), il quale non inserisce 0 a sinistra ma bit in base al segno del numero

```
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111110
```

```
srai x11, x19, 4
```

```
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111
```

Da notare che lo shift aritmetico a sinistra non ha senso perché non risolve il caso di overflow
Altre operazioni logiche

Operazioni logiche	Operatori C	Operatori Java	Istruzioni RISC-V
Shift a sinistra	<<	<<	sll, slli
Shift a destra	>>	>>>	srl, srli
Shift a destra aritmetico	>>	>>	sra, srai
AND bit a bit	&	&	and, andi
OR bit a bit	|	|	or, ori
XOR bit a bit	^	^	xor, xori
NOT bit a bit	~	~	xori

AND bit a bit

```
00000000 00000000 00000000 00000000 00000000 00000000 00010011 10000001
```

```
00000000 00000000 00000000 00000000 00000000 00000000 11111111 00000000
```

```
and x9, x10, x11
```

- Risultato:

```
00000000 00000000 00000000 00000000 00000000 00000000 00010011 00000000
```

L'AND si può usare per implementare una maschera binaria, mettendo a 1 tutte le parti che mi interessa mantenere e a 0 tutto il resto, così facendo si possono selezionare alcuni byte alla volta

OR bit a bit
Analogamente all'AND, si può usare una maschera per settare alcuni bit a 1

OR esclusivo (o XOR)
L'output è 1 solo quando i due bit in input sono diversi

```
00000000 00000000 00000000 00000000 00000000 01100011 01110000 11001101
```

```
00000000 00000000 00000000 00000000 00000000 10011011 01110000 00111101
```

```
xor x11, x9, x10
```

- Risultato:

```
00000000 00000000 00000000 00000000 00000000 11111000 00000000 11110000
```

Lo XOR si può usare in maniera pratica per cancellare il valore di un registro, ad esempio $XOR\ \ x9, x9, x9$ viene cancellato il registro $x9$ in quanto tutti i bit presi singolarmente sono uguali
In forma SP si può scrivere

$$A\ \ XOR\ \ B = (A\ \wedge\ !B)\ \vee\ (!A\ \wedge\ B)$$

NOT
Il NOT non è fornito nativamente da RISC-V (essendo un operatore unario), tuttavia possiamo ottenerlo come

$$!A = A\ \ XOR\ \ 1$$

```
00000000 00000000 00000000 00000000 00000000 01100011 01100011 11001101
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111
```

```
xor x9, x9, x10
```

- Risultato:

```
11111111 11111111 11111111 11111111 11111111 10011100 10011100 00110010
```

Istruzioni condizionali

Una caratteristica fondamentale dei calcolatori, che li distingue dalle calcolatrici, è la possibilità di alterare il flusso del programma al verificarsi di certe condizioni

Il linguaggio macchina supporta questa funzionalità con istruzioni di **salto condizionato**, tipicamente i salti avvengono sulla base di certe condizioni nei registri
Due istruzioni di salto condizionato:
branch-on-equal

$$beq\ \ rs1,\ rs2,\ L1$$

Se il registro $rs1$ è uguale al registro $rs2$ effettua un salto all'istruzione con etichetta L1
branch-on-not-equal

$$bne\ \ rs1,\ rs2,\ L1$$

Se il registro $rs1$ è diverso dal registro $rs2$ effettua un salto all'istruzione con etichetta L1
Costrutto if

```
      bne x22, x23, ELSE  # Salta a ELSE se x22 div. da x23
      add x19, x20, x21   # f = g + h
      beq x0, x0, ESCI    # Salto incondizionato a ESCI
ELSE: sub x19, x20, x21   # f = g - h
ESCI: …
```

Osservazione:
Le label vengono alla fine tradotte in indirizzi dal compilatore
Costrutto while
Ciclo while con scorrimento di array

```
Ciclo: slli x10, x22, 3     # Registro temp. x10 = 8*i
       add x10, x10, x25    # Ind. di salva[i] in x10
       ld x9, 0(x10)        # Carica salva[i] in x9
       bne x9, x24, Esci    # Esci se raggiunto limite
       addi x22, x22, 1     # i = i+1
       beq x0, x0, Ciclo
Esci: …
```

Le sequenze di istruzioni tra due salti condizionati sono così importanti da essere chiamati **blocchi di base**

Tutti i cicli in linguaggio ad alto livello vengono implementati con blocchi base a basso livello

Oltre a uguale e diverso, sono utili altri operandi di confronto, e RISC-V mette a disposizione:

Salta se minore di	`blt x5, x6, 100`	`Se (x5 < x6) vai a PC+100`
Salta se maggiore o uguale di	`bge x5, x6, 100`	`Se (x5 >= x6) vai a PC+100`
Salta se minore di senza segno	`bltu x5, x6, 100`	`Se (x5 < x6) vai a PC+100`
Salta se maggiore o uguale di senza segno	`bgeu x5, x6, 100`	`Se (x5 >= x6) vai a PC+100`

Esistono due tipi di confronto (con e senza segno) che ovviamente sono due operazioni con risultati diversi

Il confronto unsigned non fa il complemento a 2 ma lo tratta come se fosse un numero normale

Questo è utile a livello pratico quando si controlla che il valore di un registro non sia fuori dal range di un array, perchè se è un numero positivo il confronto sarà normale, invece nel caso di numeri negativi sarà sempre > al range (perchè avrà la cifra più significativa a 1) e quindi si possono fare due controlli con una istruzione

Costrutto switch-case

O si fa una sequenza in cascata di if-then-else oppure si usa una tecnica diversa tramite *jalr* (jump and link register, la vedremo più avanti), in pratica si memorizza un vettore di indirizzi e si carica quello su cui si vuole saltare

```
switch(a) {
    case 1: <code 1>;
    case 2: <code 2>;
    …
}
```

```
sll x9, x10, 3   # moltiplica per 8
add x9, x9, x11  # x11 ind. base TABLE
ld x12, 0(x9)    # carica indirizzo
jalr x1, x12
```

Procedure e funzioni

L'utilità di un computer sarebbe molto limitata se non si potessero definire/chiamare procedure/funzioni. Una procedura è una scatola nera che esegue un certo task, senza che noi ne sappiamo i dettagli
La cosa importante quando usiamo procedure (o funzioni) è definire chiaramente un protocollo di chiamata:

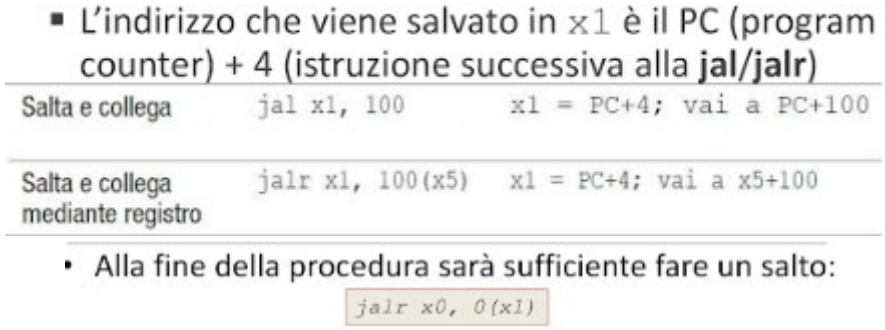
1. Caricare i parametri di input in posti noti a priori
2. Trasferire il controllo alla procedura
    a. Acquisire le risorse necessarie
    b. Eseguire il compito
    c. Caricare i valori di ritorno in posti noti
    d. Restituire il controllo al chiamante
3. Prendere il valore di ritorno della procedura e "cancellare" le tracce (pulire registri ecc.)

L'ISA e la ABI insieme definiscono proprio questi punti.

In RISC-V si cerca di usare i registri laddove è possibile, perchè è più veloce per effettuare il passaggio di parametri

Convenzioni per il RISC-V:

- Registri da $x10$ a $x17$ usati per i parametri in ingresso e per i valori di ritorno($x10$)
- Registro $x1$ (chiamato anche ra, return address) come indirizzo di ritorno (l'indirizzo dell'istruzione successiva a quella che chiama la funzione, viene salvato $PC+4$ al momento della chiamata, perchè le istruzioni sono di 32 *bit* quindi ogni 4 *byte* ho una istruzione)

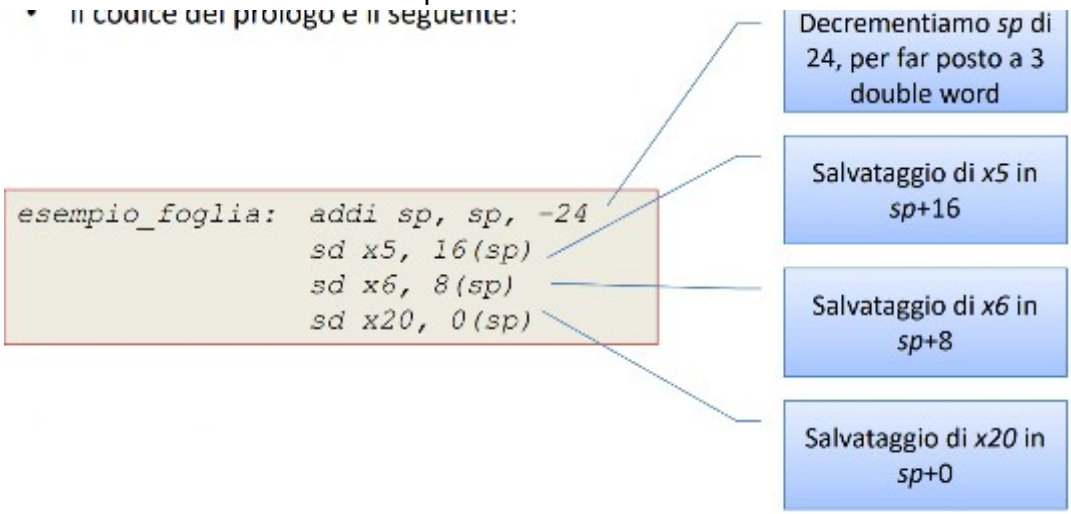


In certi casi i registri non bastano, perché il numero di parametri è maggiore rispetto al numero di registri.
In questo caso si usa uno stack (LIFO). La testa dello stack viene puntata dal registro $x2$, chiamato anche sp (stack pointer).
Questo stack si usa per salvare tutti i valori dei registri che verranno utilizzati nella procedura e, nel caso in cui i registri non bastino per le variabili locali, viene usato anche per salvare le variabili locali. Viene anche usato per salvare i valori precedenti nei registri per poi ripristinarli. Alla fine della procedura si può ripulire lo stack

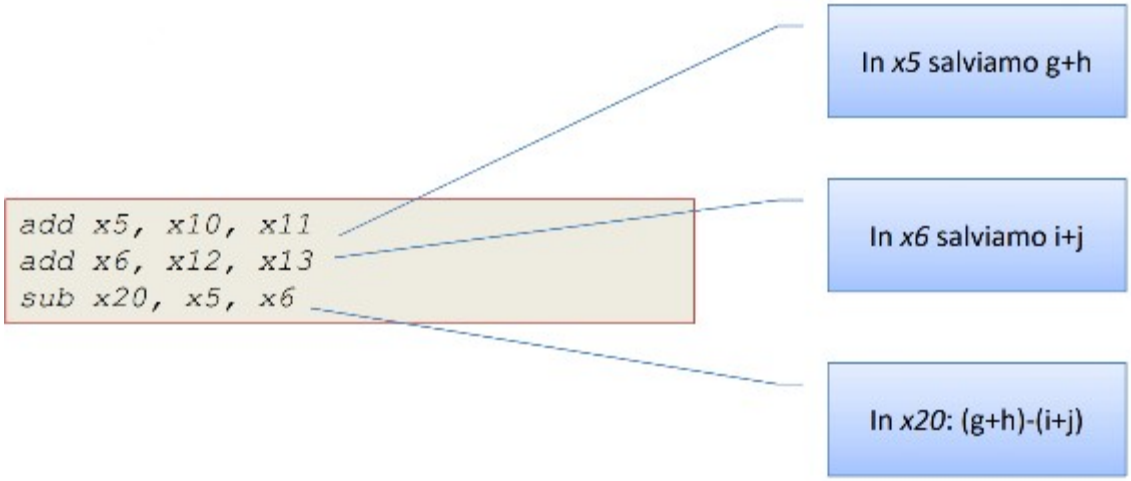
Esempio funzione:
In C++

```
long long int esempio_foglia(long long int g,
    long long int h, long long int i, long long int j) {
    long long int f;
    f = (g+h) - (i+j);
    return f;
}
```

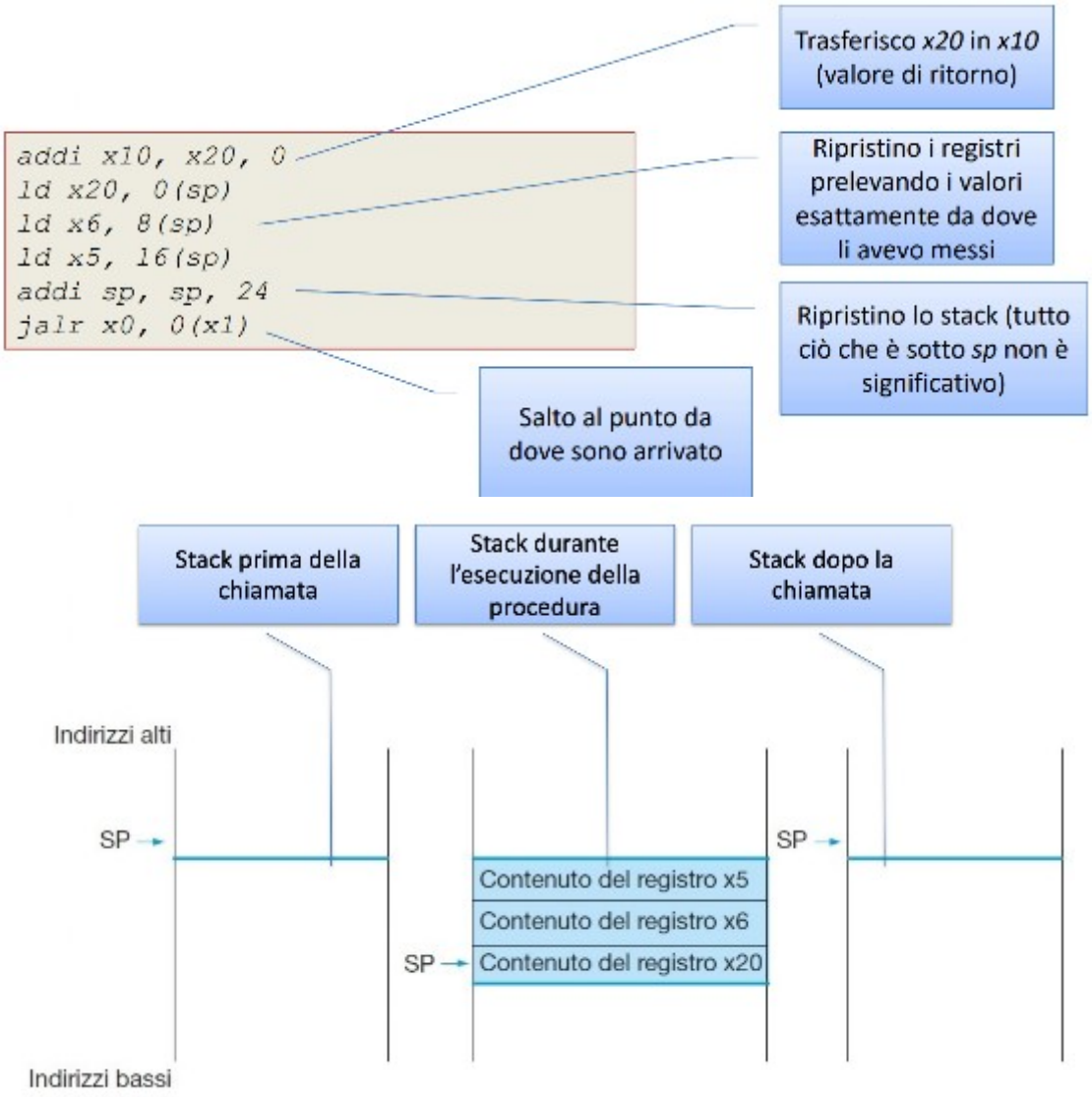
Prologo:
Salvare il valore dei registri che la procedura usa, così da non perdere l'informazione e ripristinare i loro valori una volta che la procedura termina



Esecuzione della procedura:



Epilogo:
Ripristino i valori dei registri e svuoto lo stack



In realtà i registri temporanei $x5$ e $x6$ non devono essere salvati durante la chiamata
Questo perché esistono registri fatti apposta per variabili temporanee (da $x5$ a $x7$ e da $x28$ a $x31$), i quali non conterranno mai informazioni "importanti" da dover salvare.
Invece i registri di cui bisogna salvare il contenuto sono da $x8$ a $x9$ e da $x18$ a $x27$

La situazione si può complicare nel caso in cui ci siano troppe variabili locali oppure procedure annidate, e si risolve usando sempre lo stack e salvandoci sopra i valori del registro di ritorno $x1$
Ad esempio nelle procedure ricorsive si sovrascrive sia $x1$ che i registri per eventuali variabili locali (che avranno lo stesso registro per ogni istanza)

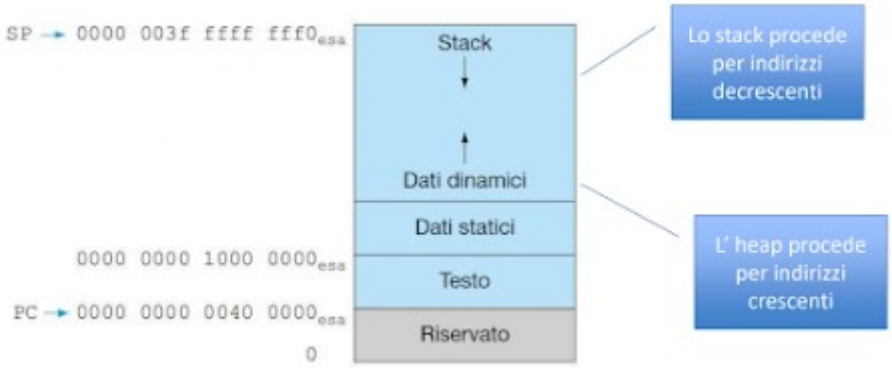
Le variabili statiche e globali vengono salvate in una zona di memoria specifica, questa zona è accessibile tramite il global pointer (gp), contenuto nel registro $x3$

Come abbiamo visto, anche quando le variabili locali sono troppe dobbiamo fare uso dello stack.
Il record di attivazione (o stack frame) è il segmento dello stack che contiene tutti i registri salvati e le variabili locali di una funzione, e si raggiungono tramite un offset a partire da un puntatore.

Il registro sp cambia (cresce verso il basso), per questo motivo RISC-V ci mette a disposizione il fp (frame pointer) in $x8$, che è l'indirizzo più "alto" delle variabili della funzione (il quale rimane costante durante una funzione, a differenza dello sp).
La differenza è che fp è costante per tutta la durata della funzione, e si utilizza con offset negativi, invece sp è variabile e si utilizza con offset positivi



Le variabili dinamiche vengono salvate nella heap.
Schema di memoria RISC-V:

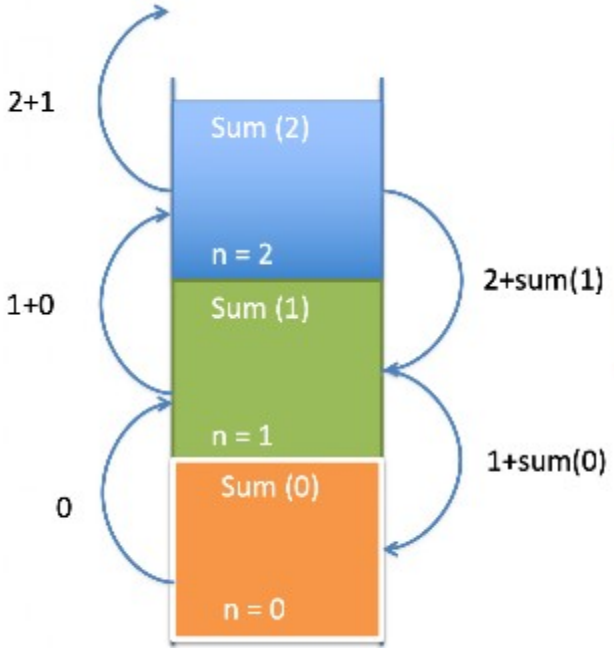


Uso dei registri in RISC-V

Nome	Numero del registro	Utilizzo	Da conservare nella chiamata?
x0	0	Costante 0	n.a.
x1 (ra)	1	Registro di ritorno (registro di collegamento)	sì
x2 (sp)	2	Stack pointer	sì
x3 (gp)	3	Global pointer	sì
X4 (tp)	4	Thread pointer	sì
x5–x7	5-7	Variabili temporanee	no
x8–x9	8-9	Variabili da preservare	sì
x10–x17	10-17	Argomenti/risultati	no
X18–x27	18-27	Variabili da preservare	sì
x28–x31	28-31	Variabili temporanee	no

Anche se le ricorsioni sono eleganti, sono causa di varie inefficienze, questo perché il valore di ritorno viene costruito tornando su per la catena di chiamate:

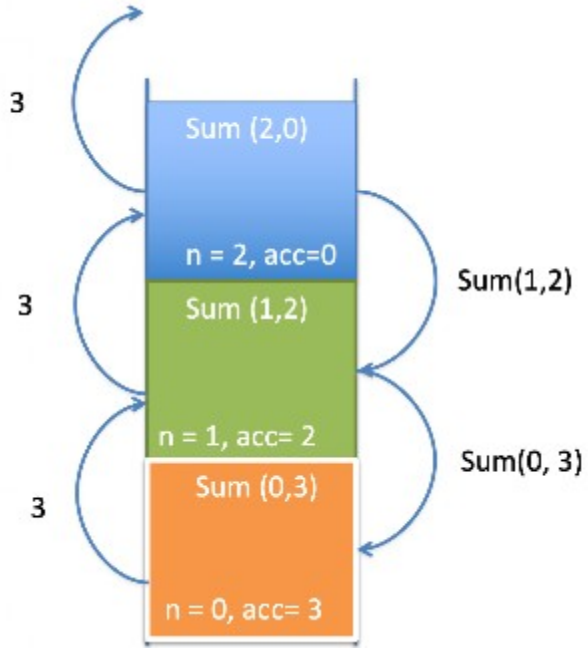
```
long long int sum(long long int n) {
    if (n > 0)
        return sum(n-1) + n;
    else
        return n;
}
```



Il valore di ritorno viene costruito tornando su per la catena di chiamate. Ogni frame di attivazione deve essere preservato per produrre il risultato corretto.

La ricorsione di coda consiste nel poter avere il valore di ritorno già nel momento della condizione di uscita, e evita calcoli nel momento di "risalita":

```
long long int sum(long long int n, long long int acc) {
    if (n > 0)
        return sum(n-1, acc+n);
    else
        return acc;
}
```



In questo caso lungo i rami di ritorno non faccio che restituire 3 (valore di ritorno della chiamata). Conservare i dati nel frame di attivazione è inutile. Posso usare un unico frame di attivazione e svolgere la ricorsione in iterazione.

```
sum:
        add x5, x0, x11      # Sposta x11 in x5
        blt x10, x0, L5      # Se x10 < 0, salta a L5

        addi sp, sp, -8      # Prologo: crea spazio nello stack
        sd x1, 0(sp)         # Prologo: salva x1
        add x11, x11, x10    # Somma x10 ad x11 (acc=acc+n)
        addi x10, x10, -1    # Decrementa x10 (n=n-1)
        jal x1, sum          # Ricorsione (modifica x1)

        ld x1, 0(sp)         # Ripristina x1
        addi sp, sp, 8       # Rimetti a posto lo stack
L5:
        add x10, x0, x5      # sposta x5 in x10 (valore ritorno)
        jalr x0, 0(x1)
```

In assembly questo si rispecchia in quanto non dobbiamo salvare nessuna variabile di quelle delle istanze precedenti, perché il risultato sarà fissato in un registro. Ci sono poi certi compilatori che riconoscono la ricorsione in coda e ritornano direttamente il valore alla fine della "discesa". Infatti nel seguente esempio vediamo quante istruzioni si risparmiano evitando la risalita:

```
sum:
        add x5, x0, x11      # Sposta x11 in x5
        blt x10, x0, L5      # Se x10 < 0, salta a L5

        addi sp, sp, -8      # Prologo: crea spazio nello stack
        sd x1, 0(sp)         # Prologo: salva x1
        add x11, x11, x10    # Somma x10 ad x11 (acc=acc+n)
        addi x10, x10, -1    # Decrementa x10 (n=n-1)
        jal x0, sum          # Ricorsione (NON modifica x1)

        ld x1, 0(sp)         # Ripristina x1
        addi sp, sp, 8       # Rimetti a posto lo stack
L5:
        add x10, x0, x5      # sposta x5 in x10 (valore ritorno)
        jalr x0, 0(x1)
```

Nel caso di funzioni foglia (una funzione che non ne richiama altre) il compilatore, in assenza di ottimizzazione, esegue prologo ed epilogo, in maniera poco furba.
Invece nel caso di un compilatore ottimizzato, se il numero delle variabili è minore o uguale al numero di registri temporanei, usa quei registri senza il bisogno di salvare nessun valore, risparmiando molte istruzioni

Registro	Nome	Uso	Chi salva
x0	zero	Costante 0	N.A.
x1	ra	Indirizzo di ritorno	Chiamante
x2	sp	Stack pointer	Chiamato
x3	gp	Global pointer	---
x4	tp	Thread pointer	---
x5-x7	t0-t2	Temporanei	Chiamante
x8	s0/fp	Salvato/Puntatore a frame	Chiamato
x9	s1	Salvato	Chiamato
x10-x11	a0-a1	Argomenti di funzione/valori restituiti	Chiamante
x12-x17	a2-a7	Argomenti di funzione	Chiamante
x18-x27	s2-s11	Registri salvati	Chiamato
x28-x31	t3-t6	Temporanei	Chiamante

Intel

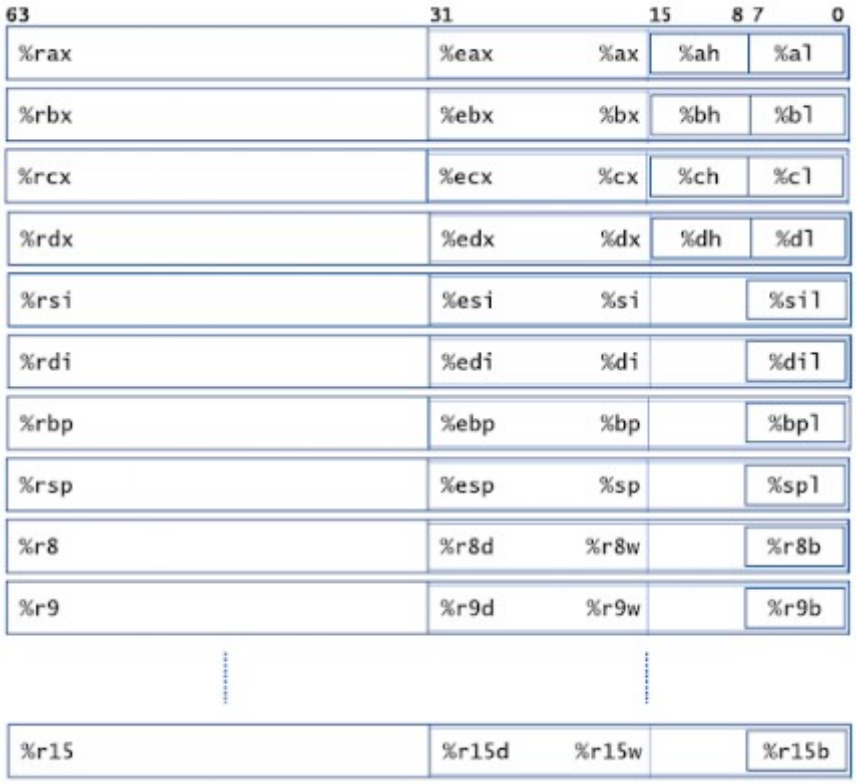
Intel è l'architettura più utilizzata (la maggior parte dei laptop, desktop e server moderni).
È stato costruito tramite un'evoluzione moderna ma costante, ad esempio le moderne CPU a 64 bit sono ancora in grado di eseguire vecchio codice a 8 bit (questo fatto porta molta complessità)
È di tipo CISC (maggior numero di istruzioni e modalità di indirizzamento "avanzate")

Registri

Tutti i registri sono preceduti da %, ne esistono 16 a 64 bit
La caratteristica è che i nomi dei registri rispecchiano il loro "scopo"

- Hanno nomi che riflettono gli strascichi di compatibilità con le versioni precedenti:
  - `%rax`, `%rbx`, `%rcx`, `%rdx`, `%rsi`, `%rdi`, `%rbp`, `%rsp`
  - `%r8`, ..., `%r15`
- Note:
  - `%rsp` → stack pointer;
  - `%rbp` → base pointer (puntatore a stack frame)
  - `%rsi` e `%rdi` derivano da due registri `%si` e `%di` della CPU 8086 per la copia di array (`%si` → source index, `%di` → destination index)

Il primo gruppo di registri (da $\%rax$ a $\%rsp$) estendono dei registri a 32 bit, che si ottengono mettendo e al posto di r ($\%eax$, $\%esp$). Questi registri a 32 bit a loro volta estendono registri a 16 bit ($\%ax$, $\%sp$).
Solamente i primi quattro registri (da a a d) sono composti da 2 registri a 8 bit ($\%ax = \%ah + \%al$, high e low), invece tutti gli altri, compresi i numerici, contengono un unico registro a 8 bit



In intel il registro di instruction pointer (puntatore all'indirizzo della prossima istruzione da eseguire, ovvero pc) è visibile al programmatore ($\%rip$)
Ci sono dei registri speciali, detti registri di flag ($\%rflag$) fatti apposta per le istruzioni di salto condizionale

- Set di flag settati da istruzioni logico aritmetiche
  - `CF` : Carry Flag → Messo a 1 se risultato è andato in unsigned overflow o se c'è carry-out
  - `ZF` : Zero Flag → Messo a 1 se risultato è zero
  - `SF` : Sign Flag → Messo a 1 se risultato è negativo
  - `OF` : 2's Overflow Flag → Messo a 1 se risultato è andato in overflow

Istruzioni

Tutte le convenzioni di chiamata(come/dove passare i parametri, quali registri preservare) non fanno propriamente parte dell'architettura, infatti data una CPU si possono usare diverse convenzioni di chiamata, le quali sono definite dalla ABI (e non dalla ISA).

I primi 6 parametri di una funzione vengono passati tramite i registri, dal settimo in poi vengono salvati nello stack.

- Primi 6 argomenti:
  - `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` ed `%r9`
- Altri argomenti ($7 \rightarrow n$): sullo stack
- Valori di ritorno:
  - `%rax` e `%rdx`
- Registri preservati:
  - `%rbp`, `%rbx`, `%r12`, `%r13`, `%r14` ed `%r15`
- Registri non preservati:
  - `%rax`, `%r10`, `%r11`, oltre ai registri per passaggio parametri: `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` ed `%r9`

Le istruzioni Intel prevalentemente sono a 2 operandi, e il risultato viene salvato nel secondo (limitato rispetto a RISC-V che permette un terzo operando per la destinazione).
Nel primo operando (sorgente) si possono inserire costanti (valori immediati che iniziano con $), operandi in registri o operandi in memoria.
Nel secondo operando (destinazione) invece non si possono inserire valori costanti, appunto perchè non possono "salvare" il risultato dell'operazione
In Intel le uniche limitazioni sulle operazioni è che non ci possono essere entrambi gli operandi in memoria. (In RISC-V invece l'accesso in memoria era concesso con due sole operazioni).

La sintassi per accedere ad un operando in memoria è la seguente:

- Sintassi:`<displ> (<base reg>, <index reg>, <scale>)`
  - `<displacement>`: costante (valore immediato) a 8, 16 o 32 bit
    - simile a RISC-V - ma RISC-V ha displacement/offset solo a 12 bit
  - `<base>`: valore in registro (come per RISC-V)
  - `<indice>`: valore in registro (semplifica iterazione su array)
  - `<scala>`: valore costante: 1, 2, 4, o 8 (semplifica accesso ad array con elementi di dimensione 1, 2, 4 o 8 byte)
- Indirizzo `<displacement> + <base> + <indice> * <scala>`

Si possono omettere alcuni campi che verranno inizializzati di default (scala=1 e il resto=0, però la base ci deve essere sempre).

Nome	Forma	Esempio	Descrizione
Immediata	`$Num`	**movl** $–500, %rax	`rax = $Num`
Accesso diretto	`Num`	**movl** 500, %rax	`rax = Mem[Num]`
Registro	r_i	**movl** %rdx, %rax	`rax = rdx`
Accesso indiretto	(r_i)	**movl** (%rdx), %rax	`rax = Mem[rdx]`
Base e spiazzamento	$Num(r_i)$	**movl** 31(%rdx), %rax	`rax = Mem[rdx+31]`
Indice scalato	(r_b, r_i, s)	**movl** (%rdx, %rcx, 4), %rax	`rax = Mem[rdx+rcx*4]`
Indice scalato + spiazzamento	$Num(r_b, r_i, s)$	**movl** 35(%rdx, %rcx, 4), %rax	`rax = Mem[rdx+rcx*4+35]`

s è il fattore di scala e può assumere i valori: 1, 2, 4, o 8

La sintassi delle operazioni Intel in genere è:

$$<opcode> \ <source>, \ <destination>$$

Di solito il carattere finale, seguito dal nome dell'operazione, in $<opcode>$ rappresenta la "larghezza" in bit degli operandi:

- byte: 8 bit
- word: 16 bit (in assembly le word sono lunghe 32 bit lo so…)
- long word (double word): 32 bit
- q (quad word): 64 bit

I nomi dei registri iniziano con $\%$, i valori costanti iniziano con $\$$, gli indirizzi costanti (diretti) sono semplici numeri, invece se si vuole accedere alla memoria tramite un indirizzo contenuto in un registro bisogna usare le tonde $(\%x\ldots)$, altrimenti il valore viene trattato come una costante

- `mov`: copia dati da sorgente a destinazione
  - `movsx`: copia dati con estensione del segno
  - `movzx`: copia dati con estesione con `0`
- `add` / `adc` (add with carry)
- `sub` / `sbc` (sub with carry)
- `mul` / `imul`: signed / unsigned multiplication
- `div` / `idiv`: signed / unsigned division
- `inc` / `dec`: somma / sottrae 1
- `rcl`, `rcr`, `rol`, `ror`: varie forme di "rotate"
- `sal`, `sar`, `shl`, `shr`: shift (aritmetico e logico)
- `and` / `or` / `xor` / `not`: operazioni booleane bit a bit
  - Istruzioni aritmetico / logiche modificano flag (carry, zero, sign, ...)
  - Altre istruzioni per modificare flag: `clc` / `stc`, `cld`, `cmc`...
- `neg`: complemento a 2 (negazione)
- `nop`
- `push`: inserisce dati sullo stack
  - Sintassi: **pushq** %reg
    - Decrementa `%rsp` di 8
    - Scrive `%reg` nella memoria specificata in `%rsp`
  - Esempio: **pushq** %rax
    - Equivale a: **subq** $8, %rsp
      movq %rax, (%rsp)
- `pop`: rimuove dati da stack
  - Sintassi: **popq** %reg
    - Legge memoria all'indirizzo specificata in `%rsp` e memorizza valore in `%reg`
    - Incrementa `%rsp` di 8
  - Esempio: **popq** %rdx
    - Equivale a: **movq** (%rsp), %rdx
      addq $8, %rsp

- `cmp` e `test`
  - `cmp`: **cmp** arg1, arg2
    - Compara arg2 con arg1 (e.g. arg2 < arg1, arg2 = arg1, ...)
    - Effettua arg2 – arg1 e setta flags del flag register
    - arg1 e arg2 non sono modificati (risultato di sottrazione non memorizzato)
  - `test`: **test** arg1, arg2
    - Compara arg2 con arg1 (e.g. arg2 = arg1, ...)
    - Effettua arg2 & arg1 e setta flags del flag register
    - arg1 e arg2 non sono modificati (risultato di and bit a bit non memorizzato)
    - Spesso usato con arg1 = arg2 (e.g. **test** %eax, %eax per verificare se il valore è 0 o negativo (`ZF` o `SF`).
  - Da usarsi prima di salti condizionali

- `jmp` : salto incondizionato
- `je` / `jnz` / `jc` / `jnc`: salti condizionati
  - La condizione di salto è stabilita in base ai valori nel flag register (jump if equal, jump if not zero, jump if carry, ...)
  - In generale, "`j<condition>`"

Istruzione	Sinonimo	Cond. Flags	Descrizione
je label	**jz** label	`ZF`	Uguali o zero
jne label	**jnz** label	`~ZF`	Differenti o Non zero
js label		`SF`	Negativo
jns label		`~SF`	Non Negativo
jg label	**jnle** label	`~(SF^OF)&~ZF`	Signed >
jge label	**jnl** label	`~(SF^OF)`	Signed >=
jl label	**jnge** label	`(SF^OF)`	Signed <
jle label	**jng** label	`(SF^OF)|ZF`	Signed <=
ja label	**jnbe** label	`~CF&~ZF`	Unsigned >
jae label	**jnb** label	`~CF`	Unsigned >=
jb label	**jnae** label	`CF`	Unsigned <
jbe label	**jna** label	`CF|ZF`	Unsigned <=

- Varie istruzioni "condizionali" (conditional move, set on condition, ...)

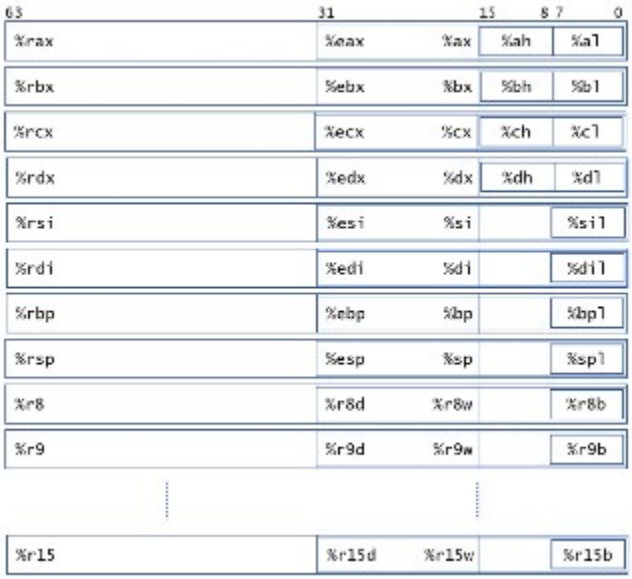
- `call` chiamata a procedura
  - Sintassi: **call** label
    - Fa push sullo stack dell'indirizzo della prossima istruzione
    - Modifica il program counter per andare all'inizio della procedura desiderata (specificato con una label)
    - Implicitamente esegue: **subq** $8, %rsp
      movq %rip, (%rsp)
- `ret`: ritorno da procedura
  - Sintassi: **ret**
    - Fa pop dallo stack dell'indirizzo di ritorno e lo memorizza in `%rip`
    - Modifica il program counter per andare alla prossima istruzione del chiamante
    - Implicitamente esegue: **movq** (%rsp), %rip
      addq $8, %rsp
- `lea`: `l`oad `e`ffective `a`ddress
- Sintassi: **lea** src, dest
  - Istruzione "strana", ma talvolta molto utile!
  - Nata per calcolare indirizzi (con indirizzamento indiretto) senza fare accessi
    - Usa l'hardware del calcolo di indirizzamento per "normali" operazioni aritmetiche
- Copia indirizzo di sorgente (calcolato tramite displacement, base, indice e scala) in registro destinazione
  - Calcola l'indirizzo e lo memorizza nel registro di destinazione senza "caricare" nulla dalla memoria.
  - Esempio:
    - **lea** 80(%rdx, %rcx, 2), %rax → `%rax = %rdx + 2*%rcx +80`
  - Viene spesso utilizzata come istruzione aritmetica che effettua contemporaneamente due somme (un valore immediato e due registri) shiftando uno degli addendi.

L'istruzione *lea* è utile nel caso in cui si voglia fare la somma di due indirizzi senza sovrascrivere nessuno dei due:

- Su intel come possiamo sommare `%rbx` e `%rcx` salvando il risultato in `%rax`?
  - Non è possibile specificare un registro destinazione per `add` diverso dal secondo operando
  - Ma se condiseriamo un accesso a memoria con base `%rbx` ed indice `%rcx`...
  - ...l'indirizzo acceduto sarebbe `%rbx + %rcx`
  - **lea** (%rbx, %rcx), %rax
- L'istruzione `lea` è spesso utilizzata per fare più somme (con eventuali shift) salvando il risultato in un registro diverso

Le istruzioni *inc* e *dec* possono sembrare apparentemente inutili, perchè basterebbe fare una *add* di $1 / -1$, in realtà, essendo le codifiche Intel di un numero variabile di bit, permette di risparmiare un sacco di bit rispetto alla *add*

- %rsp → stack pointer;
- %rbp → base pointer
- Primi 6 argomenti:
  - %rdi, %rsi, %rdx, %rcx, %r8 ed %r9
- Altri argomenti (7 → n): sullo stack
- Valori di ritorno:
  - %rax e %rdx
- Registri preservati:
  - %rbp, %rbx, %r12, %r13, %r14 ed %r15
- Registri non preservati:
  - %rax, %r10, %r11
  - registri per passaggio parametri: %rdi, %rsi, %rdx, %rcx, %r8 ed %r9



Guarda esempi codice sulla slide di Intel

ARM

Utilizzato sulla maggior parte di tablet, smartphone e sistemi embedded
ARM= Acorn Risc Machine da una università.
Poi venne preso in mano da un'azienda e diventò Advanced Risc Machine

Evoluzione meno "complicata", nato subito a 32 bit, recentemente nuova architettura a 64 bit ma non è completamente compatibile con quella a 32 bit.

È un RISC pragmatico, nel senso che ha cercato di prendere il meglio da RISC e Intel

Modalità di indirizzamento meno complesse di Intel ma lo stesso molto potenti.
Obiettivo: risparmio energetico, risolvere i problemi classici del RISC

Registri

Solo 16 registri a 32 bit general purpose (da $r0$ a $r14$, $r15$ in realtà non è general purpose)
Alcuni hanno un nome simbolico:

- $r13 = sp$ (stack pointer)
- $r14 = lr$ (link register, ovvero return address)

- **Nomi: da r0 a r15**
  - Tecnicamente, r15 non è un registro general purpose, spesso usato come PC
- **Alcuni registri accessibili tramite un nome simbolico che ne identifica l'utilizzo**
  - r13 == sp (stack pointer)
  - r14 == lr (link register)
- **Primi 4 argomenti:**
  - r0 ... r3
  - registri non preservati! Utilizzabili anche come registri "temporanei" da non salvare!
- **Altri argomenti (4 → n): sullo stack**
- **Registri preservati: r4 ... r11**
  - Eccezione: in alcuni ABI r9 non è preservato
- **Valori di ritorno: r0 e r1**
- **I registri che una subroutine può utilizzare senza doverli salvare sono**
  - r0, r1, r2, r3 ed r12
  - più eventualmente r9 (dipende da piattaforma / ABI)

$r15$ contiene program counter e vari flags (distinzione chiara per flags per esecuzione condizionale $apsr$ e altri flags)

Flag Register

- `eq` (equal): esegui se la prec. op. era fra numeri uguali (flag `Z` vale `1`);
- `ne` (not equal): esegui se la prec. op. era fra numeri diversi (flag `Z` vale `0`);
- `hs` (higher or same) o `cs` -carry set- (flag `C` vale `1`);
- `lo` (lower) o `cc` -carry clear- (flag `C` vale `0`);
- `mi` (minus): esegui se risultato ultima op. è negativo (flag `N` vale `1`);
- `pl` (plus): esegui se risultato ultima op. è positivo (flag `N` vale `0`);
- `vs` (overflow): esegui se ultima op. è risultata in un overflow (flag `V` vale `0`);
- `vc` (overflow clear): opposto di `vs` (esegui se il flag `V` vale `0`);
- `hi` (higher): esegui se in un'op. di confronto primo operando è maggiore del secondo, assumendo unsigned (flag `C` vale `1` e `Z` vale `0`);
- `ls` (lower or same): esegui se in un op. confronto primo operando è minore o uguale al secondo, assumendo unsigned (flag `C` vale `0` o `Z` vale `1`);
- `ge` (greater or equal): esegui se `N` vale `1` e `V` vale `1`, o se `N` vale `0` e `V` vale `0`;
- `lt` (less than): esegui se `N` vale `1` e `V` vale `0`, o se `N` vale `0` e `V` vale `1`;
- `gt` (greater than): come `ge`, ma con `Z` che vale `0`;
- `le` (less or equal): come `lt`, ma esegue anche se `Z` vale `1`.

Le convenzioni di chiamata sono definite dall'ABI (e ne esistono varie, anche molto diverse tra loro)
In ARM solo 4 registri per parametri di una funzione (da $r0$ a $r3$) che non devono essere preservati, tutti gli altri argomenti vanno nello stack
I registri preservati sono da $r4$ a $r11$
I valori di ritorno in $r0$ e $r1$
Quindi non devono essere preservati $r0, r1, r2, r3, r12$ e in alcune ABI anche $r9$

Istruzioni

Istruzioni prevalentemente a 3 argomenti (ma esistono anche sono istruzioni a 2 argomenti, come *mov*).
Non si possono avere operandi in memoria:

- Operando sinistro: registro
- Operando destro: immediato, o registro (eventualmente shiftato)

L'operando destro, infatti, è più "potente" grazie allo shift
Anche in ARM ci sono solo due operazioni per accedere in memoria, che però si possono fare sia su un singolo registro sia su registri multipli

- Uniche operazioni che accedono alla memoria: `load` e `store`
  - Load register / Store register (`ldr` / `str`)
  - Load / Store multiple (`ldm` / `stm`)

Ci sono varie modalità di indirizzamento.

Si parte da una base che è l'indirizzo di partenza, a cui può essere sommato o un valore immediato o un indice contenuto in un registro (eventualmente shiftato). Inoltre si può decidere di aggiornare l'indirizzo base.

- Operandi in memoria: varie modalità di indirizzamento
- Semplificando un po':

  Base + Offset (immediato)
  Base + Indice (eventualmente scalato)

  - In più, possibilità di aggiornare il registro base (`[!]`)
    - Aggiornamento prima dell'accesso (pre indexed) o dopo l'accesso (post indexed)
- `i = <base> + {<offset> | <indice (shiftato)> }`
  - `<base>`: valore in registro (come per RISC-V)
  - `<offset>`: costante (valore immediato) codificato su 12 bit
  - `<indice>`: valore in registro (semplifica iterazione su array)
    - Opzionalmente shiftato o ruotato
- Pre Indexed:
  - `[rb, #i]` *`[!]`*
  - `[rb,{+|-}ro` *`[,<shift>]`* `]` *`[!]`*
- Post Indexed:
  - `[rb],#i`
  - `[rb],{+|-}ro` *`[,<shift>]`*

Nella modalità post indexed il registro base viene aggiornato sempre (sennò non avrebbe senso come modalità), invece nella pre indexed si può scegliere aggiungendo *!*
Se l'offset (o shift) è zero, può essere omesso

- Pre Indexed con registro indice: `[rb,{+|-}ro,<shift>]` *`[!]`*
  - `<shift>` può indicare shift (aritmetici o logici) o rotazioni sul registro `ro`
  - Se non ci sono shift/rotazioni, `<shift>` può essere omesso
    - `[rb,{+|-}ro]` *`[!]`*
- Pre Indexed con offset: `[rb, #i]` *`[!]`*
  - Offset $= 0 \Rightarrow$ può essere omesso
    - `[rb]` *`[!]`*
- Post indexed con indice: `[rb],{+|-}ro,<shift>`
  - No shift $\Rightarrow$ si omette: `[rb],{+|-}ro`
- Post indexed con offset: `[rb],#i`
  - Offset $= 0 \Rightarrow$ si omette: `[rb]`
  - Come pre indexed con offset 0 e senza update!!!

Indirizzamento: Esempi esplicativi

- Accesso a memoria tramite registro e offset
  - **ldr** r0, [r1] `@ r0 = mem[r1]`
- Tre modi di specificare offset:
  - Costante:
    ldr r0, [r1, #4] `@ r0 = mem[r1 + 4]`
  - Registro:
    ldr r0, [r1, r2] `@ r0 = mem[r1 + r2]`
  - Scalato:
    ldr r0, [r1, + r2, lsl #8] `@ r0 = mem[r1 + r2 << 8]`
    ldr r0, [r1, – r2, lsl #8] `@ r0 = mem[r1 - r2 << 8]`

In sintesi le modalità di indirizzamento ARM sono più potenti rispetto a RISC-V (registro indice (eventualmente scalato), aggiornamento automatico del registro base) e un po' meno potenti di Intel (+aggiornamento automatico (utile per scorrere array), -possibilità di usare contemporaneamente offset e indice)

Tutte le istruzioni permettono un esecuzione condizionale, cioè che vengono eseguite solo in base a certi flag, aggiungendo all'istruzione il suffisso:

- *eq* (equal)
- *ne* (not equal)
- *hs* (higher or same)
- *lo* (lower)
- *mi* (minus)
- …

I flag (contenuti in *apsr*) vengono aggiornati da istruzioni specifiche (come compare *cmp*), oppure con suffisso opzionale (*s*) in operazioni aritmetiche
Le costanti in ARM iniziano con #

Istruzioni Aritmetiche e logiche

- `<opcode>[<cond>][s] rd, rl, <r>`
  `<r> := #const | <reg> [, <sor> #const | <reg>]`
  - Come RISC-V, registro destinazione e 2 operandi
  - No argomenti in memoria
- Perché "`<r>`"?
  - Secondo argomento può essere immediato o registro
    - differenza da RISC-V: `add` invece che `add` ed `addi`
  - Se registro, può essere shiftato o rotato
    - `lsl` (`asl`), `lsr`, `asr`, `ror`, `rrx`

Le istruzioni più comuni sono *add* (*adc* con carry), sub (*sbc*) e *rsb* (*rsc*) che è la reverse sub.
La reverse sub è utile perchè appunto il secondo operando può essere shiftato.
Altre operazioni sono quelle booleane and, *orr*, *eor* (xor) e *bic* (and not)

Istruzioni Più Comuni - 3

- `mul`, `mla` & friends: varie forme di moltiplicazione

```
mul r0, r1, r2          @ r0 = r1 * r2
mla r0, r1, r2, r3      @ r0 = r1 * r2 + r3
```

- Istruzioni di shift e rotazione: tramite manipolazione del terzo argomento

```
add r0, r1, r2, lsl #4      @ r0 = r1 + r2 << 4
add r0, r1, r2, lsl r4      @ r0 = r1 + r2 << r4
```

- Nota: molti ARM core **NON** forniscono una istruzione di divisione tra interi
  - L'operazione di divisione è fornita da codice:

```
int divide(int A, int B) {
  int Q = 0; int R = A;
  while( R >= B ) {
    Q = Q + 1;
    R = R - B;
  }
  return Q;
}
```

- Movimenti di registri:
  - `mov`

```
mov r0, r1                    @ r0 = r1
mov r0, #21                   @ r0 = 21
```

  - `mvn`: move not. Muove il complemento ad `1` di un registro

```
mvn r0, r1                    @ r0 = not r1
```

- `b`: branch (anche salto condizionale, tramite suffisso...)

Branch incondizionato:

```
        b label
        ...
label:  ...
```

Branch condizionato:

```
        mov r0, #0
loop:
        ...
        add r0, r0, #1
        cmp r0, #10
        bne loop
```

- `bl`: branch and `l`ink (per invocazione di subroutine)
  - Salva l'indirizzo di ritorno nel `l`ink `r`egister `r14`

```
        bl sub      ; chiama sub
        cmp r1, #5  ; ritorna qui
        moveq r1, #0
        ...
sub:                ; codice funzione sub
        ...
        mov pc, lr  ; return
```

- `bx`: solo su alcune CPU, equivalente a `mov r15, r`

- `cmp`: setta flags come sub, ma senza risultato!
  - Da usarsi prima di salti condizionali

```
if ((r0 == r1) && (r2 == r3)) r4++;
```

```
; Encoding semplice          ; Encoding piu' piccolo e veloce
cmp r0, r1                   cmp r0, r1
bne skip                     cmpeq r2, r3
cmp r2, r3                   addeq r4, r4, #1
bne skip                     ...
add r4, r4, #1
skip: ...
```

- Altre istruzioni simili: `tst` (esegue and), `teq` (xor), `cmn` (somma), ...

```
tst r0, r1          @ set cc a r0 and r1
teq r0, r1          @ set cc a r0 xor r1
cmn r0, r1          @ set cc a r0 + r1
```

- `ldr` / `str`: load / store register

```
ldr r0, [r1]        @ r0 = mem[r1]
str r0, [r1]        @ mem[r1] = r0
```

  - Valgono le regole per indirizzamento discusse in precedenza
  - Esistono versioni per 32, 16, 8 bit

```
ldr r0, [r1] ; 32 bit   ldrh r0, [r1] ; 16 bit   ldrb r0, [r1] ; 8 bit
str r0, [r1] ; 32 bit   strh r0, [r1] ; 16 bit   strb r0, [r1] ; 8 bit
```

- `ldm` / `stm`: load / store multiple registers

```
ldm r0, {r1, r2, r3}        @ r1 = mem[r0]
                            @ r2 = mem[r0+4]
                            @ r3 = mem[r0+8]

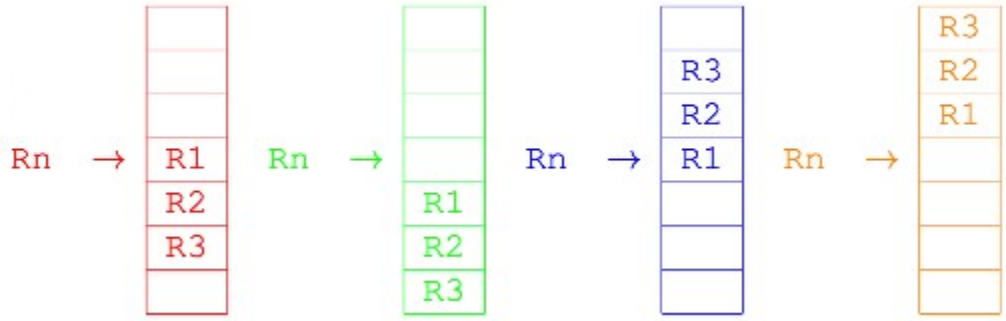
stm r0, {r1, r2, r3}        @ mem[r0] = r1
                            @ mem[r0+4] = r2
                            @ mem[r0+8] = r3
```

  - Consentono di trasferire grandi quantità di dati in modo efficiente
  - Utilizzate in prologo/epilogo di funzioni per salvare/ripristinare i registri e l'indirizzo di ritorno

- `ldm<mode> r [!], <register list>` / `stm<mode> r, <register list>`
- `<mode>` può essere
  - `ia` (increment after): registri salvati / recuperati nelle locazioni `r`, `r+4`, `r+8`, etc...
  - `ib` (increment before): registri salvati / recuperati nelle locazioni `r+4`, `r+8`, `r+12`, etc...
  - `da` (decrement after): registri salvati / recuperati nelle locazioni `r`, `r−4`, `r−8`, etc...
  - `db` (decrement before): registri salvati / recuperati nelle locazioni `r−4`, `r−8`, `r−12`, etc...
- Se `!` è specificato, `r` viene aggiornato di conseguenza
  - Nomi alternativi per suffissi: `ea` (empty ascending), `ed` (empty descending), `fa` (full ascending), `fd` (full descending)

```
LDM<mode> [!] Rn, { <registers> }
IA: addr := Rn
IB: addr := Rn
DA: addr := Rn
DB: addr := Rn
foreach Ri in sorted(<registers>)
      IB: addr := addr + 4
      DB: addr := addr - 4
          Ri := Mem[addr]
      IA: addr := addr + 4
      DA: addr := addr - 4
<!>: Rn := addr
```

IA : Increment After
IB : Increment Before
DA : Decrement After
DB : Decrement Before



In ARM per accedere a sotto registri più piccoli, basta usare dei suffissi b e h

Toolchain

Il calcolatore capisce solo il linguaggio macchina, ovvero sequenze di 0 e di 1.

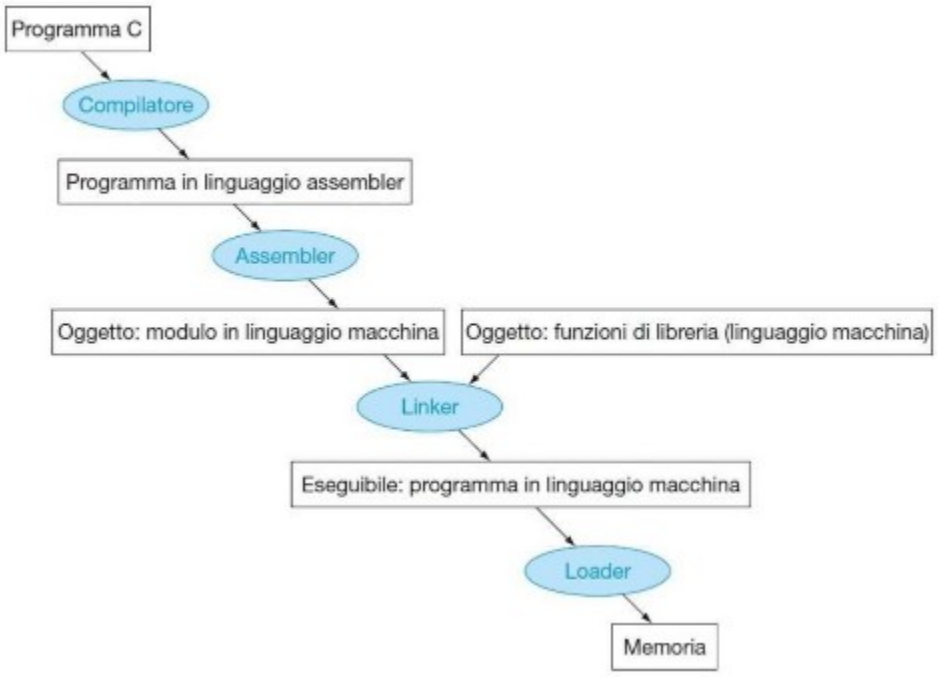
Per questo utilizziamo Assembly, ma è comunque troppo complesso.
In genere si programma in un linguaggio ad alto livello, e successivamente viene generato l'Assembly partendo dal programma ad alto livello. La conversione viene fatta dal compilatore

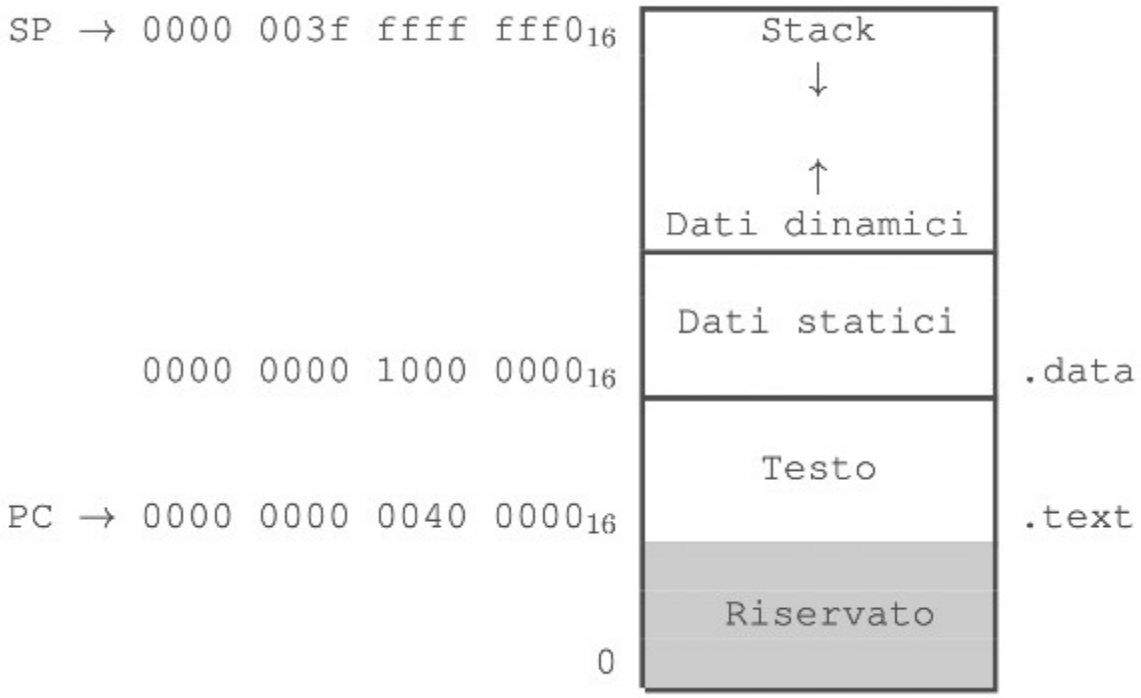
I passi che in genere vengono svolti per ottenere un file eseguibile sono i seguenti:

1. Preprocessore
   Generalmente sostituzione di codice dove presenti macroistruzioni
2. Compilatore
   Prende un file scritto in C e genera un file scritto in $Assembly$ ($.s$)
3. Assembler
   Prende un file scritto in $Assembly$ e genera un file oggetto, in linguaggio macchina ($.o$)
4. Linker
   Prende un file scritto in linguaggio macchina e lo associa a librerie per generare un file eseguibile, sempre in linguaggio macchina ($.exe$ / $.out$)

Di solito un driver si occupa di svolgere tutte queste operazioni

- Preprocessore: poco interessante per noi, ignoriamolo





GCC

gcc (gnu compiler collection) è un compilatore che supporta vari linguaggi ad alto livello e genera linguaggio macchina per varie architetture di CPU (RISC-V compreso)
Per quanto riguarda la creazione di un eseguibile partendo da un file C, gcc mette a disposizione i seguenti sottoprogrammi:

1. Preprocessore
   cpp (che sta per "**C p**re**p**rocessor")
2. Compilatore
   cc (che sta per "**C c**ompiler")
3. Assembler
   as (che sta per "**As**sembly compiler" o per "**As**sembler")
4. Linker
   ld (che sta per "**l**ink e**d**itor" o per "**l**oa**d**er")

gcc solitamente chiama tutti questi comandi, però si può scegliere che livelli "compiere":

- Default: invoca tutti i programmi necessari
- `gcc -S` si ferma dopo aver invocato `cc` (genera file assembly `.s`)
- `gcc -c` si ferma dopo aver invocato `as` (genera file oggetto `.o`)

Compilatore

Partendo da un file in C $file.c$ si può chiamare il comando gcc $-S$ per invocare cc e generare un file in Assembly $file.s$
cc conosce l'architettura target (l'architettura di destinazione su cui eseguire il programma) meglio di un umano, infatti attua tutte le ottimizzazioni (esistono vari livelli di ottimizzazione)
L'assembly generato potrebbe non essere di facile lettura, appunto perché le ottimizzazioni potrebbero riordinare il codice e cambiare delle cose.

Assembler

Partendo da un file in C $file.c$ o da un file in Assembly $file.s$ si può chiamare il comando gcc $-c$ per invocare cc e as (oppure solo as) e generare un file oggetto $file.o$
as solitamente fa sempre qualcosa di più rispetto al sostituire semplicemente i codici mnemonici con sequenze di bit:

- Converte pseudo-istruzioni in istruzioni
  - ```
    mv x10, x11 // x10 assume valore di x11
                  ↓
    addi x10, x11, 0 // x10 riceve il contenuto di x11 + 0
    ```
  - ```
    li x9, 123 // carica il valore 123 in x9
                 ↓
    addi x9, x0, 123 // x9 assume il valore x0 + 123
    ```
  - ```
    j LABEL // Jump non condizionato a LABEL
              ↓
    jal x0, LABEL
    ```
- Converte numeri da esadecimale/decimale in binario
- Gestisce le label
- Gestisce i salti
- Genera metadati

Il file oggetto generato è composto da i seguenti **segmenti** distinti:

- Header
  Specifica dimensione e posizione degli altri segmenti del file oggetto
- Segmenti
  - Segmento di testo
    Codice in linguaggio macchina (sequenze di bit)
  - Segmento dati
    Contiene tutti i dati, sia statici che dinamici, allocati per la durata del programma
- Tabella dei simboli
  Associa simboli ad indirizzi (**relativi**)
  Enumera i simboli non definiti (cioè definiti in altri moduli/file)
- Tabella di rilocazione
  Enumera tutte le istruzioni che fanno riferimento ad altre istruzioni o dati che dipendono da indirizzi assoluti, che quindi saranno da **patchare** (sistemare) quando il programma verrà caricato in memoria
    ```

- Altre informazioni
  Debugging, etc.

Linker

Partendo da un file in C $file.c$, da un file in Assembly $file.s$ o un file oggetto $file.o$ si può chiamare il comando gcc, senza opzioni, per invocare ld e generare un eseguibile $file.exe$ / $file.out$
ld mette assieme uno o più file oggetto, eseguendo le necessarie rilocazioni:

- Decide come codice e dati sono disposti in memoria
- Associa indirizzi assoluti a tutti i simboli
- Risolve simboli che erano lasciati indefiniti
- Patcha le istruzioni contenute nella tabella di rilocazione, inserendo gli indirizzi assoluti

Lo scopo di ld quindi è eliminare le tabelle dei simboli e di rilocazione, generando codice macchina con i giusti riferimenti, mettendo assieme più file oggetto.

Il linker gestisce vari tipi di simboli:

- Simboli definiti
  I simboli che sono associati ad un indirizzo relativo nella tabella dei simboli
- Simboli non definiti
  I simboli che non sono associati ad un indirizzo relativo, perchè sono definiti in altri file
- Simboli locali
  I simboli definiti, che però non possono essere usati in altri file

In ogni caso associa ad **ogni** simbolo un indirizzo **assoluto**, per i simboli non definiti cerca in altri file, se non vengono trovati viene dato un errore di linking
L'operazione di linking quindi si occupa di:

1. Disporre in memoria i vari segmenti ($.text$, $.data$, etc) dei vari file $.o$
   I segmenti testo vengono messi uno dopo l'altro, idem per i data e così via
2. In base alla posizione dei vari segmenti (passo precedente) assegnare un indirizzo assoluto ad ogni simbolo contenuto nelle varie tabelle dei simboli
3. In base alle tabelle di rilocazione, correggere le varie istruzioni con gli indirizzi assoluti appena calcolati

Il risultato viene poi "incapsulato" in un file eseguibile (segmenti, informazioni per il caricamento in memoria etc)

Librerie

Esistono funzioni "predefinite" fornite dal compilatore/sistema, esse sono definite in dei file $.o$ che vengono sempre inclusi in ogni operazione di linking.
Però linkare tanti file $.o$ è poco pratico e poco efficiente.
Infatti di solito si riordinano questi file all'interno di una **libreria**, e al posto di linkare una grande quantità di file oggetto, si linka un'unica libreria:

- statiche ($.a$)
  Semplici collezioni di file oggetto, viene fatto tutto da ld
  ld inserisce nell'eseguibile tutto il codice della libreria utilizzato dal programma, di

conseguenza le dimensioni dell'eseguibile aumentano però il caricamento del programma da parte del SO risulta semplice

- dinamiche (*.so*)
  ld non fa molto, perchè il vero linking avviene a tempo di esecuzione
  ld inserisce solamente dei riferimenti alle librerie usate e alle funzioni invocate.
  Ogni eseguibile contiene un riferimento ad un linker dinamico, il quale si occuperà di fare il linking. Di conseguenza il caricamento del programma da parte del SO risulta complesso e il programma non è autocontenuto.
  I vantaggi però sono che l'eseguibile non avrà grandi dimensioni e poi si possono modificare le librerie anche dopo aver creato l'eseguibile

Esistono delle operazioni che fanno si che la seconda volta che viene chiamata una procedura il processo sia più veloce, questo perchè la procedura è già stata caricata.

- Calcolo valore per `jal`:
  - le istruzioni utilizzano indirizzo relativo al `PC`, basta fare differenza tra indirizzo della `jal` e indirizzo della procedura:
    - il campo indirizzo di `jal` a 400004_{16} che salta a 400100_{16} (indirizzo procedura `func_2`), conterrà $400100_{16} - 400004_{16} = 252_{10}$
    - il campo indirizzo di `jal` a 400104_{16} che salta a 400000_{16} (indirizzo procedura `func_1`), conterrà $400000_{16} - 400104_{16} = -260_{10}$

- Calcolo offset per `ld/sd`
  - Sono più complessi da calcolare perchè dipendono da indirizzo base (`x3`, per semplicità assumiamo `x3` = 0000000010000000_{16}):
    - inseriamo 0_{16} nel campo indirizzo di `ld` a 400000_{16} per ottenere indirizzo di `x` (0000000010000000_{16})
    - inseriamo 20_{16} (ovvero 32_{10}) nel campo indirizzo di `sd` a 400100_{16} per ottenere indirizzo di `y` (0000000010000020_{16})
    - NOTA: gli indirizzi associati alle operazioni di store vengono gestiti come per le load, ad eccezione del fatto che il formato istruzioni tipo `S` rappresenta le costanti diversamente dal formato `I` delle load.

Il processore

Cerchiamo di capire come è strutturato un processore. Le reti logiche viste in precedenza servono proprio per implementare a livello hardware le funzionalità della CPU.

Per ogni istruzione che viene eseguita dalla CPU vengono fatti i seguenti passi, e i primi due sono **uguali per ogni istruzione**:

1. Prelievo dell'istruzione dalla memoria (fetch)
2. Lettura del valore di uno o più registri operandi che vengono estratti direttamente dai campi dell'istruzione (decode)
3. Questo passo dipende ed è diverso per ogni specifica istruzione e infatti si tratta di eseguire ciò che l'operazione richiede (execute)

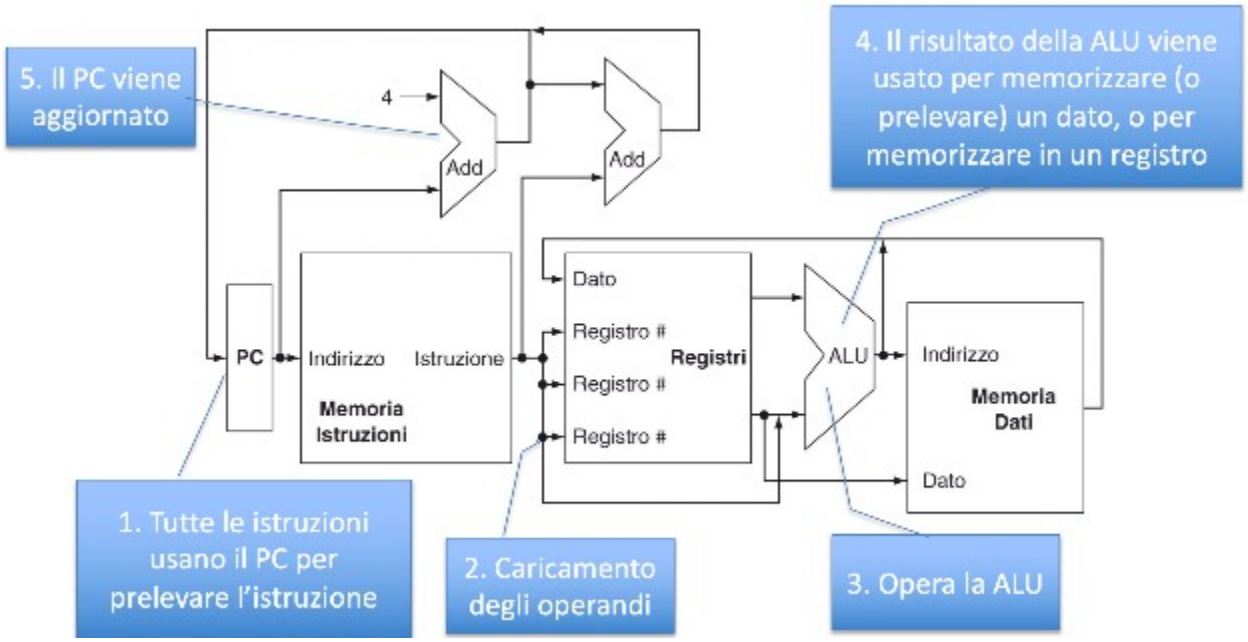
È importante notare che tutti i tipi di istruzioni che vedremo usano la ALU dopo aver letto gli operandi:

- Le istruzioni di accesso alla memoria per calcolare l'indirizzo
- Le istruzioni aritmetico logiche per motivi ovvi
- I salti condizionali per effettuare il confronto

Ovviamente dopo l'uso della ALU hanno tutte un comportamento diverso.

- Richiedono o salvano il dato in memoria
- Salvano il risultato nel registro target
- Cambiano il valore del registro PC in base all'esito del confronto

Il datapath è il percorso che le istruzioni e i dati fanno all'interno del processore:



In questa figura ovviamente ci sono dei pezzi mancanti, perchè appunto servono degli strumenti che permettano di "scegliere" un percorso in base a delle condizioni (ad esempio il PC in una istruzione normale viene aggiornato basicamente con un +4, invece in una istruzione di salto gli viene aggiunto l'offset, o un altro esempio è un operazione immediata invece di una che opera su due registri)

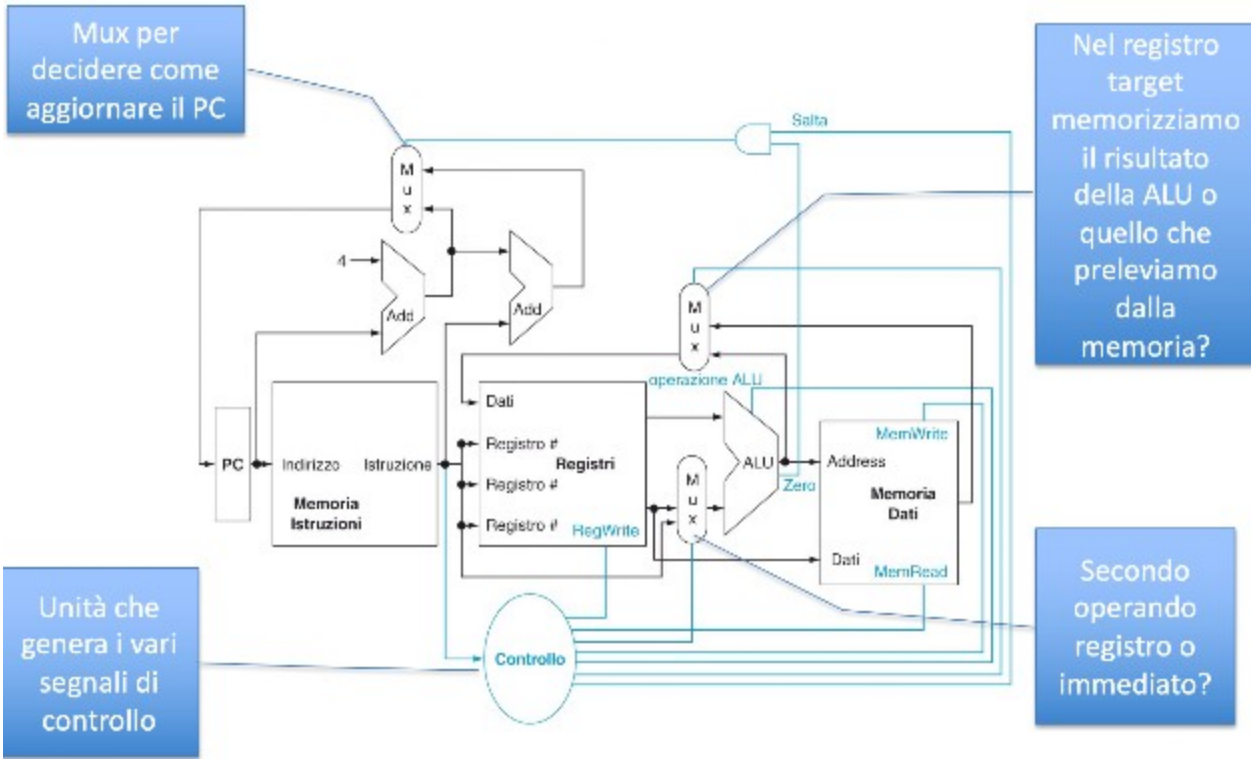
Per poter implementare queste funzionalità viene usata una particolare rete combinatoria, il multiplexer, che appunto funge da selettore dati.

Il Multiplexer infatti ha due (o più) ingressi dati, e un ingresso di controllo, sulla base dell'ingresso di controllo si decide quale degli input debba finire in output. Il valore di ingresso del multiplexer viene scelto dall'unità di controllo in base al tipo di istruzione.

Oltre ai multiplexer, anche i vari blocchi funzionali hanno ulteriori ingressi di controllo:

- La ALU per decidere quale tipo di operazione effettuare
- Il banco registri per decidere se scrivere o meno in un registro
- La memoria dati per decidere se vogliamo effettuare scritture oppure letture

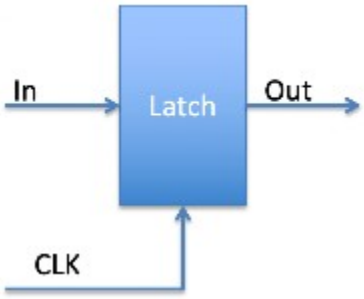
Per decidere come impiegare i vari ingressi di controllo c'è un'unità che funge da "direttore d'orchestra", l'**unità di controllo**



Ricordiamoci che il processore lavora sincronizzandosi con i cicli di clock, ma facciamo l'assunzione semplificativa che tutte le istruzioni si riescano a svolgere in un ciclo di clock (abbastanza lungo)

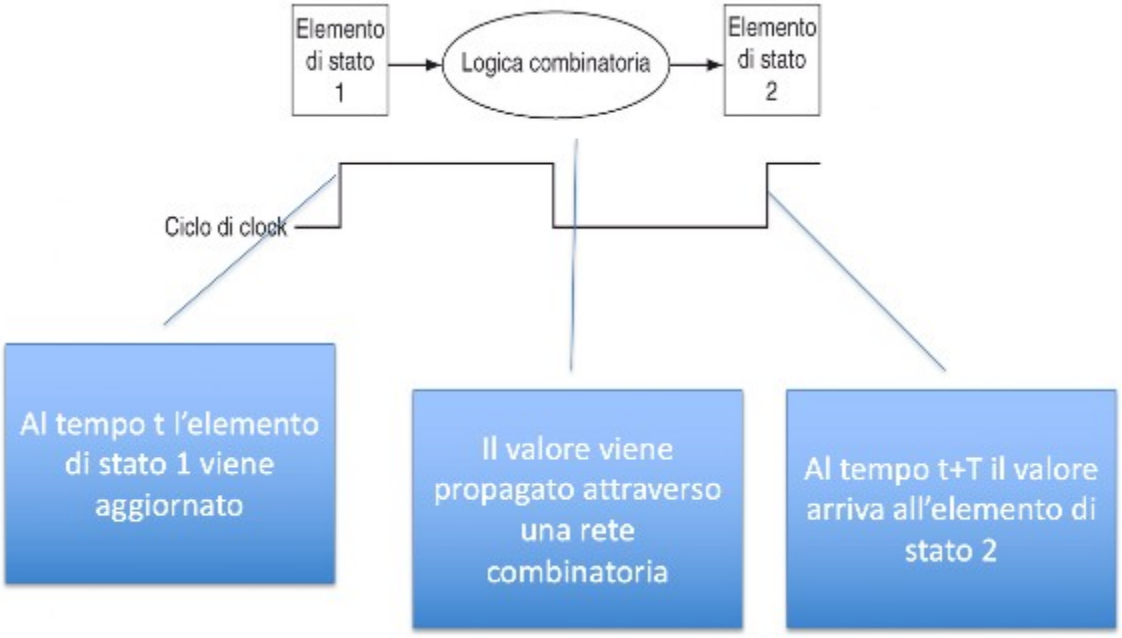
Ricordiamo che una rete logica combinatoria (come i multiplexer) è statica rispetto agli input. Invece esistono elementi detti "di stato" (registri, memoria dati, memoria istruzioni), questo perché ad un certo punto viene "salvato" il valore degli elementi e viene ricaricato, quindi l'ingresso dipende dalla storia e queste sono le reti sequenziali.
Gli elementi di stato hanno almeno due ingressi: il **valore** da immettere nello stato e un **clock** per sincronizzare le transizioni di stato

Come abbiamo visto l'elemento base per memorizzare un bit è il flip-flop di tipo D-latch



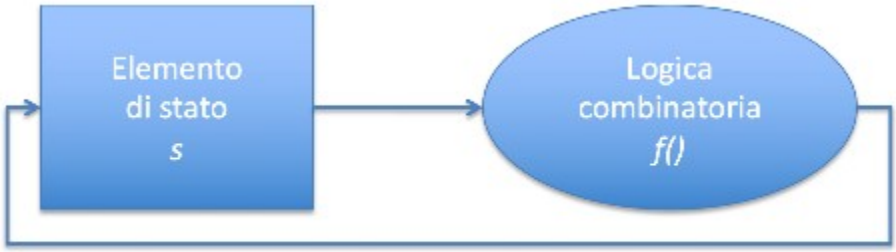
- I registri possono essere ottenuti come array di latch (o in altri modi simili)
- Terminologia
  - Asserito: segnale logico a livello alto
  - Non Asserito: segnale logico a livello basso

La metodologia di temporizzazione ci permette di capire quando i segnali possono essere letti o scritti, e ad esempio sapere se il dato che leggo è quello precedente o successivo alla scrittura. In genere si usa la tecnica di temporizzazione sensibile ai fronti (cioè il dato viene memorizzato in corrispondenza della salita/discesa del clock) e i dati presi dagli elementi di stato sono relativi al ciclo precedente



Considerazioni:

- Il tempo del clock T deve essere scelto in modo da dare tempo ai dati di attraversare la rete combinatoria
- Nel caso di RISC-V quasi tutti gli elementi di stato hanno ingressi e uscite a 64 bit
- La memorizzazione sincronizzata al clock permette di avere sempre qualcosa di determinato (cosa non vera nel caso in cui non ci sia un clock)



- Se non avessimo temporizzazioni precise dovremmo scrivere (qualcosa di indecidibile):

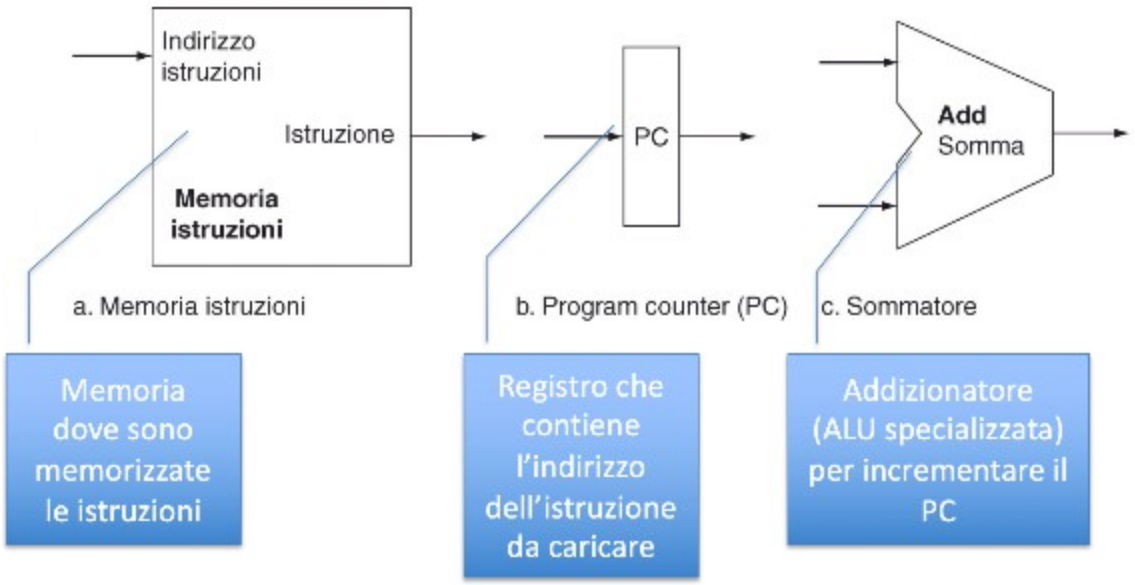
$$s = f(s)$$

- Grazie alla temporizzazione sensibile al clock abbiamo:

$$s(t+T) = f(s(t))$$

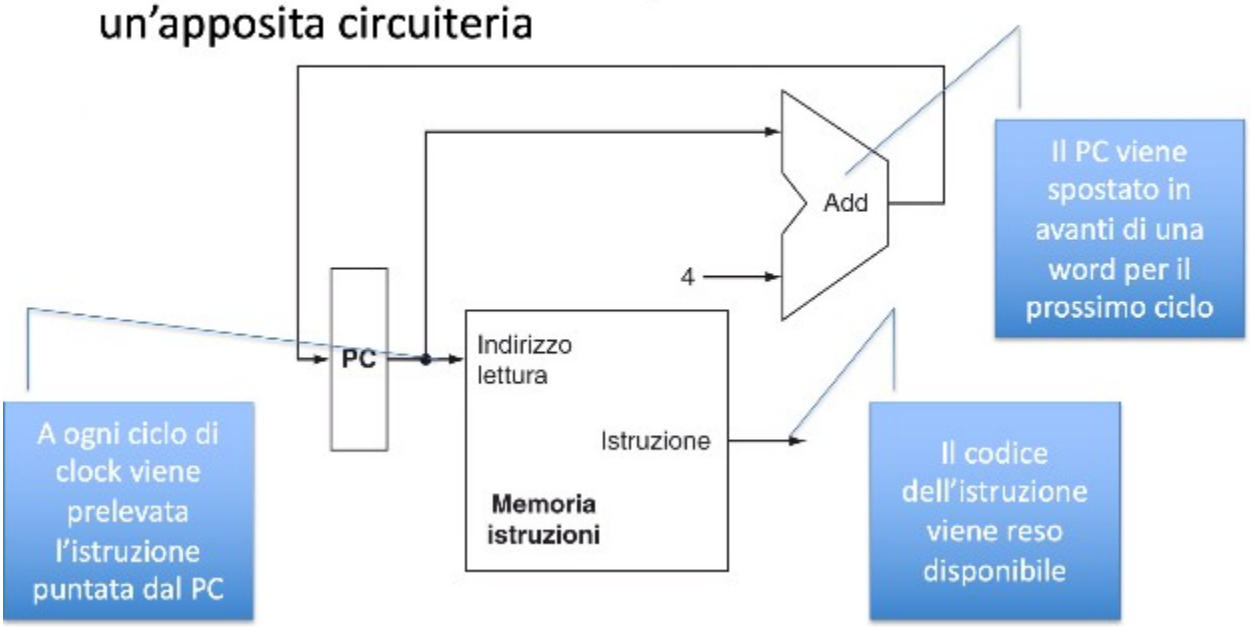
che invece produce un'evoluzione ben determinata

- Passiamo ora in rassegna le varie componenti che ci servono per la realizzazione del datapath



I multiplexer (usati per creare le condizioni e le scelte del path) vengono applicati bit-a-bit (infatti si usano una cascata di MUX) ai dati che sono composti da 32 o 64 bit

- Usando gli elementi che abbiamo visto possiamo mostrare come effettuare il prelievo dell'istruzione con un'apposita circuiteria



Istruzioni aritmetiche (tipo R)

- Esempio:

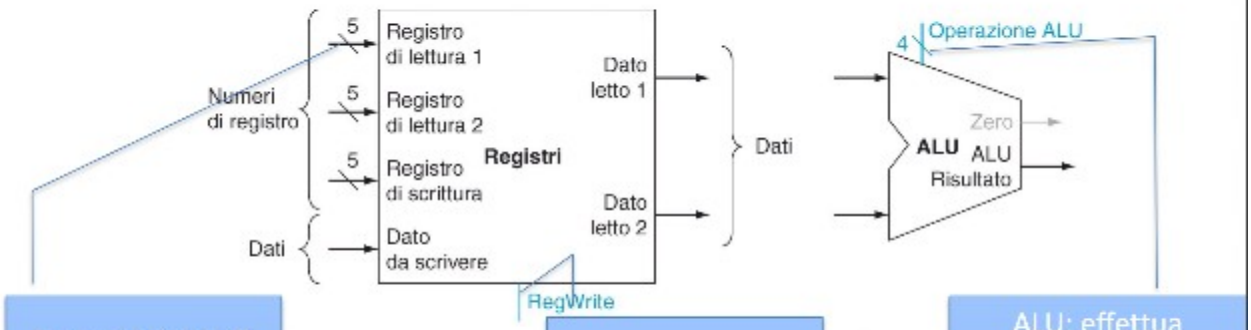
```
add x5, x6, x7
```

- Come noto, il codice binario corrispondente ha la forma:

7 bit	5 bit	5 bit	3 bit	5 bit	7 bit
funz7	rs2	rs1	funz3	rd	codop
31:25	24:20	19:15	14:12	11:7	6:0
0000000	00111	00110	000	00101	0110011

Tipo R

- Per effettuare questi calcoli ho bisogno di due ulteriori blocchi funzionali.



Banco registri: dà in output i registri specificati nel registro di lettura 1 e 2

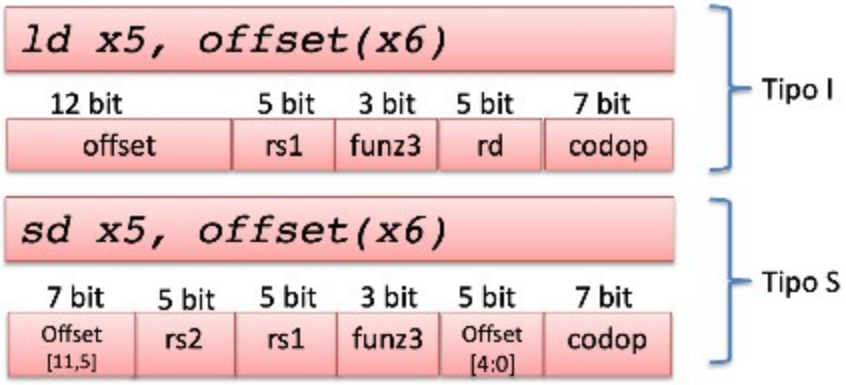
tri

Se abilitato in scrittura scrive nel registro specificato il dato in ingresso

b. ALU

ALU: effettua l'operazione aritmetica codificata in 4 bit. Setta un bit in uscita se il risultato è zero

Istruzioni di accesso alla memoria:



- Per entrambe si deve calcolare un indirizzo di memoria dato dalla somma di x6 con l'offset
- Per entrambe occorre leggere dal register file
- Quindi per eseguire queste istruzioni ci servono ancora la ALU e il register file

- Notare che l'offset viene memorizzato in un campo a 12 bit che occorrerà estendere a 64 bit (replicando per 42 volte il bit di segno)
- In aggiunta alle componenti che abbiamo visto prima occorre un'unità di memoria dati dove memorizzare eventualmente con *sd* (o da cui leggere con *ld*)



A differenza della memoria istruzioni questa può essere usata in lettura e scrittura. Quindi ho bisogno di comandi appositi.

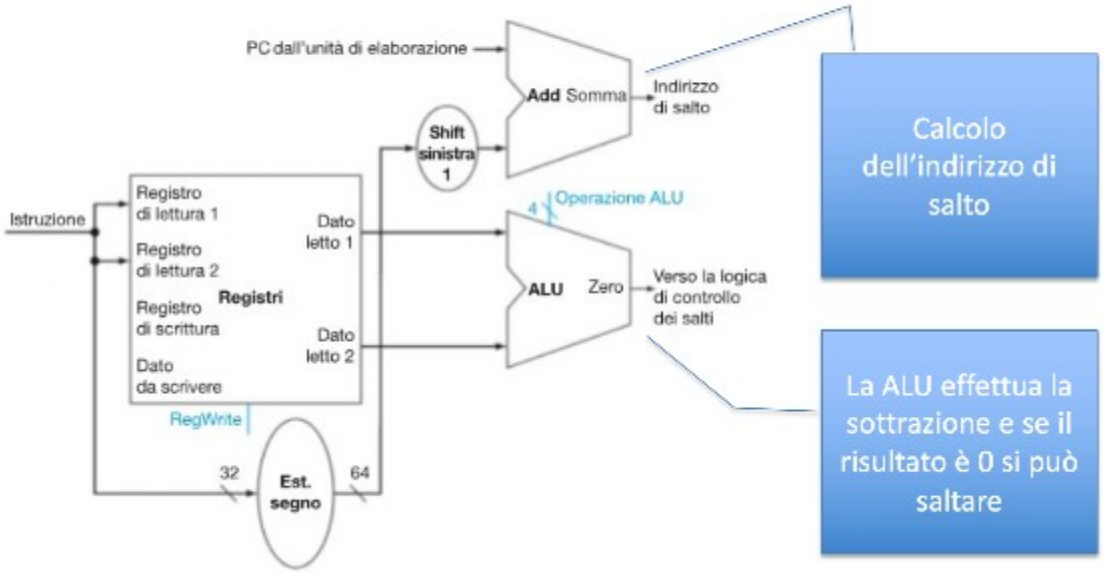
Istruzioni di salto:

- L'istruzione di salto condizionato ha la forma

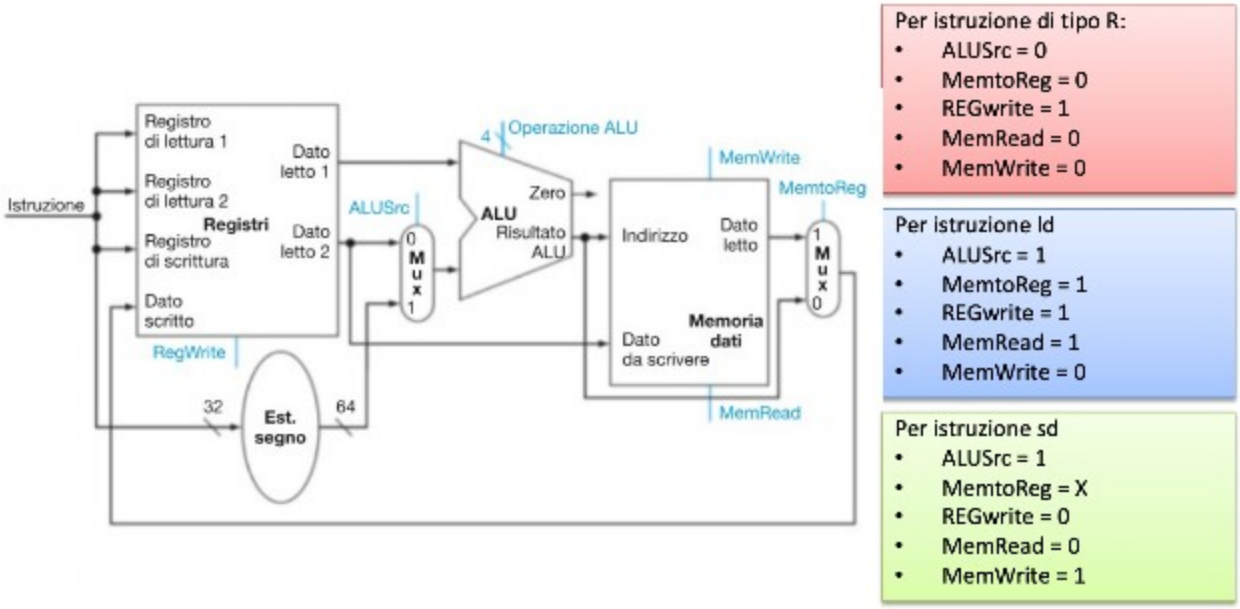


- Anche in questo caso bisogna sommare all'attuale PC l'offset a 12 bit (dopo averlo esteso a 64 bit con segno) che consente di fare salti da -2^{12} a 2^{12}.
- Due note:
  - L'architettura dell'insieme delle istruzioni specifica che l'indirizzo di base per il calcolo dell'indirizzo di salto è quello dell'istruzione di salto stessa.
  - L'architettura stabilisce che il campo offset sia spostato di 1 bit a sinistra per fare sì che l'offset codifichi lo spiazzamento in numero di mezze parole (aumentando lo spazio di indirizzamento dell'offset di un fattore 2 rispetto a codifica dello spiazzamento in byte).
  - La ragione per lo shift di uno (anziché di due) è dovuta alla presenza non documentata sul libro di istruzioni compresse a 16 bit per alcuni processori RISC-V.

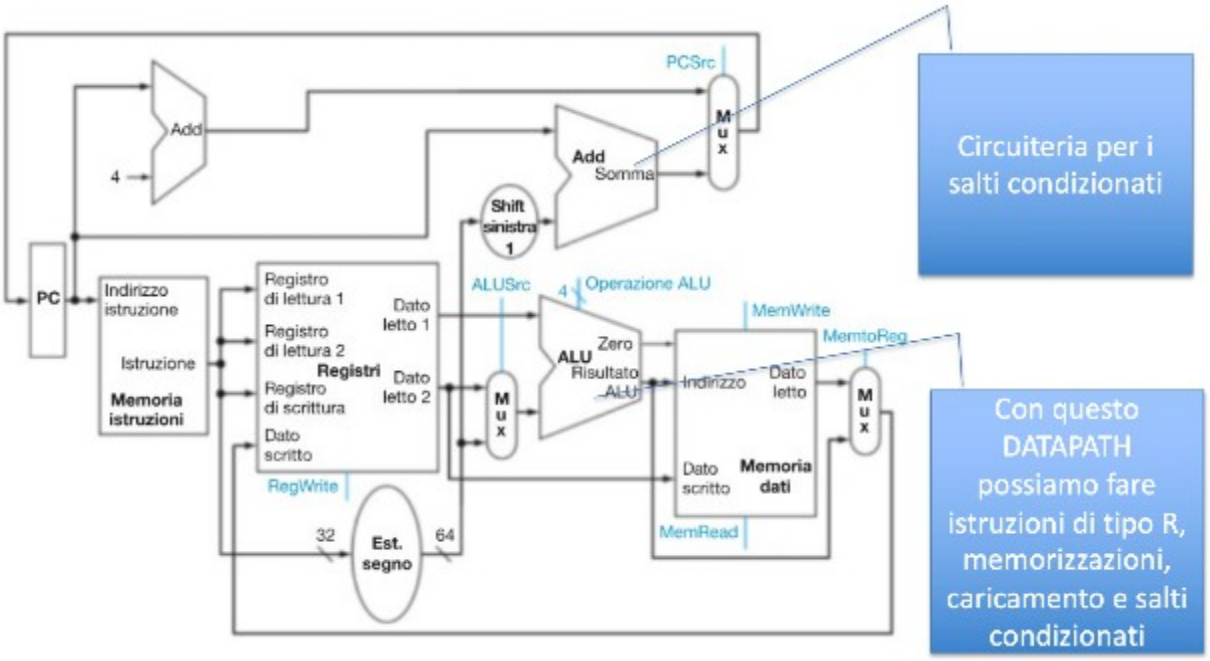
- Nell'esecuzione della *beq* occorre anche un meccanismo in base al quale decidere se aggiornare il PC a PC + 4 o a PC + offset



Visto che abbiamo assunto che il clock è in grado di eseguire ogni istruzione in un ciclo, non possiamo usare più di una volta un'unità funzionale all'interno di un ciclo, quindi dobbiamo fare la distinzione tra memoria istruzioni e memoria dati (visto che per le istruzioni load e store dobbiamo accedere ad entrambe le memorie).
Inoltre dobbiamo fare in modo di condividere più possibile le varie unità, mettendo appositi multiplexer



Il cammino critico è il cammino più lungo in termini di tempo che un segnale può fare (tenendo conto dei componenti che vengono eseguiti in parallelo)



Siccome la ALU viene impiegata per vari scopi (operazioni aritmetico/logiche, calcolo indirizzi, sottrazioni per confronto) dobbiamo avere diverse configurazioni in base a quello che vogliamo che faccia:

Linea controllo ALU	Operazione
0000	AND
0001	OR
0010	Somma
0110	Sottrazione

Per poter generare i bit di controllo da mandare in input alla ALU useremo una unità di controllo che riceve in ingresso:

- funz7
- funz3
- ALUOp, due bit che specificano le seguenti cose:
  - 00
    somma (nel caso di istruzioni di sd e ld in cui bisogna calcolare indirizzi)
  - 01
    sottrazione (per istruzioni di confronto)
  - 10
    operazioni aritmetico/logiche di tipo R (specificate da funz3 e funz7)

Codice operativo istruzione	ALUOp	Operazione eseguita dall'istruzione	Campo funz7	Campo funz3	Operazione dell'ALU	Ingresso di controllo alla ALU
ld	00	load di 1 parola doppia	XXXXXXX	XXX	somma	0010
sd	00	store di 1 parola doppia	XXXXXXX	XXX	somma	0010
beq	01	salto condizionato all'uguaglianza	XXXXXXX	XXX	sottrazione	0110
Tipo R	10	add	0000000	000	somma	0010
Tipo R	10	sub	0100000	000	sottrazione	0110
Tipo R	10	and	0000000	111	AND	0000
Tipo R	10	or	0000000	110	OR	0001

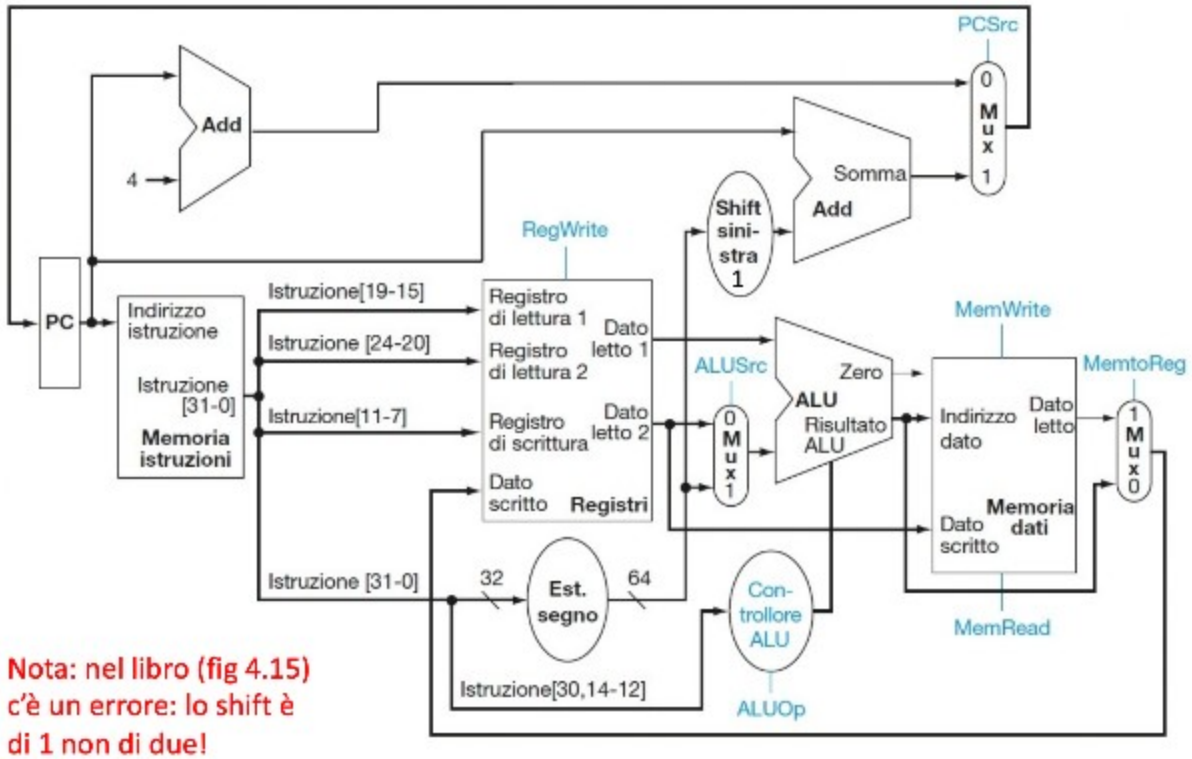
Notiamo quindi che abbiamo due livelli di generazione degli input:

- Unità di controllo (generale)
  In base a codop, funz7 e funz3 genera i 2 bit di ALUOp
- Unità di controllo della ALU
  In base ai 2 bit di ALUOp di ingresso (e funz3 e funz7) genera i segnali di controllo per la ALU di 4 bit (per eseguire effettivamente le operazioni)

In ingresso all'unità di controllo della ALU arrivano in tutto 12 bit (Ingresso di controllo della ALUOp, funz7 e funz3), quindi in totale avremmo $2^{12} = 4096$ combinazioni, però non vengono usate tutte, infatti con le X segneremo i bit che non sono importanti per quella operazione.

Tabella di verità (compressa):

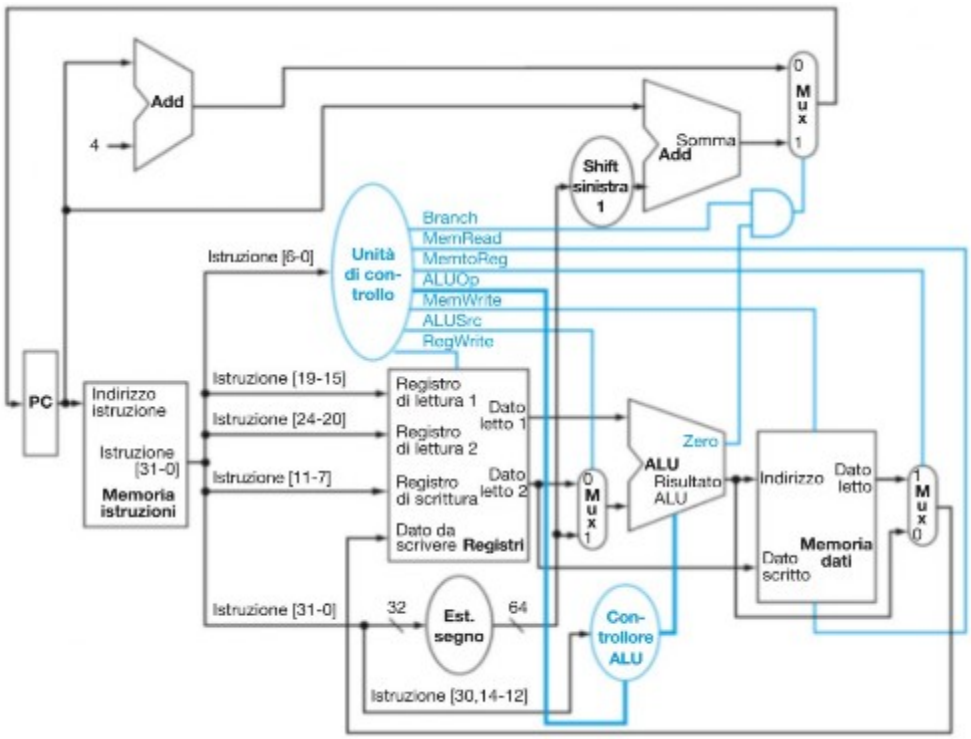
ALUOp		Campo funz7							Campo funz3			Operazione
ALUOp1	ALUop2	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010 ld/sd
X	1	X	X	X	X	X	X	X	X	X	X	0110 beq
1	X	0	0	0	0	0	0	0	0	0	0	0010 add
1	X	0	1	0	0	0	0	0	0	0	0	0110 sub
1	X	0	0	0	0	0	0	0	1	1	1	0000 AND
1	X	0	0	0	0	0	0	0	1	1	0	0001 OR



Nota: nel libro (fig 4.15) c'è un errore: lo shift è di 1 non di due!

Nome del segnale	Effetto quando non asserito	Effetto quando asserito
RegWrite	Nullo	Il dato viene scritto nel register file nel registro individuato dal numero del registro di scrittura
ALUSrc	Il secondo operando della ALU proviene dalla seconda uscita del register file (Dato letto 2)	Il secondo operando della ALU proviene dall'estensione del segno dei 12 bit del campo immediato dell'istruzione
PCSrc	Nel PC viene scritta l'uscita del sommatore che calcola il valore di PC + 4	Nel PC viene scritta l'uscita del sommatore che calcola l'indirizzo di salto
MemRead	Nullo	Il dato della memoria nella posizione puntata dall'indirizzo viene inviato in uscita sulla linea "Dato letto"
MemWrite	Nullo	Il contenuto della memoria nella posizione puntata dall'indirizzo viene sostituito con il dato presente sulla linea "Dato scritto"
MemtoReg	Il dato inviato al register file per la scrittura proviene dalla ALU	Il dato inviato al register file per la scrittura proviene dalla Memoria Dati

Schema complessivo con unità di controllo:



L’unità di controllo genera tutti i segnali di controllo, che prende in input il codice operativo dell’istruzione e genera i comandi del caso.

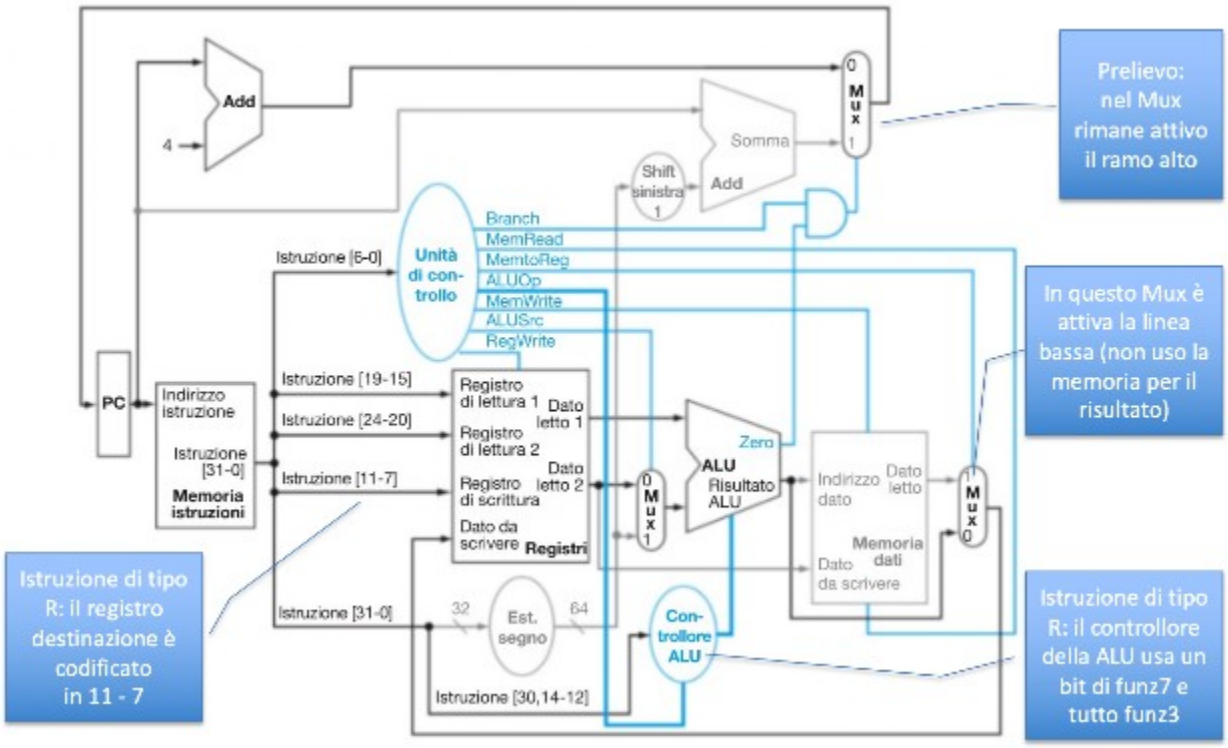
Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
ld	1	1	1	1	0	0	0	0
sd	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Add

- Per eseguire un'istruzione di tipo R (ad esempio: *add x5, x6, x7*) occorre:

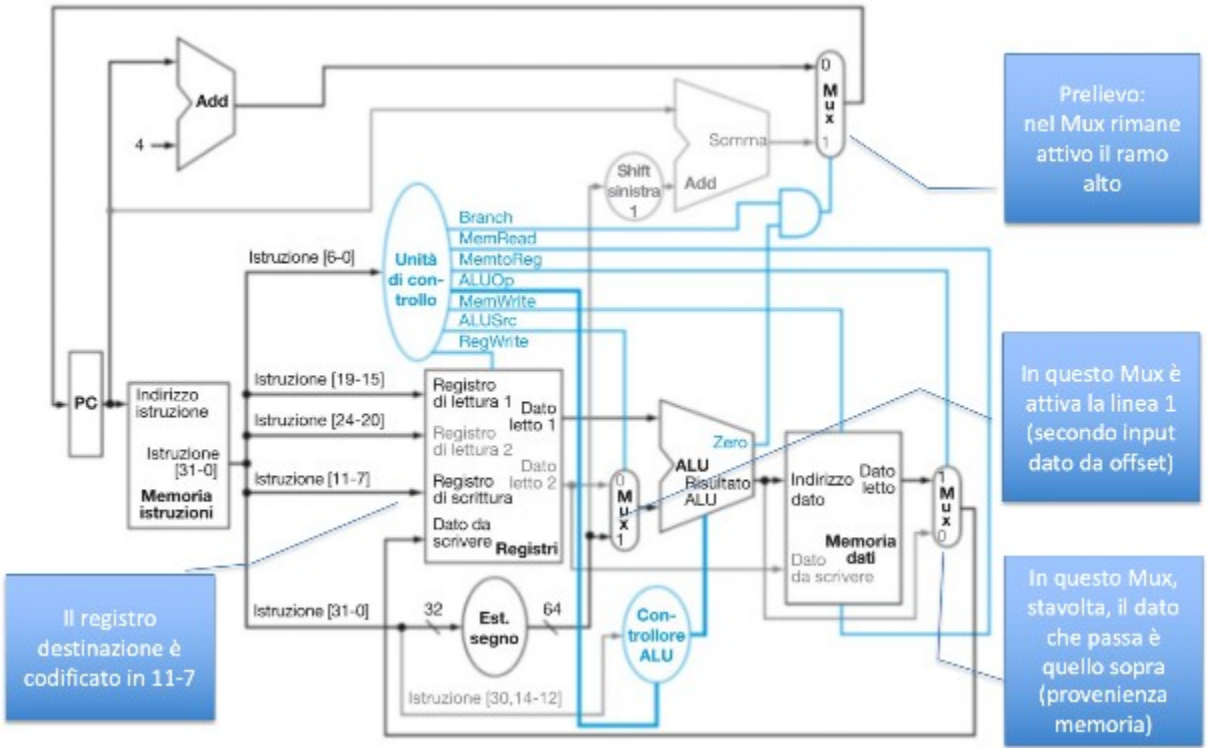
1. prelevare l'istruzione dalla memoria e incrementare il PC di 4
2. leggere x6 e x7 dal register file mentre l'unità di controllo principale calcola il valore da attribuire alle linee di controllo
3. attivare la ALU con in input i dati dal register file usando alcuni bit del codice operativo per selezionare l'operazione della ALU (ad esempio: add)
4. memorizzare il risultato nel registro destinazione (x5)

- Tutto questo avviene in un solo ciclo



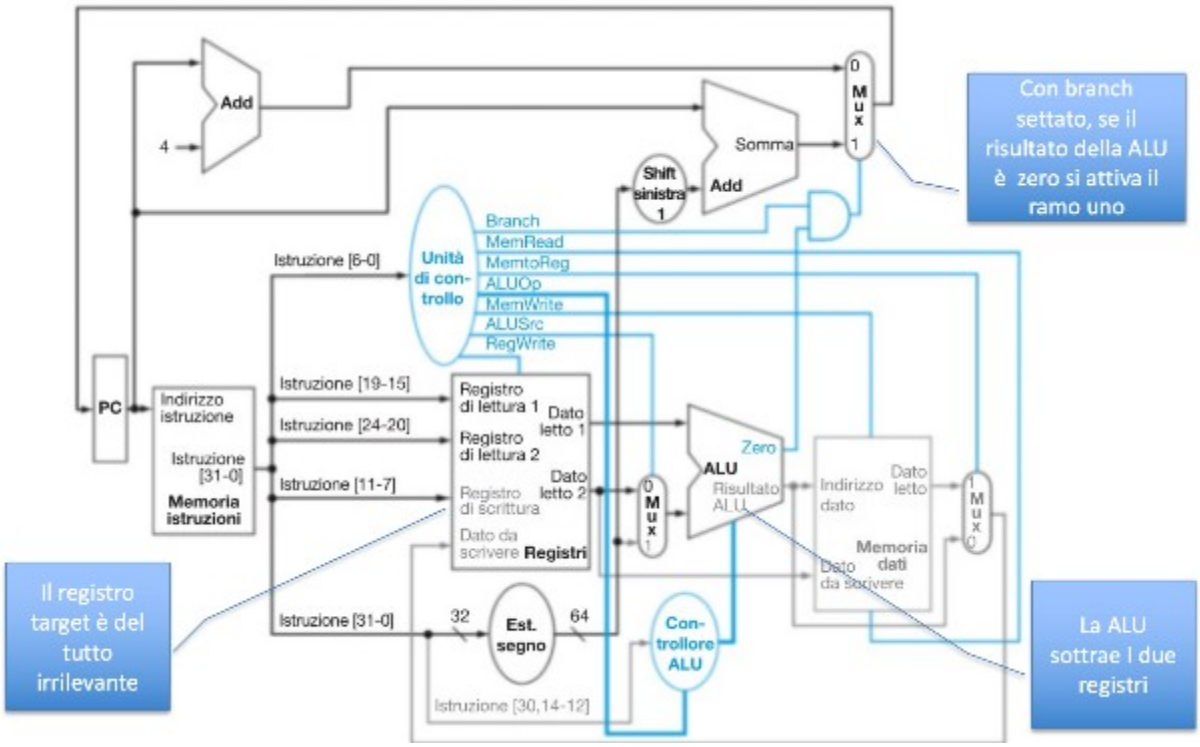
Load

- Consideriamo ora l'istruzione
  ld x5, offset(x6)
- Fasi
  1. la fase di prelievo istruzione ed incremento del PC è uguale a prima
  2. prelevare x6 dal register file
  3. la ALU somma il valore letto dal register file i 12 bit del campo offset dell'istruzione, dotati di segno ed estesi a 64 bit.
  4. il risultato della somma viene usato come indirizzo per memoria dati
  5. il dato prelevato dall'unità di memoria dati viene scritto nel register file nel registro x5



Beq

- Consideriamo ora l'istruzione
  beq x5, x6, offset
- Fasi
  1. la fase di prelievo istruzione ed incremento del PC è uguale a prima
  2. prelevare x5 e x6 dal register file
  3. la ALU sottrare x5 da x6. Il valore del PC viene sommato ai 12 bit del campo offset dell'istruzione, dotati di segno, estesi a 64 bit e fatti scorrere di una posizione a sinistra. Il risultato costituisce l'indirizzo di destinazione del salto
  4. la linea Zero in uscita dalla ALU viene usata per decidere da quale sommatore prendere l'indirizzo successivo da scrivere nel PC.



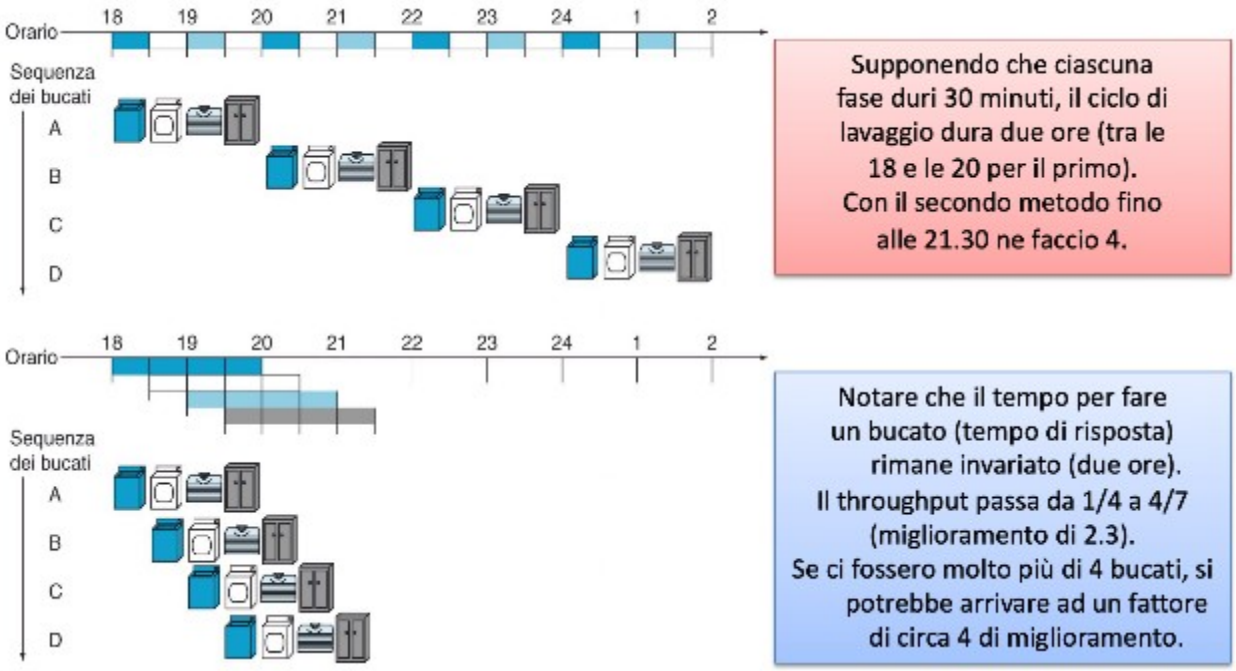
Nella realtà però non si fa più durare un ciclo di clock in modo che possa eseguire tutte le istruzioni, questo perché a dettarlo sarà sempre l'istruzione più lunga (e ne esistono di molto complesse) e in più non si riescono a fare ottimizzazioni sulle operazioni fatte più di frequente

Pipeline

Il trucco per superare il problema della durata del ciclo di clock deriva dall'ideatore della catena di montaggio, Henry Ford, infatti facendo lavorare contemporaneamente le componenti della cpu, su istruzioni diverse nello stesso momento, mantenendo una logica sequenziale, si possono ottimizzare i tempi in maniera molto elevata.

Un esempio è che se abbiamo una istruzione che ha 4 fasi, che vengono tutte compiute da un componente diverso, e dobbiamo eseguire 4 istruzioni, il throughput totale, se manteniamo la logica a singolo ciclo, sarà di 4 task in $4*4$ unità di tempo $\frac{4}{4*4}=\frac{4}{16}=\frac{1}{4}$.

Invece implementando una **pipeline**, facendo si che mentre la prima istruzione viene elaborata dalla seconda componente, facciamo elaborare alla prima componente la seconda istruzione e così via, diventa 4 task in 7 unità di tempo quindi $\frac{4}{7}$.

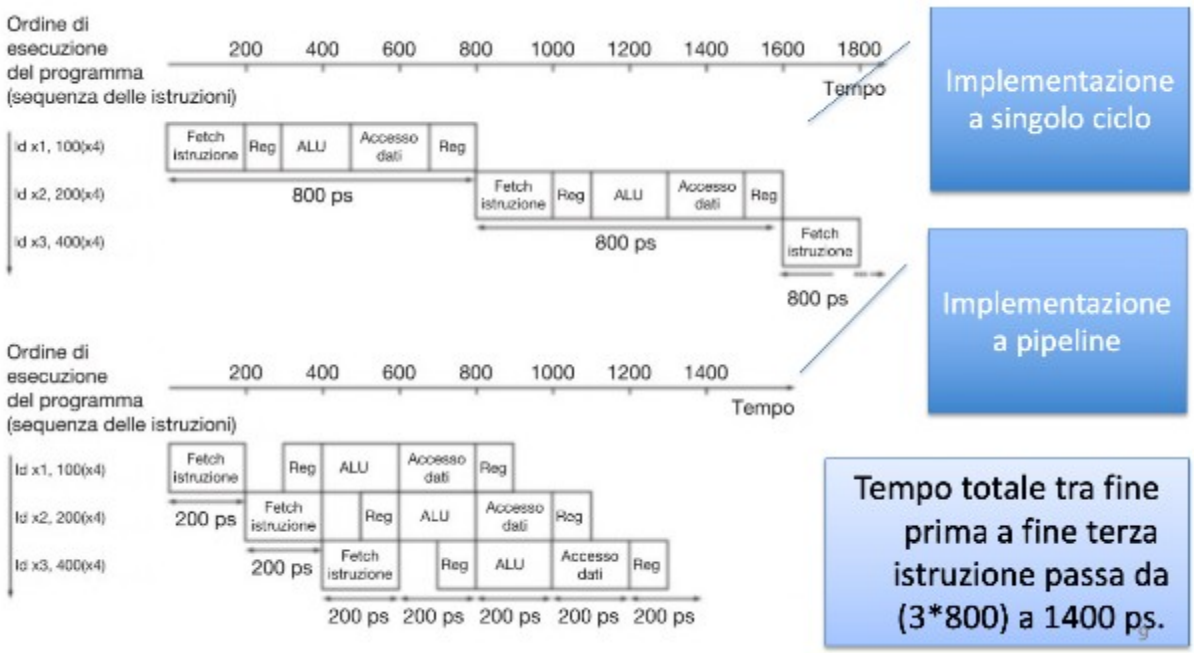


La pipeline è una struttura che parallelizza l'esecuzione di più task

Passando ad analizzare le istruzioni che abbiamo già visto in precedenza, notiamo che il clock deve adeguarsi all'operazione più lenta della componente più lenta:

Tipo di istruzione	Lettura dell'istruzione	Lettura dei registri	Operazione con la ALU	Accesso ai dati in memoria	Scrittura del register file	Tempo totale
Load doubleword (ld)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store doubleword (sd)	200 ps	100 ps	200 ps	200 ps		700 ps
Formato R (add, sub e or)	200 ps	100 ps	200 ps		100 ps	600 ps
Salto condizionato (beq)	200 ps	100 ps	200 ps			500 ps

Quindi con un implementazione a singolo ciclo il clock avrà un ciclo lungo $800\ ps$, invece con un implementazione a pipeline, ogni "step" dovrà durare $200\ ps$, ed essendo 5 step avremo che un ciclo durerà $1000\ ps$, però anche con un ciclo più lungo, l'implementazione a pipeline sarà molto più veloce ed efficiente a larga scala (tante istruzioni)



Osserviamo che con poche istruzioni non riusciamo a riempire completamente la pipeline, e ovviamente durante la fase iniziale e quella finale c'è sempre uno "spreco" essendo che certi componenti non lavorano.
La formula per calcolare i vantaggi di usare la pipeline è la seguente:

$$Tempo\ \ tra\ \ due\ \ istruzioni\ \ con\ \ pipeline = \frac{Tempo\ \ tra\ \ due\ \ istruzioni\ \ senza\ \ pipeline}{Numeri\ \ di\ \ stadi\ \ della\ \ pipeline}$$

Con tempo tra due istruzioni che indica il tempo che passa dall'inizio di una all'inizio di un'altra.
Nell'esempio precedente passiamo da $2400\ ps$ a $1400\ ps$, dunque un miglioramento di 1.7 invece di 5 come suggerirebbe la formula.
Questo perchè:

- Per avere un miglioramento di 5 dovremmo far durare ciascuna fase $160\ ps$ portando il clock a $160\ ps$ per fase, ma questo è impossibile avendo delle componenti che hanno bisogno di $200\ ps$
- Avendo usato poche istruzioni non siamo riusciti a riempire la pipeline e a sfruttare al meglio i suoi vantaggi.

Vantaggi RISC

1. Tutte le istruzioni hanno la stessa lunghezza (una word), e questo facilita molto il prelievo
2. I codici degli operandi sono in posizione fissa, questo permette di leggere il register file in parallelo con la decodifica dell'operazione, e quindi compattarle in un'unica operazione nella pipeline
3. L'uso degli operandi in memoria è permesso solo per le operazioni che fanno solo ed esclusivamente quello (cioè operazioni in memoria), quindi possiamo usare la ALU per il calcolo di indirizzi (altrimenti se ci fossero operazioni aritmetiche avremmo dovuto usare la ALU due volte durante la stessa istruzione).

4. L’uso di accessi allineati fa sì che gli accessi in memoria avvengano sempre in un ciclo di trasferimento (impegnando un solo stadio della pipeline)

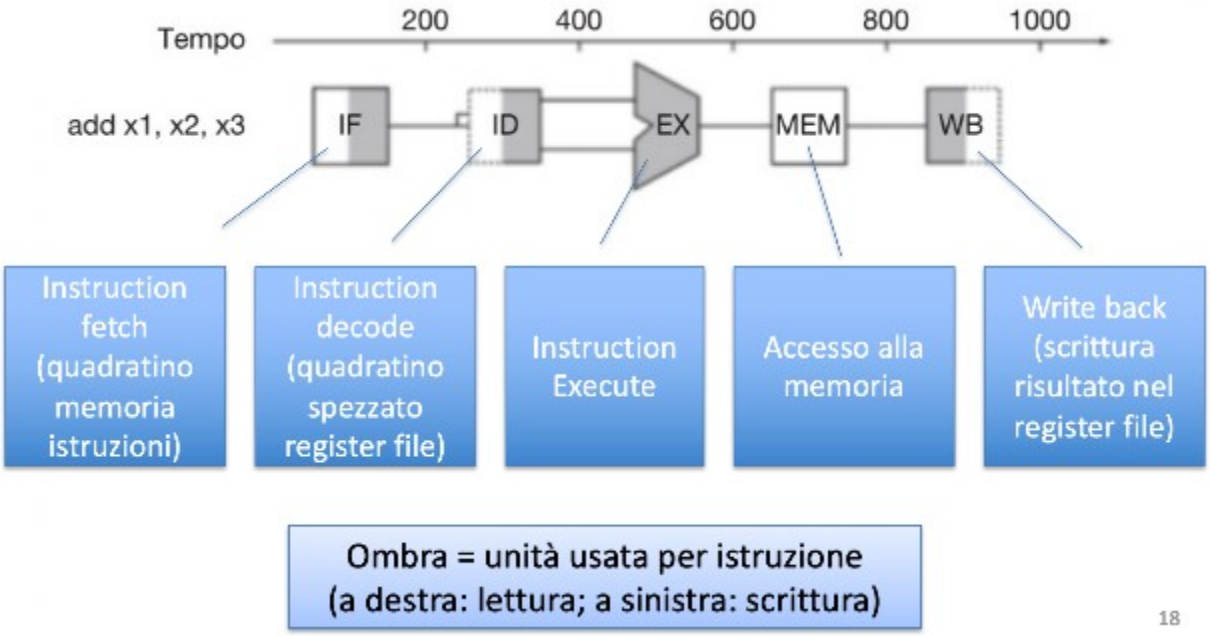
Hazard

In certe situazioni un ciclo di clock non riesce ad eseguire completamente un’istruzione, per via del verificarsi di alcune situazioni critiche, dette **hazard**.

- Strutturali
  L’architettura dell’elaboratore rende impossibile l’esecuzione di alcune sequenze di istruzioni in pipeline (ad esempio con un’unica memoria non potrei caricare istruzioni e anche prelevare/scrivere operandi in memoria, infatti esistono la memoria istruzioni e memoria dati, due componenti diverse)
- Sui dati
  Quando la pipeline deve essere messa in stallo per ottenere informazioni dagli stadi precedenti

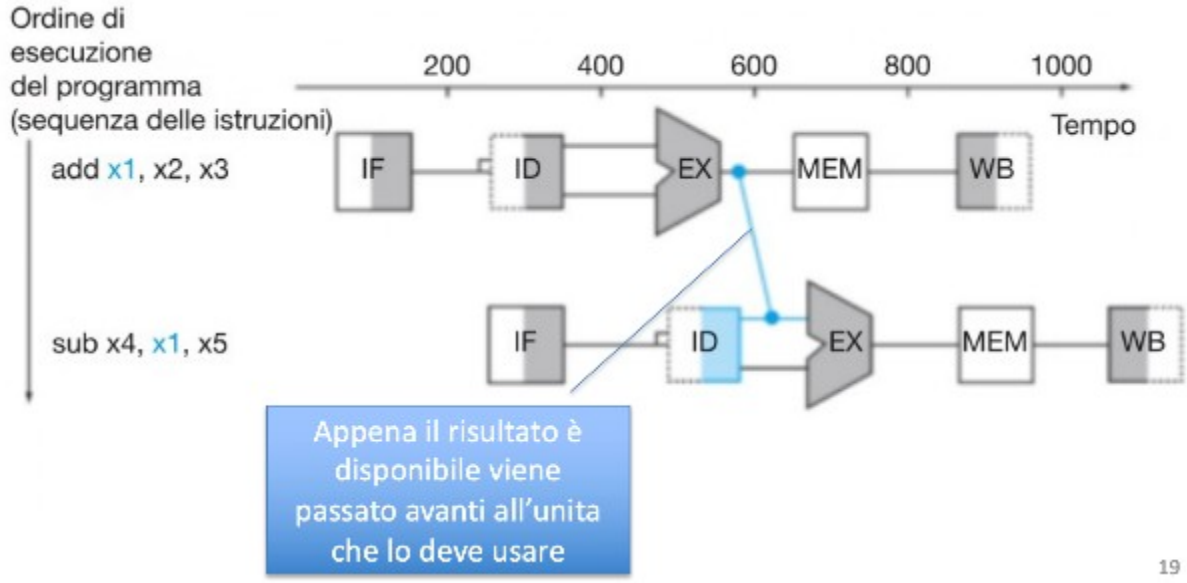
```
add x1, x2, x3
sub x4, x1, x5
```

In questa situazione quindi la sub ha bisogno di $x1$, però durante la lettura dei registri, che viene fatta nella seconda unità di tempo totale, la add è appena alla terza unità di tempo totale, e quindi deve aspettare.
In alcuni casi il compilatore esegue lo switch con altre istruzioni che non hanno bisogno di dati precedenti e avvia la sub solo quando il dato è disponibile, ma non sempre è possibile una ottimizzazione di questo tipo.
Però possiamo osservare che non serve aspettare che il risultato venga memorizzato



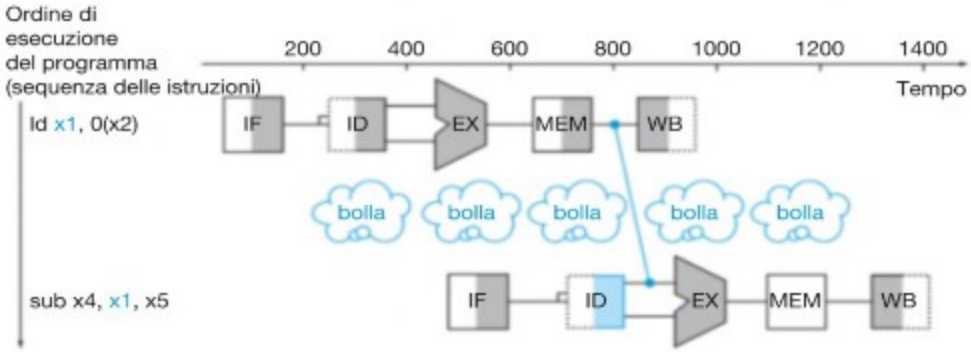
Si usa quindi una tecnica chiamata **operand forwarding**, che consiste nel rendere il

risultato disponibile prima di arrivare all'operazione di write back:



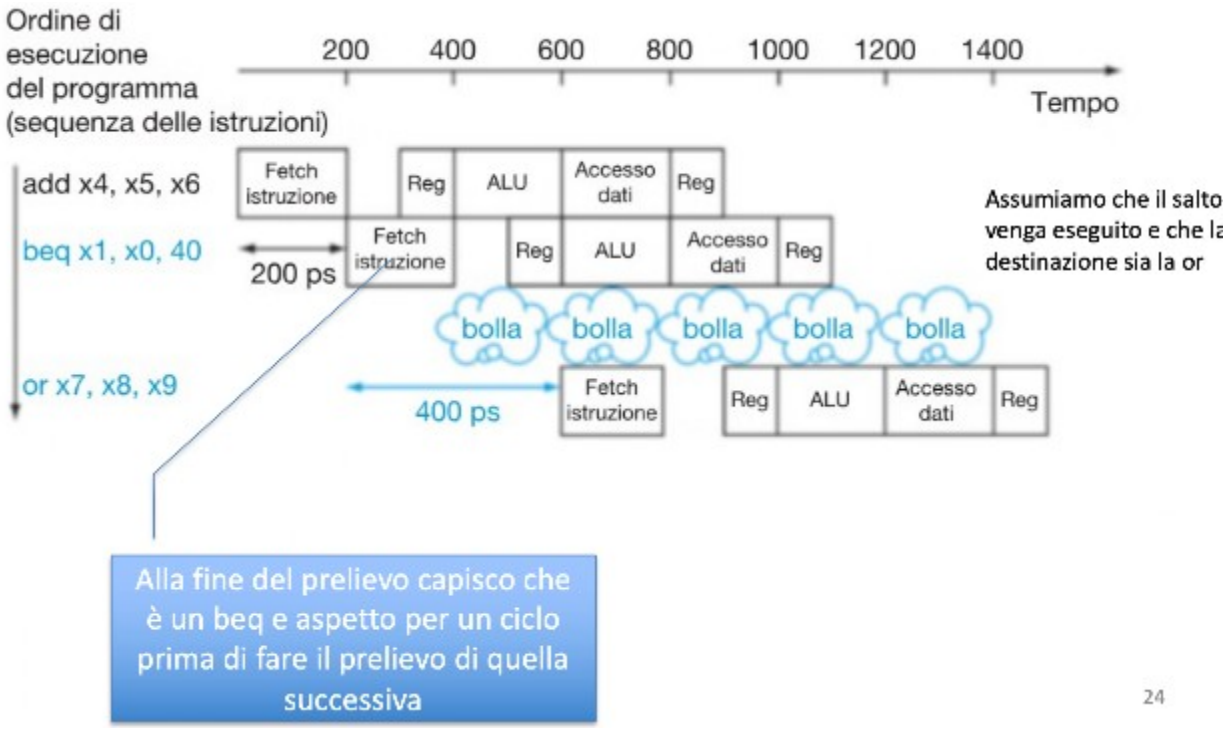
19

Ovviamente questa tecnica funzione se lo stadio in cui il dato viene propagato è successivo allo stadio dal quale viene prelevato
Quindi appunto nel caso di operazioni che non sono sequenziali da questo punto di vista serve imporre uno stallo per riportarci alla situazione precedente:



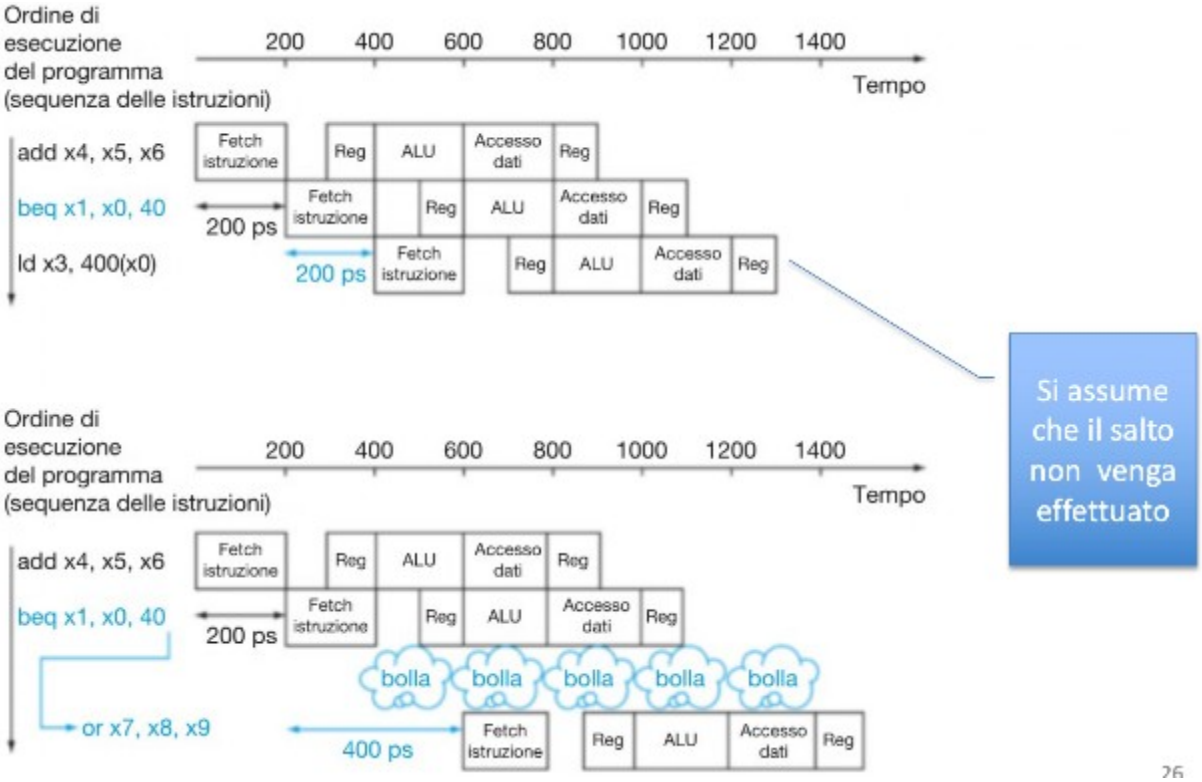
21

- Sul controllo
  Si tratta sostanzialmente dei salti condizionati, in cui fino ad un certo stadio, non saprò quale è il percorso, cioè le istruzioni successive, che dovranno venir compiute.
  Quindi o si blocca la pipeline fino a che non viene presa la decisione, come nel seguente esempio:



24

Oppure in maniera più furba, si può scegliere una delle due opzioni (salta o non salta), di solito scegliendo quella più probabile tramite alcuni circuiti che prevedano il salto, e si iniziano a caricare quelle istruzioni, per poi correggersi in caso contrario, così facendo in media risparmiamo tempo

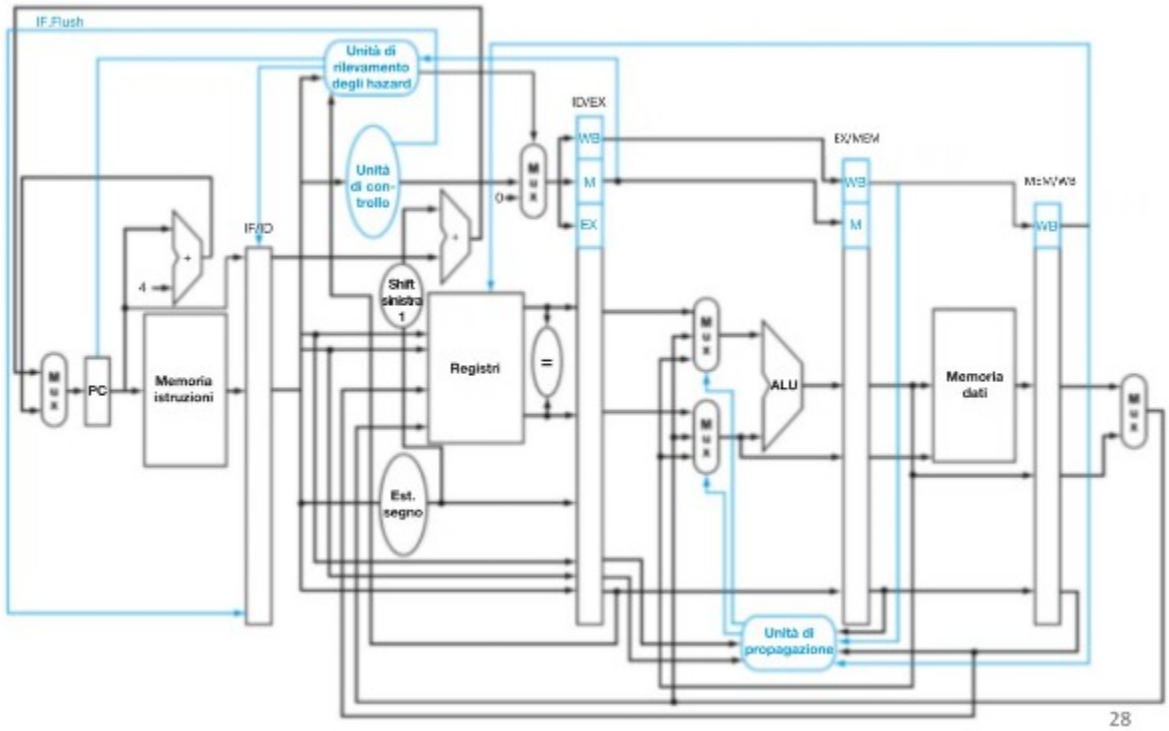


26

Ovviamente esistono circuiterie più efficienti che riescono a capire o prima o comunque

in maniera migliore quale sarà l'esito della condizione, ad esempio memorizzando l'esito del branch precedente e assumendo che il comportamento si mantenga coerente.

Quadro finale delle componenti del processore:



La gerarchia di memoria

Un elaboratore ha per forza bisogno di una qualche memoria per poter funzionare, senza memoria sarebbe inutilizzabile.

La memoria nei calcolatori non è tutta uguale, esistono vari tipi di memorie, ognuna con scopi e funzionamenti diversi (ci sono dei compromessi tra costi, prestazioni e dimensioni).

La memoria su può classificare in due gruppi principali:

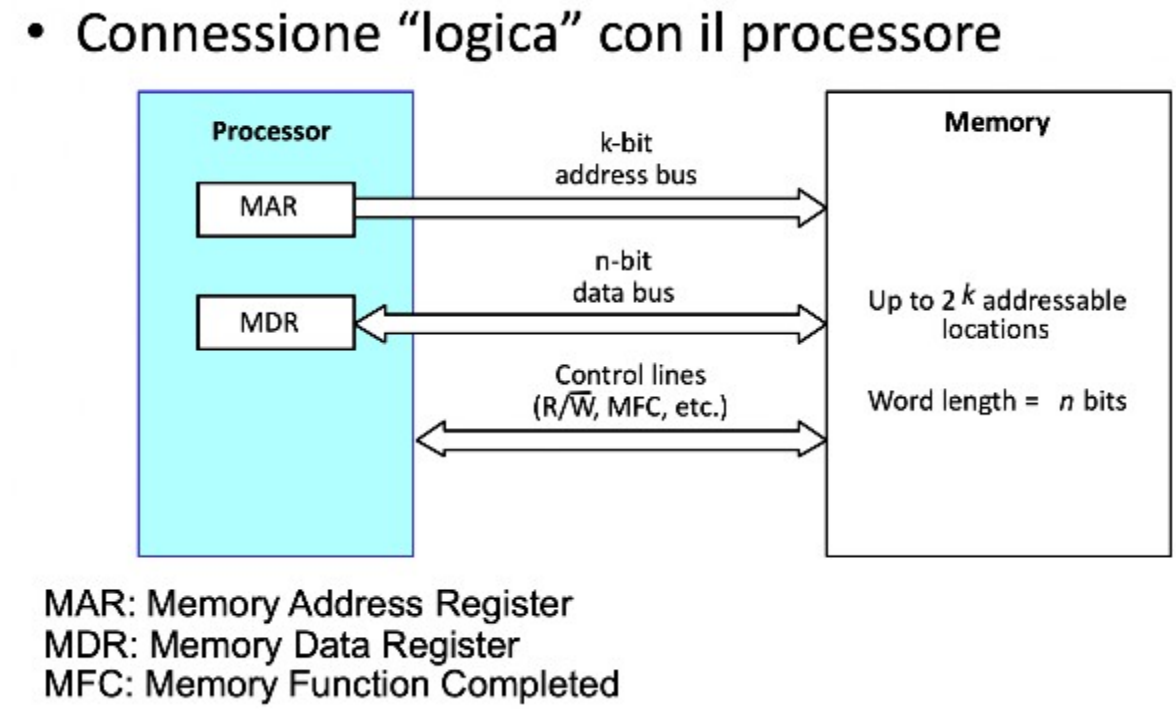
- Memoria indirizzata direttamente
  Fanno parte di questa categoria la memoria principale (la RAM) e la cache, sono memorie volatili e possono essere raggiunte direttamente dal processore.
  Questa memoria ha grandezza limitata in base allo spazio di indirizzamento del processore (non avrebbe senso avere più memoria se il processore non può raggiungerla)
- Memoria indirizzata in modo indiretto
  Fa parte di questa memoria la memoria periferica (hard disk, ssd, usb, dischi).
  È di tipo permanente, ovvero mantiene il suo contenuto anche senza alimentazione.
  Questa memoria ha uno spazio di indirizzamento "software" non limitato dal processore e viene gestita tipicamente dal sistema operativo.

Le informazioni della memoria principale sono accessibili al processore in qualsiasi momento, invece le informazioni della memoria periferica devono prima essere trasferite nella memoria

principale per poter essere lette, si occupa il software (solitamente il sistema operativo) di trasferire i dati tra memoria principale e la memoria periferica.
Il **tempo di accesso** è il tempo richiesto per svolgere **una** operazione di lettura/scrittura nella memoria
Il **tempo di ciclo** è il tempo che intercorre tra l'inizio di due operazioni consecutive tra locazioni diverse (in genere è leggermente superiore al tempo di accesso).

Con il termine **accesso casuale** si intende che non vi è alcuna relazione od ordine nei dati memorizzati, quindi il tempo di accesso ad un dato è costante e non dipende dalla sua posizione (tipico delle memoria a semiconduttori).
Con il termine **accesso sequenziale** invece si usa quando l'accesso in memoria è ordinato (o semiordinato) e quindi il tempo di accesso dipende dalla posizione in memoria (tipico dei dischi e dei nastri)

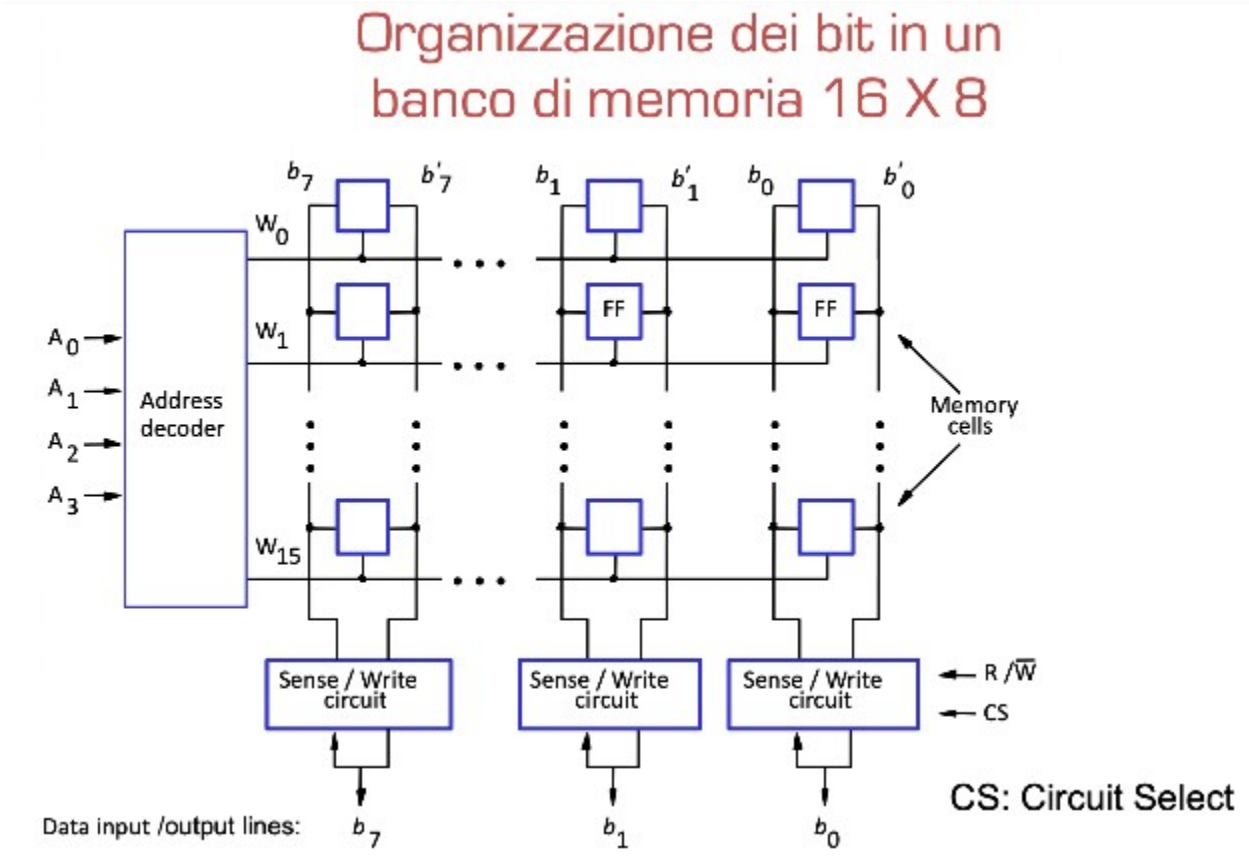
La **RAM** (Random Access Memory) è una memoria sia scrivibile che leggibile a semiconduttori, ha un accesso casuale, quindi indipendente dalla posizione dell'informazione
La **ROM** (Read Only Memory) è una memoria in sola lettura e anch'essa fa uso di semiconduttori. Può essere implementate sia con un accesso casuale sia con un accesso sequenziale



La RAM

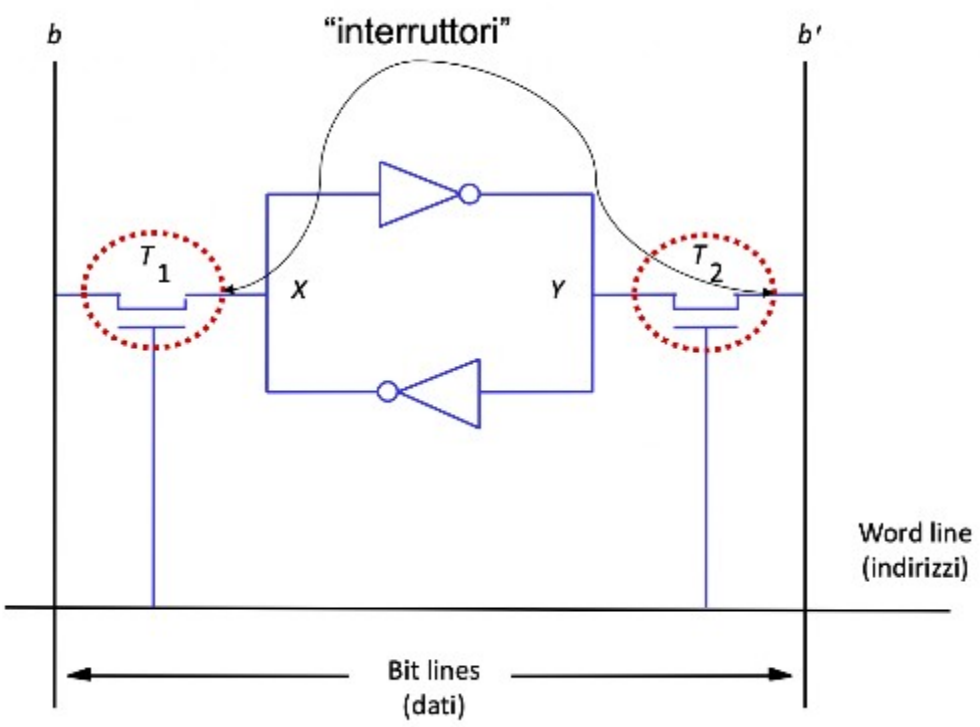
Le memorie RAM memorizzano singoli bit, normalmente organizzati in byte o word.
Data una capacità (numero di bit che può contenere la memoria), la memoria può essere organizzata in svariati modi, a seconda del parallelismo (quante righe e quante colonne).

L’organizzazione scelta influenza il numero di pin I/O del circuito integrato (del banco di RAM) che implementa la memoria



SRAM

Le memorie **statiche** SRAM sono memorie in cui i bit possono essere tenuti indefinitamente, cioè, a patto che non manchi l’alimentazione, le informazioni possono restare salvate per un periodo indeterminato.
Queste memoria sono estremamente veloci e consumano poca corrente (quindi non scaldano), il problema è che per implementarle ci vogliono molti componenti per ciascuna cella, e quindi si “spreca” spazio e costano di più.

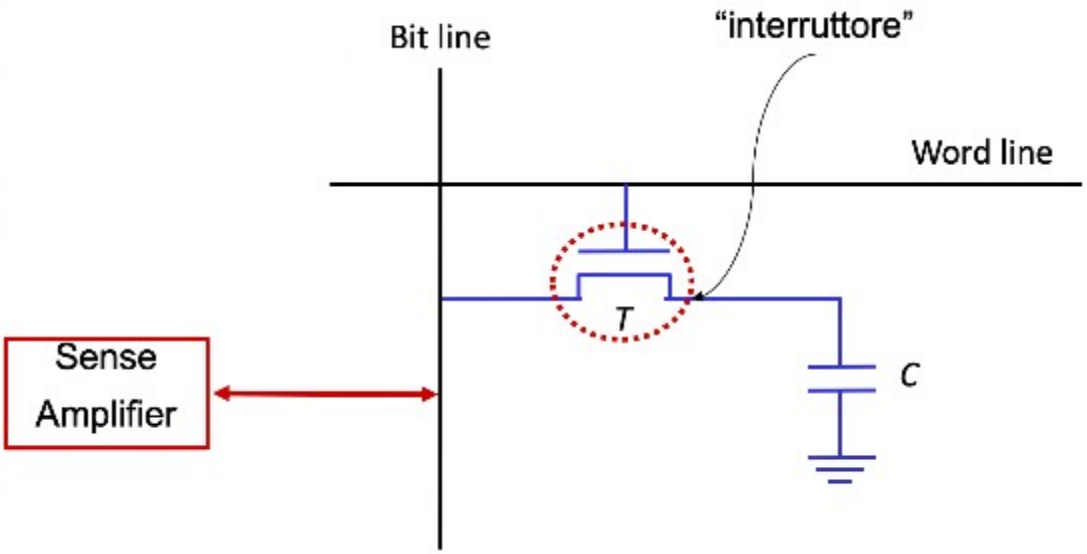


Dall'immagine qui sopra, che rappresenta una cella di memoria SRAM, notiamo che sia durante la lettura/scrittura, sia durante lo stato di mantenimento, abbiamo sempre due bit, uno normale e uno negato, così da ridurre il tasso di errori.
Durante la fase di **scrittura**, la linea di word è alta, e quindi "accendono" gli interruttori(T_1, T_2), che si chiudono e fanno passare il dato che viene così memorizzato nel LATCH a doppio nodo NOT.
Durante la fase di **lettura**, nuovamente la linea di word è alta, e i valori nei punti X e Y vengono copiati su b e b'.
Se la linea di word è bassa, gli interruttori sono aperti, e il consumo di energia è praticamente nullo.

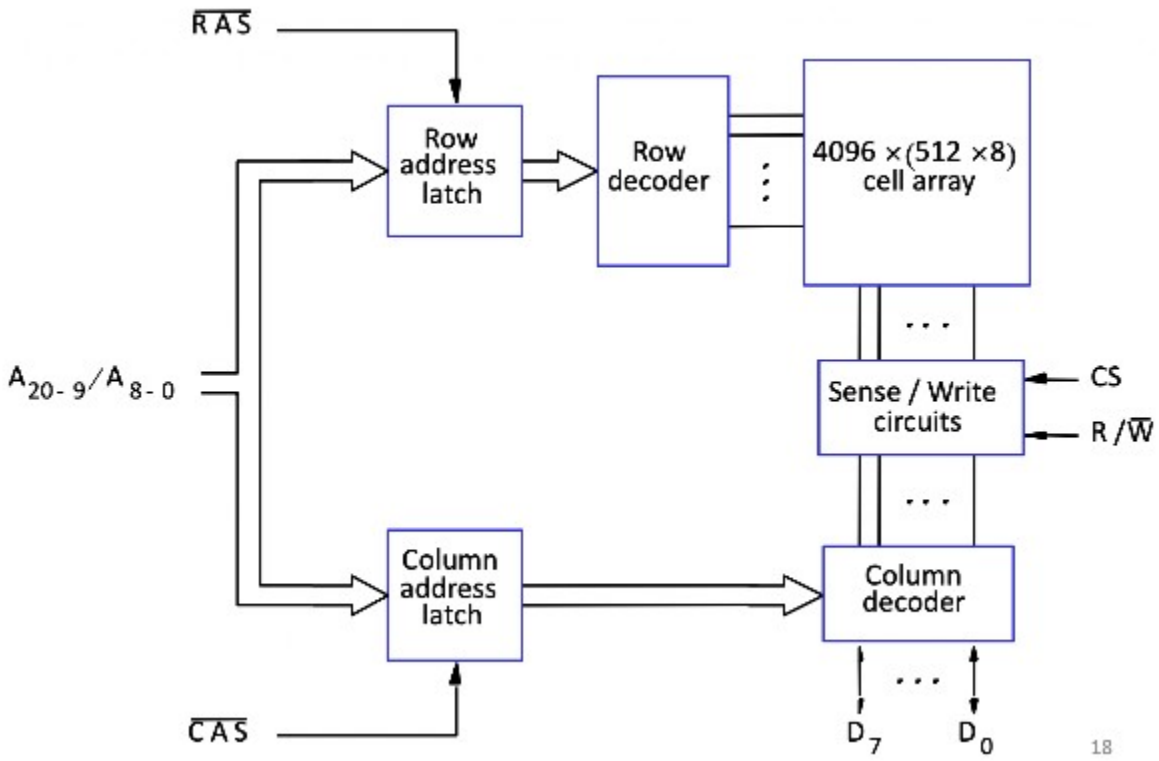
DRAM

Le DRAM sono le memorie più diffuse al giorno d'oggi, perché sono economiche e a densità elevatissima (1 componente (un condensatore) per cella, decisamente meglio di SRAM)
Però hanno bisogno di un aggiornamento (refresh) periodico perché in assenza di energia per troppo tempo il condensatore perde l'informazione. Questo refresh continuo causa consumi elevati

DRAM: cella di memoria

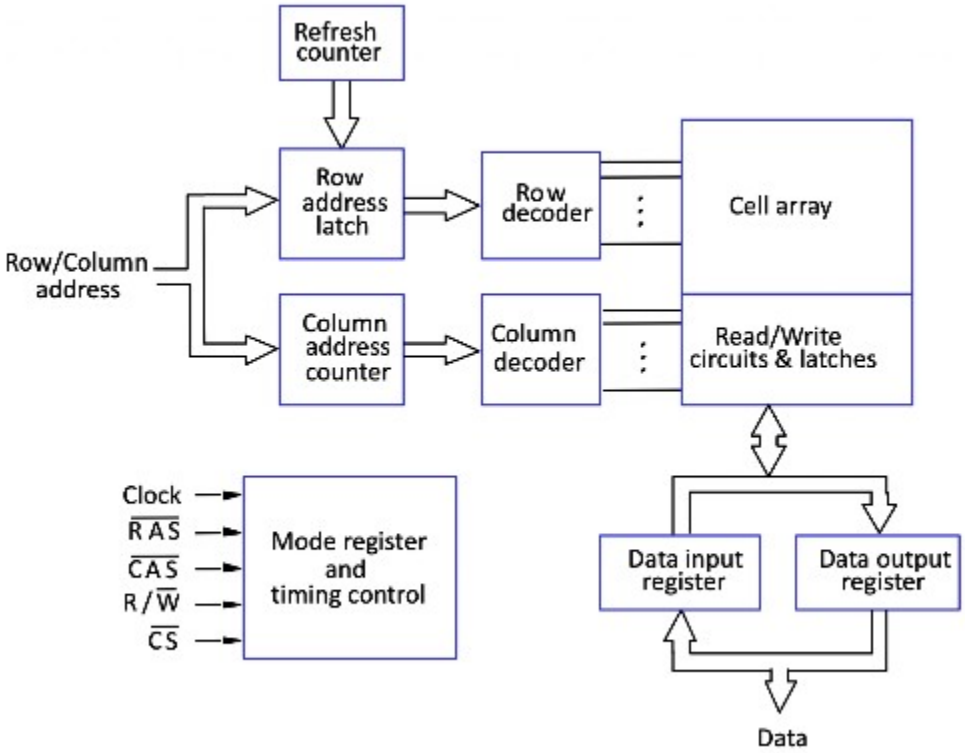


Durante la fase di **scrittura** la linea di word è alta e chiude T, quindi il valore presente su b viene copiato sul condensatore C (carica il transistor)
Durante la fase di **lettura** la linea di word e alta, e un apposito circuito (chiamato **sense amplifier**) misura la tensione su C. Questo dispositivo, oltre a leggere, anche amplifica la tensione, caricandola o scaricandola in base a cosa ha misurato.
Ogni volta che T viene aperto, il condensatore inizia a scaricarsi, ma basta fare un ciclo di lettura periodicamente, che come abbiamo visto refresha il condensatore, per continuare a mantenere l'informazione.
Siccome le DRAM hanno moltissime celle, e di conseguenza se per ogni cella si ha un pin di I/O, diventa difficile l'implementazione, si usa riferirsi a byte e non a singoli bit

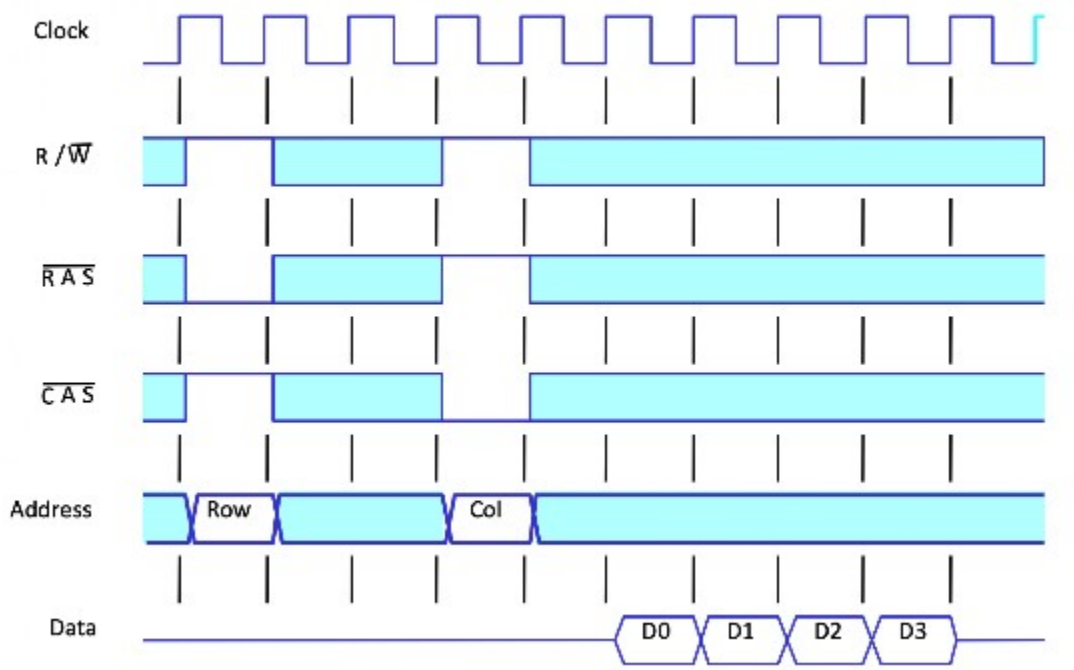


Siccome solitamente non si legge un singolo byte, ma blocchi di byte consecutivi, esiste una pratica chiamata **fast page mode** (**FPM**), nella quale si evite di “riselezionare” la riga ad ogni accesso (se consecutivo ovviamente), e quindi si fissa la riga facendo scorrere le colonne, così facendo si guadagna tantissimo in tempi di calcolo e tempo.

La DRAM nell’immagine qui sopra è detta **asincrona**, questo perchè non esiste una precisa temporizzazione, e la dinamica viene controllata dai segnali CAS e RAS; siccome in caso di rinfresco, questa potenziale “asincronicità” può essere fastidiosa, si possono aggiungere dei buffer (di memorizzazione degli ingressi e delle uscite) ottenendo un funzionamento **sincrono** (SDRAM) pilotato dal clock e per disaccoppiare lettura e scrittura dal refresh



La **latenza** misura il tempo di accesso a una singola parola. Questa è la misura “principe” delle prestazioni di una memoria e da un’idea di quanto il processore potrebbe aspettare un dato dalla memoria nel caso peggiore.
La **velocità** o **banda** misura la velocità di trasferimento massima in FPM (è anche importante per le operazioni in DMA, ovvero direct memory access, da parte delle periferiche, che accedono alla memoria senza passare dalla CPU).



Questa immagine mostra un accesso FPM a una DRAM.
Notiamo che il trasferimento dei dati avviene solo sul fronte positivo del clock.
Nelle **DDR-SDRAM** (ovvero Double Data Rate SDRAM) il trasferimento dei dati avviene sia sul fronte positivo che su quello negativo, lasciando invariata la latenza, ma raddoppiando la banda. Questo viene ottenuto organizzando la memoria su due banchi separati pari/dispari (locazioni contigue quindi sono in banchi separati, e quindi è possibile effettuare l'accesso in modo interlacciato)

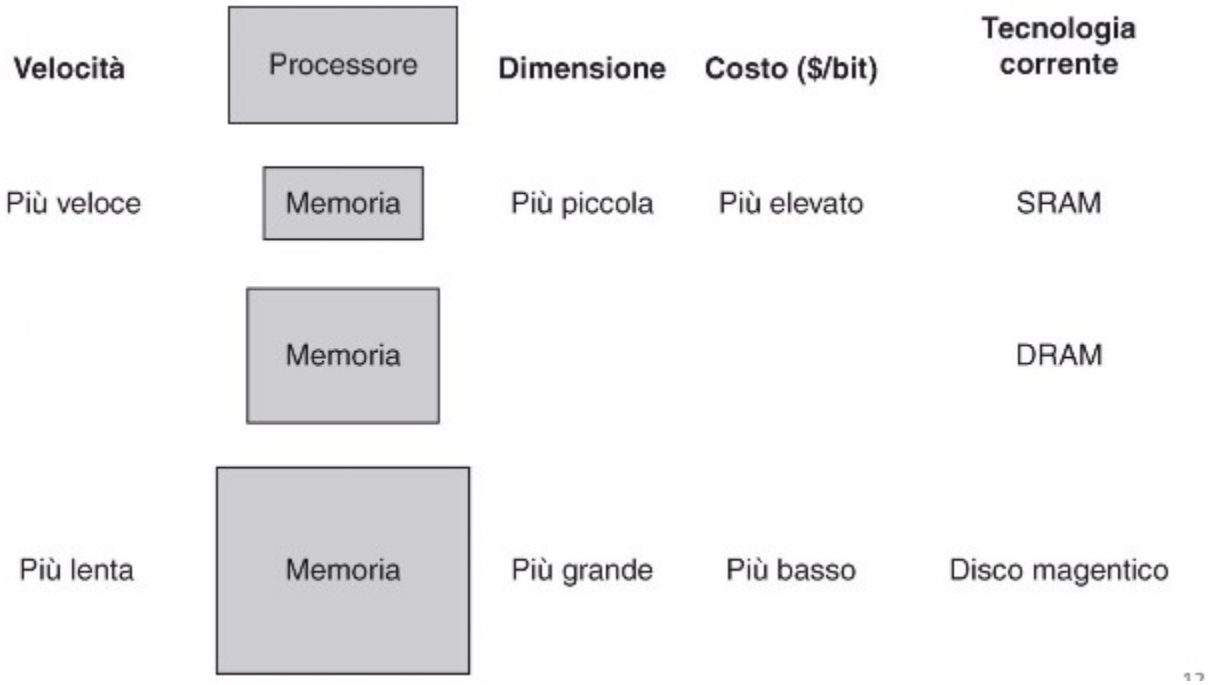
Gestione della memoria

Il problema più grande quando bisogna progettare una memoria è trovare il giusto compromesso tra la grandezza e la velocità.
Esiste un modo nei calcolatori per riuscire a dare l'impressione di avere a disposizione un grande spazio e una alta velocità; questo è possibile grazie a due principi

- Principio di località spaziale
  Quando si fa uso di una locazione di memoria, nei passi successivi si farà riferimento a locazioni vicine (quando si leggono le istruzioni una dopo l'altra, quando si scorre un array)
- Principio di località temporale
  Quando si fa uso di una locazione di memoria, ci sono alte probabilità che venga riutilizzata in tempi brevi (istruzioni di un ciclo)

Tecnologia di Memoria	Tempo di accesso tipico	$ per GB (2010)	$ per GB (2012)
SRAM	0.5-2.5 ns	$2000 - $5000	$500 - $1000
DRAM	50 – 70 ns	$20 - $75	$10 - $20
Memoria flash	70 – 150 ns	$4 - $12	$0.75 - $1
Dischi magnetici	5 000 000 – 20 000 000 ns	$0.2 - $2	$0.05 - $0.1

Queste cifre suggeriscono l'idea della gerarchia della memoria, quindi memoria piccole ma veloci saranno posizionate vicine al processore e memorie grandi ma lente saranno posizionate più lontano



L'idea quindi è di far scorrere al processore le memorie, dalla più vicina alla più lontana, fino a che non si trova l'informazione.
Un **blocco** è l'unità minima che può essere presente o assente in ciascun livello (byte, word, double word).
L'**hit rate** è la frequenza (probabilità) di successo di trovare il dato nel livello superiore (o più vicino)
Invece la **miss rate** è la frequenza (probabilità) di insuccesso di trovare il dato nel livello superiore, quindi la frequenza con cui devo cercare il dato in livelli più lontani, quindi si trova facendo $1 - Hit\ \ Rate$
Il **tempo di hit** è il tempo nella quale accedo al dato quando l'ho trovato nel livello superiore.
La **penalità di miss** è il tempo nella quale accedo al dato quando non lo trovo nel livello superiore, quindi è il tempo per spostare il dato nel livello superiore sommato al tempo di hit.

Nella pratica il tempo di hit misura con che velocità accedo ad un dato della cache, invece la penalità di miss misura il tempo nella quale prima trasferisco il dato dalla ram alla cache, e poi lo leggo dalla cache.
Ovviamente la penalità di miss è molto più alta rispetto al tempo di hit. Quindi riducendo la miss rate, posso guadagnare molto in termini di tempo, e dobbiamo sfruttare i principi di località!

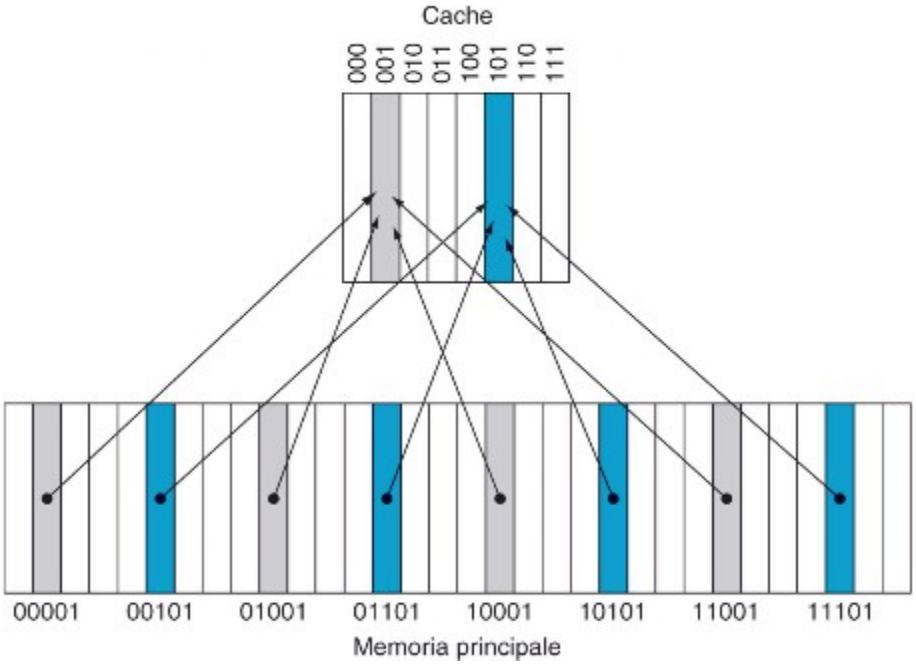
Cache

La cache è un posto nascosto dove riporre le cose, nascosto perchè il programmatore non la vede e non vi ci può accedere, quindi l'uso della cache è interamente trasparente
Esistono due tipi di cache:

- A mappatura diretta
- Completamente associative

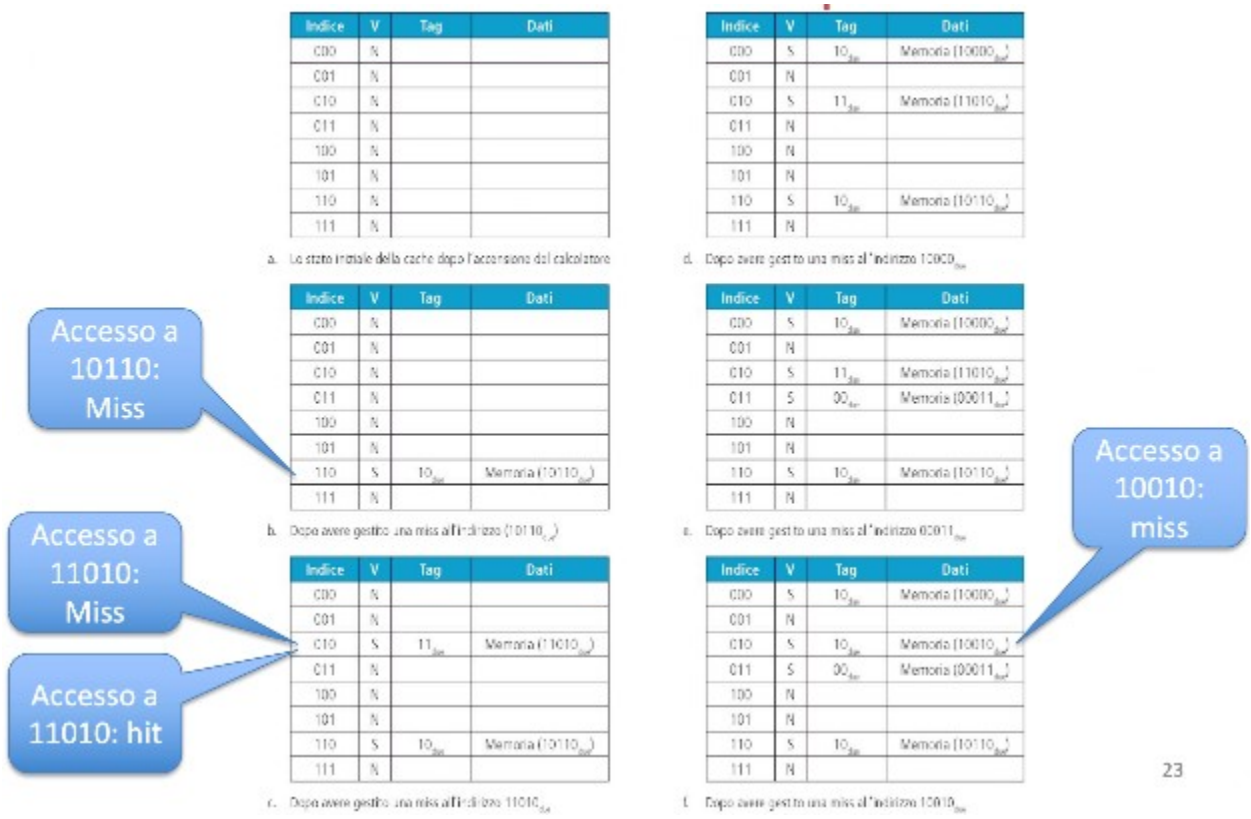
Mappatura diretta

Nella cache a **mappatura diretta** abbiamo che ogni singolo indirizzo della memoria principale, corrisponde ad una precisa locazione della cache.
Per poter implementare ciò, essendo la cache più piccola della memoria principale, dobbiamo applicare un meccanismo come il modulo (in base al numero di blocchi in cache) per far si che più indirizzi della memoria principale si riconducano allo stesso blocco della cache.
Notiamo che se il numero di blocchi della cache è una potenza di due, possiamo prendere i bit meno significativi dell'indirizzo di memoria.
Ad esempio se abbiamo una cache di 8 blocchi, basta prendere i 3 bit meno significativi (perché $2^3 = 8$):



Il problema è: come facciamo a capire se il blocco su cui è mappato il nostro indirizzo, contiene effettivamente il valore del nostro indirizzo e non quello di un altro con lo stesso modulo?

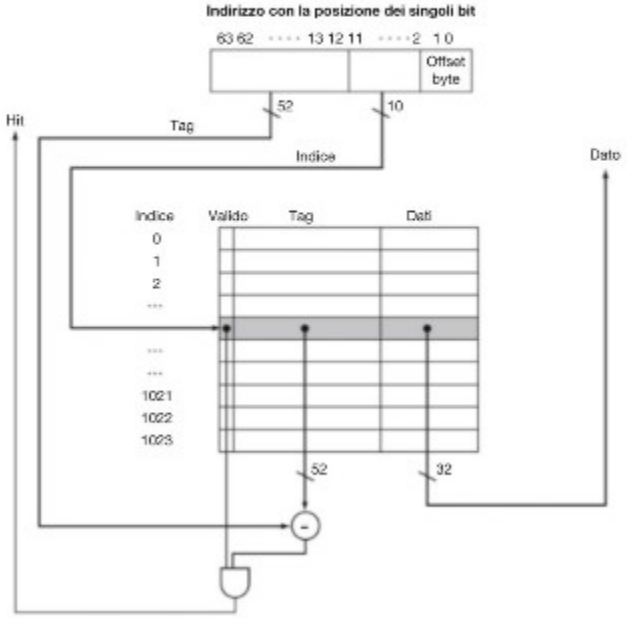
Si ricorre a un campo detto **tag**, che contiene la prima parte dell'indirizzo il cui valore è salvato nel blocco; nell'esempio qui sopra il tag sarà composto da due bit.
In più viene aggiunto un bit di validità, che ci dice se quello che è memorizzato in un blocco sia valido in quel determinato momento (potrebbe non essere valido per vari motivi e in quel caso bisogna rileggerlo nella memoria periferica)



- Esempio
  - Indirizzo su 64 bit
  - Cache a mappatura diretta
  - Dimensioni della cache: 2^n blocchi, di cui n bit usati per l'indice
  - Dimensione del blocco di cache: 2^m parole, ossia 2^{m+2} byte (m bit usati per individuare una parola nel blocco), due bit per individuare un byte in una parola

In questo caso la dimensione del tag è data da:
64-(n+m+2)

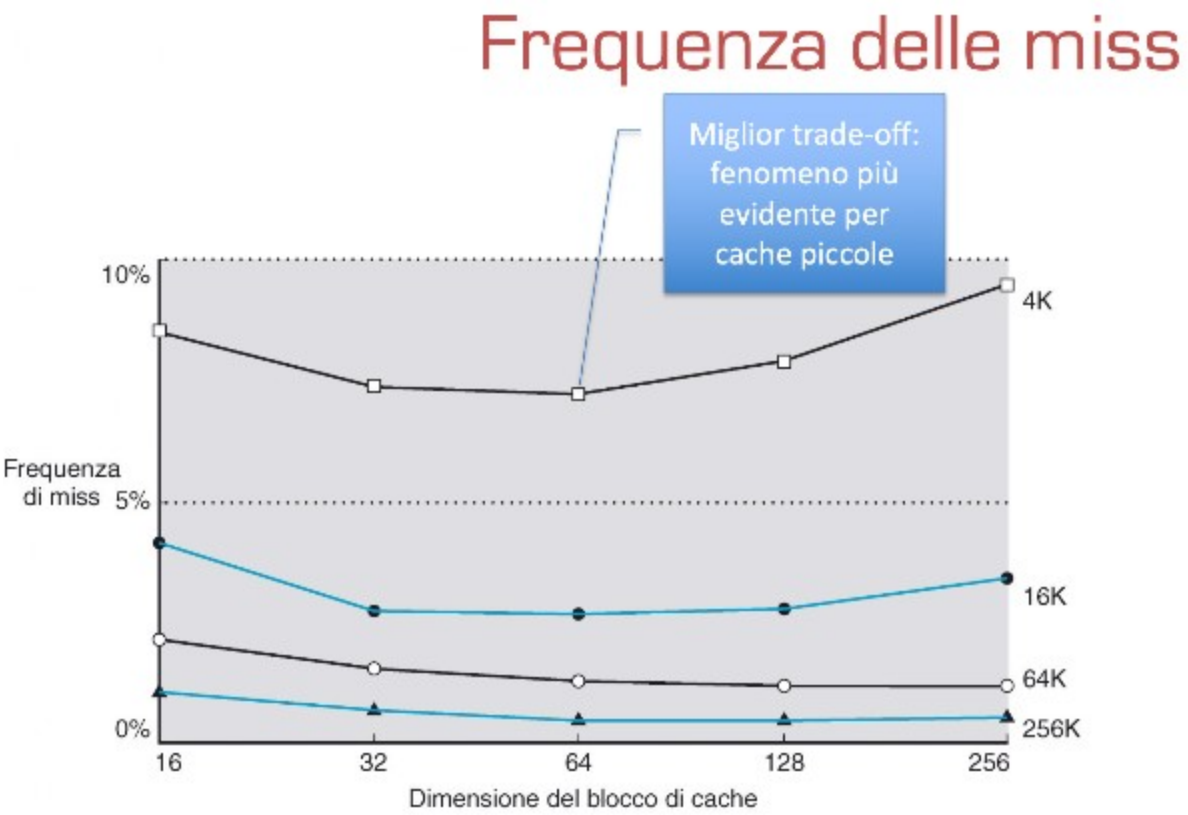
- Un indirizzo viene risolto in cache con il seguente schema (n=10, m=0, dimensione tag = 52):



Se il campo tag è uguale ai 52 bit più significative dell'indirizzo e se il bit di validità è 1, allora si ha una hit e si da il dato al processore, altrimenti scatta una miss.

25

Bisogna ovviamente fare un trade-off. I blocchi di cache molto grandi, i quali esaltano la località spaziale e diminuiscono le probabilità di miss, hanno dei vantaggi, però, a parità di dimensioni totali della cache, averli di grandi dimensioni, e quindi pochi blocchi, diminuiscono l'efficacia dello sfruttamento della località temporale. Bisogna anche considerare che delle miss con blocchi grandi portano a un elevato costo di gestione.



La presenza della cache non modifica di molto il funzionamento del processore quando abbiamo delle hit, perché il processore non si accorge nemmeno della presenza della cache. Però nel caso delle miss, bisogna generare uno stallo nella pipeline e gestire il trasferimento dalla memoria principale alla cache

- Ad esempio, per una miss sulla memoria istruzioni, bisognerà:
  1. Inviare il valore PC – 4 alla memoria (PC viene incrementato all'inizio, quindi la miss è su PC-4)
  2. Comandare alla memoria di eseguire una lettura e attenderne il completamento
  3. Scrivere il blocco che proviene dalla memoria della cache aggiornando il tag
  4. Far ripartire l'istruzione dal fetch, che stavolta troverà l'istruzione in cache

Gli accessi in lettura quindi li abbiamo appena visti, ecco come funzionano invece gli accessi in scrittura, i quali possono generare problemi di consistenza in quanto potrebbe succedere che si aggiorni solo il dato nella RAM e non nella CACHE o viceversa:

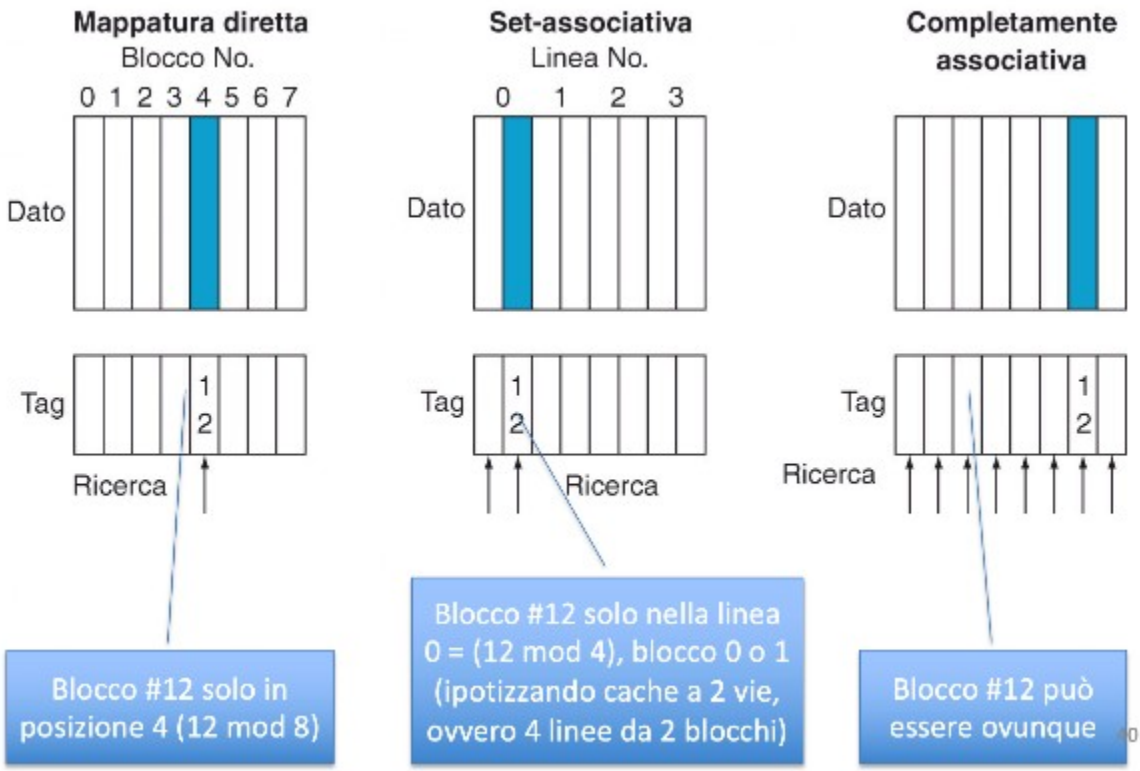
- **Write-through**
  Ogni scrittura viene effettuata direttamente in memoria principale
  Non ho problemi di consistenza ma le scritture sono molto costose
  Ovviamente devo settare coerentemente il bit di controllo nella cache
- **Write-back**
  Le scritture vengono fatte localmente in cache e l'update nella memoria principale viene eseguito solo quando il blocco viene rimpiazzato
  Molto comodo ad esempio per un indice di un ciclo
  Conveniente quando le scritture nel metodo precedentemente richiedono troppe risorse e il processore non riesce a "stargli dietro"

Associative

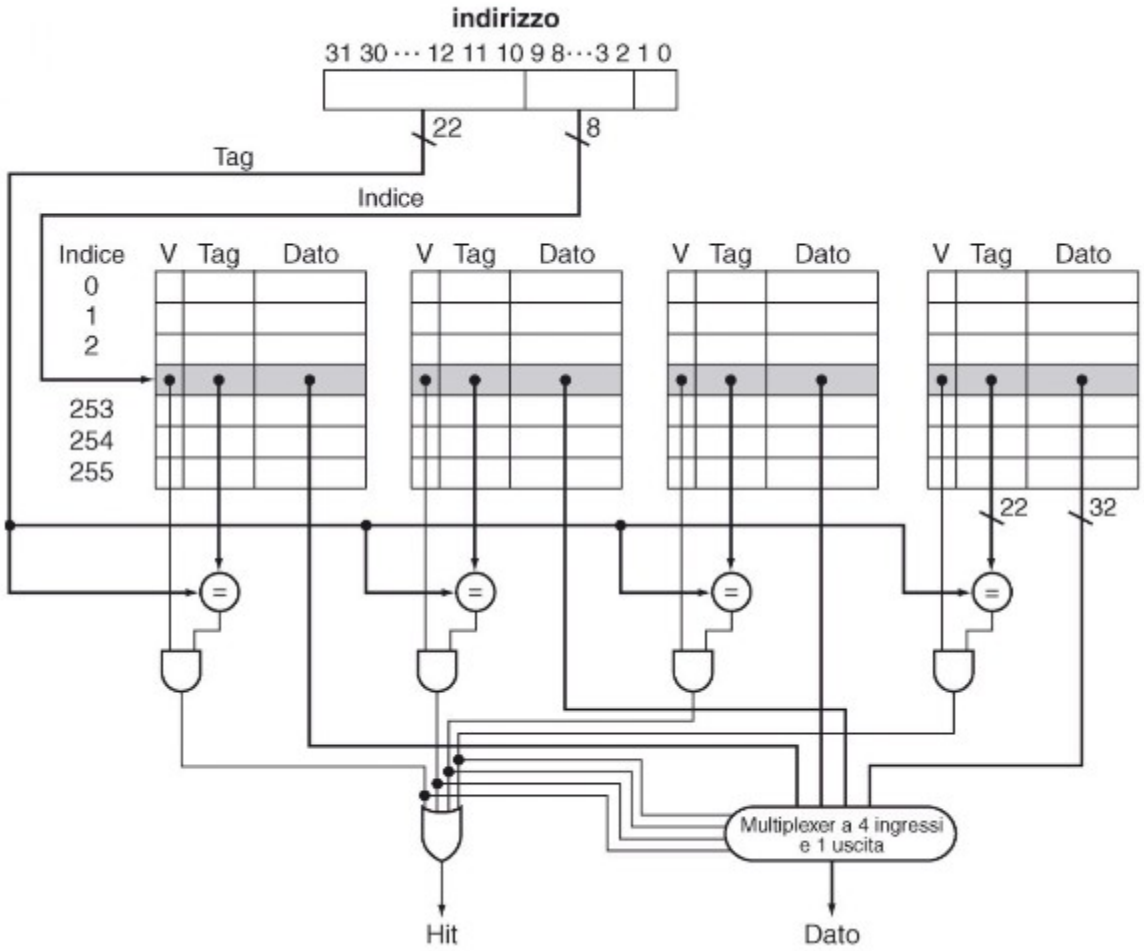
Un problema delle cache a mappatura diretta è quello che se ho spesso bisogno di locazioni di memoria che si mappano sullo stesso indirizzo, ho delle cache miss in continuazione.
Nella modalità completamente associativa, qualsiasi indirizzo può essere mappato in un qualsiasi blocco della cache, quindi il tag è composto da tutto l'indirizzo e quindi devo scorrere tutta la cache per trovarlo.
Per farlo in maniera efficiente dovrei controllare tutti i blocchi in parallelo ma questa operazione è molto costosa (devo avere n comparatori, uno per ogni blocco della cache).

Set-associative

Esistono le cache set-associative, che sono una via di mezzo tra le due modalità che abbiamo visto. Quindi ogni blocco di memoria può essere mappato in una linea di n blocchi diversi di cache (ha quindi n vie all'interno della linea). All'interno di ogni linea effettueremo una ricerca parallela associativa.
Quindi per trovare la linea di una locazione di memoria faremo l'operazione
$(Indirizzo\ \ di\ \ blocco)\ \ modulo\ \ (Numero\ \ di\ \ linee\ \ della\ \ cache)$



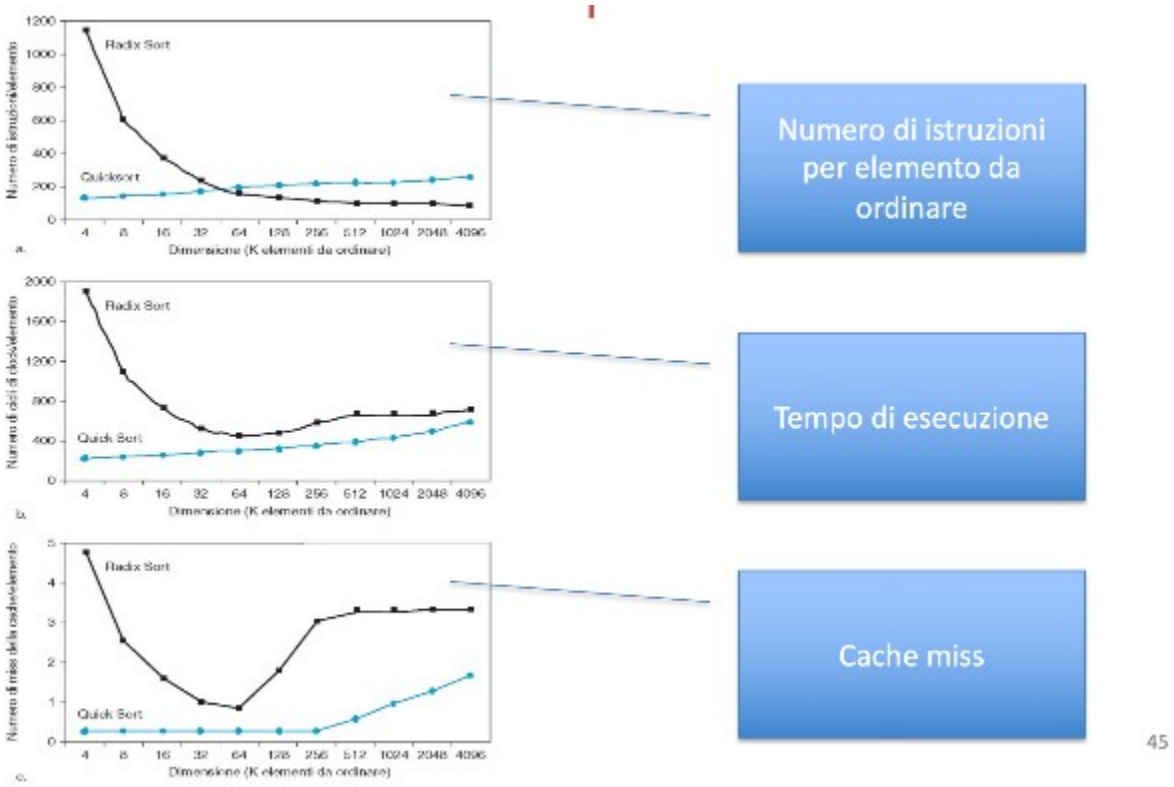
Una cache associativa a una via (solo un blocco per linea) diventa una cache a mappatura diretta, invece una cache associativa a n vie, con una sola linea, diventa completamente associativa.



Aumentando l'associatività (cioè quante vie) abbiamo vantaggi, come la frequenza di miss, e svantaggi, come la complessità, come sempre bisogna trovare un trade off

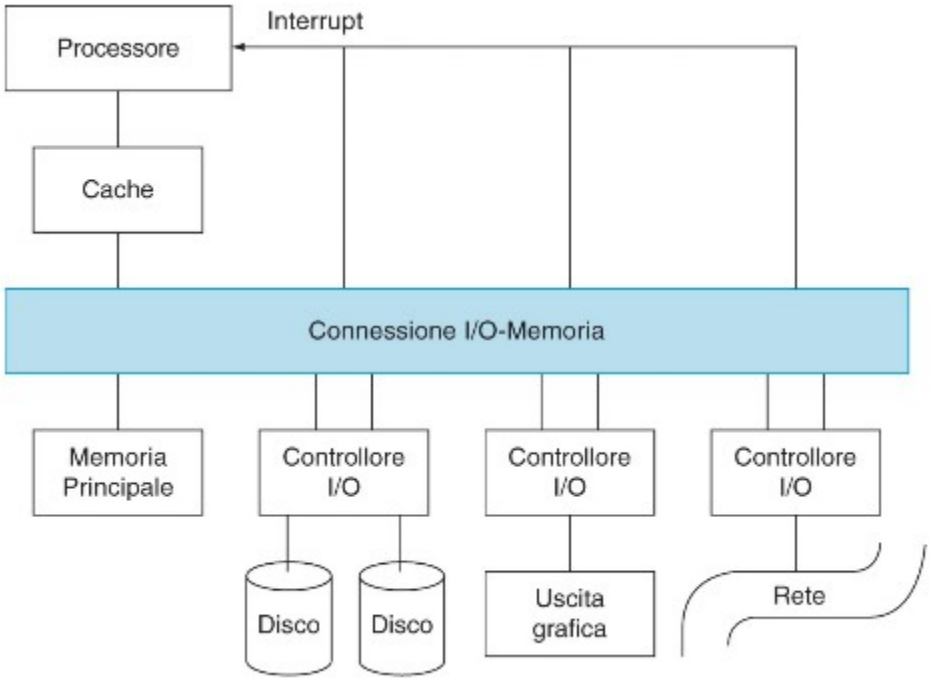
In una cache a mappatura diretta nel caso di una miss so sempre chi sostituire (l'unico blocco su cui vengo mappato).
Al contrario, in una cache associativa, posso operare in più modi per scegliere quale elemento della cache "eliminare":

- FIFO
- Last Recently Used (richiede una serie di bit in più per calcolare l'ultimo accesso)



Dispositivi I/O

Un calcolatore ovviamente sarebbe inutile senza la possibilità di trasmettere dati dall'interno all'esterno e viceversa.
Questi dispositivi, chiamati dispositivi di Input/Output, sono **espandibili** (cioè si deve avere la possibilità di aggiungerne altri) ed **eterogenei** (possono essere diversi tra loro, sia come tecnologia sia come dati su cui lavorano), ed ovviamente in base allo scopo della periferica, siamo interessati a tipi diverso di prestazioni, ad esempio la latenza per tastiera e mouse oppure il throughput per le memorie e le interfacce di rete.
Questi dispositivi sono collegati al processore da un dispositivo di comunicazione chiamato **bus**.



I dispositivi di I/O hanno la necessità di poter leggere e scrivere dalla memoria principale (RAM), perchè è l'unico modo per poter far leggere i dati dal processore.
I dispositivi I/O hanno due linee perchè una è riservata ai dati e una al controllo

I dispositivi di I/O possono essere classificati in base:

- Comportamento
  Quali operazioni può effettuare (Read o Write)
- Partner
  Con chi è che "comunica" la macchina con quel dispositivo
- Velocità di trasferimento

In base al tipo di applicazione su cui operano, sono interessato a diversi aspetti (throughput, latenza, concorrenza)

Esistono due tipi di bus:

- Bus processore/memoria
  Specializzati, corti e veloci
- Bus dispositivi I/O e memoria
  Possono essere lunghi e permettono il collegamento con periferiche eterogenee.
  Tipicamente non sono collegati alla memoria in maniera diretta ma si appoggiano ad un altro bus (talvolta direttamente quello processore/memoria)

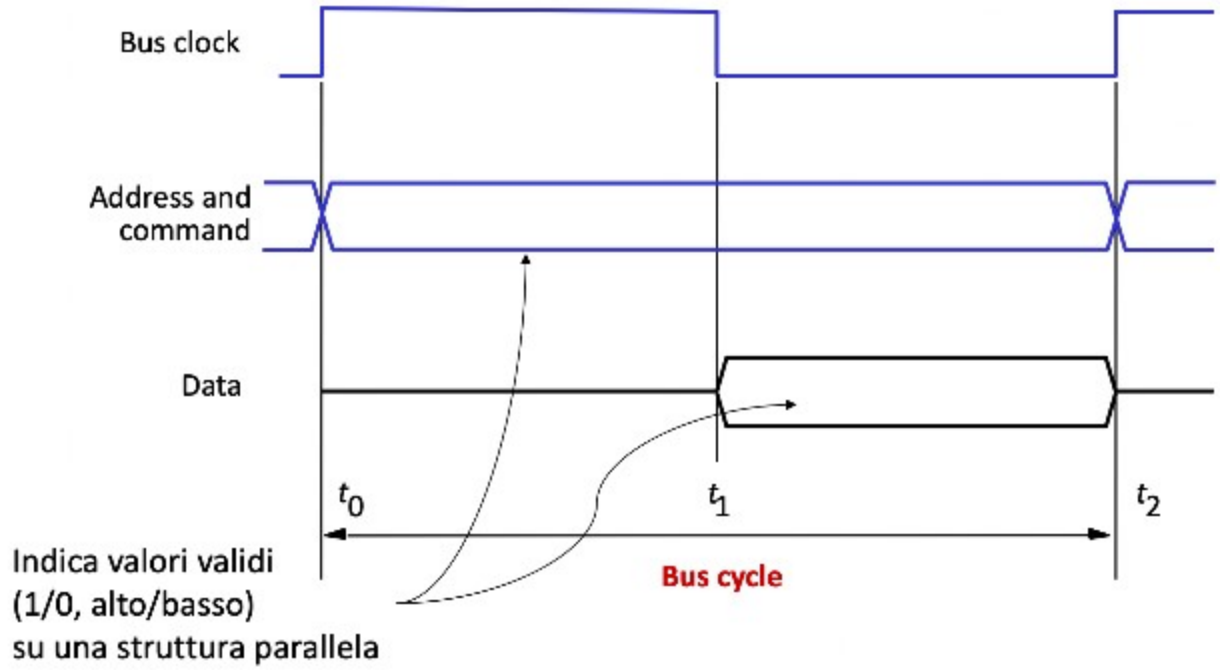
Nelle prime architetture esistevano anche i bus di sistema, che collegavano tutto, ma ora si preferisce questa divisione di più bus paralleli per ovviare a problemi di clock e frequenze

Una **transazione i/o** è la spedizione/ricezione dell'indirizzo e dei dati
Un **input** è un trasferimento di dati da una periferica alla memoria principale

Un **output** è un trasferimento di dati dalla memoria principale alla periferica

Bus sincrono

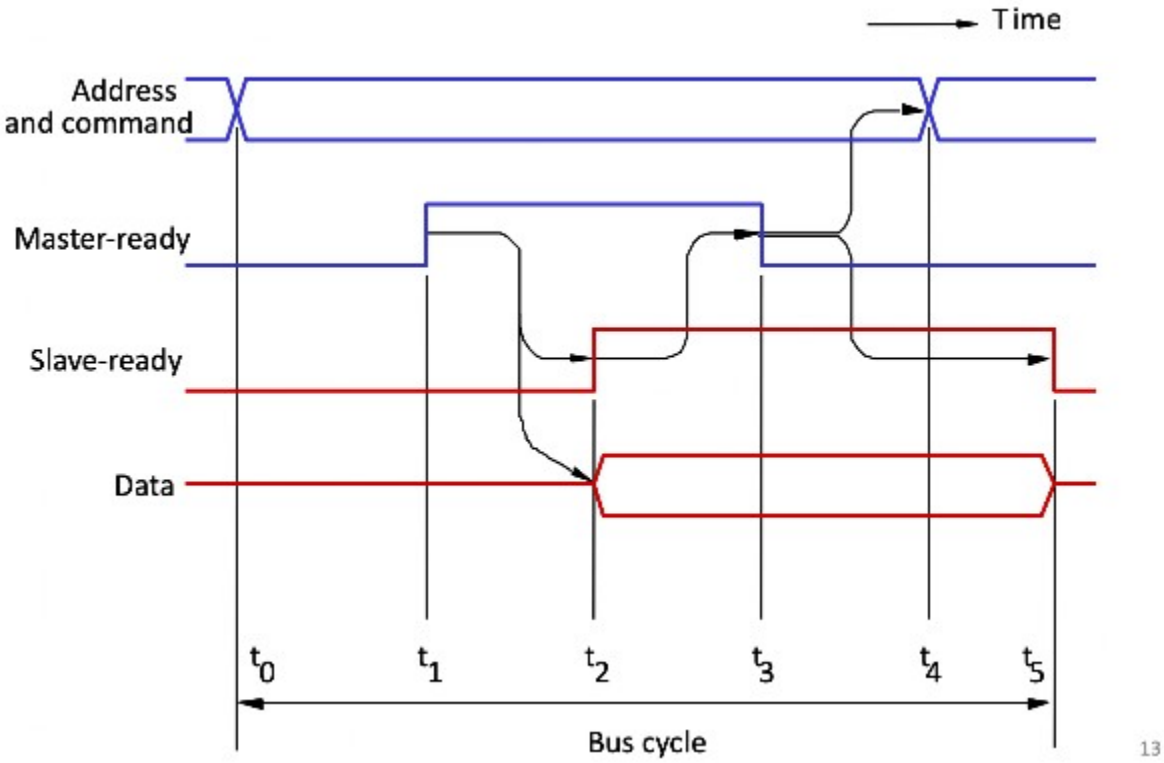
In questa tipologia di bus, tra le linee di controllo è presente un clock, il quale definisce il tempo delle operazioni.



I pro di questa tipologia sono la semplicità di implementazione e la velocità (ci sono pochi segnali di controllo)
I contro sono la poca robustezza e il fatto che tutte le periferiche dipendono dalla velocità del clock.

Bus asincrono

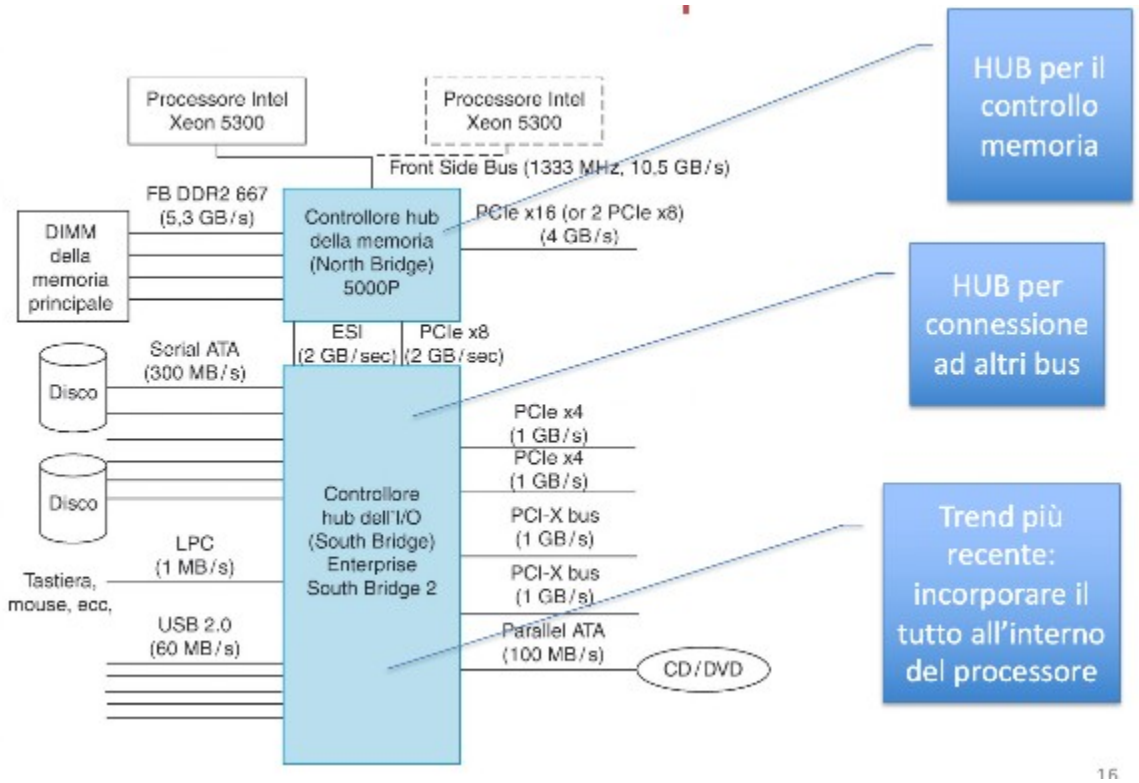
In questa tipologia, per evitare i problemi discussi precedentemente, le periferiche non si affidano più ad un clock, ma ad una serie di segnali di **handshake**, infatti si introducono delle apposite linee di controllo per poter segnalare inizio e fine di transazioni.
Così facendo si possono collegare periferiche che vanno a velocità diverse, anche se abbiamo un costo di implementazione più elevato



Master (legato al processore) e slave (legato alla periferica) indicano quando il dispositivo è pronto (ready) a ricevere/spedire l'informazione

I pro di questa tipologia sono la robustezza ai ritardi e la possibilità di comunicare con periferiche di diverso tipo e tempistiche
I contro sono che tutti i segnali di controllo rallentano le interazioni e la circuiteria diventa di conseguenza più complessa

Spesso si usano tecnologie ibride, cioè prevalentemente asincrone ma in cui c'è comunque un segnale di clock.



Osserviamo che dal lato del sistema operativo abbiamo svariati problemi:

- I programmi che condividono lo stesso processore, condividono anche le stesse periferiche
- I trasferimenti dati vengono eseguiti usufruendo delle interrupt (che verranno spiegate dopo), con impatto sulle funzionalità del sistema operativo
- Il controllo di operazioni I/O si interseca con problematiche di concorrenza

Il SO deve garantire agli utenti, che hanno i permessi, di accedere ai dispositivi I/O, deve fornire comandi di alto livello per le operazioni a basso livello, deve gestire le interruzioni generate dai dispositivi I/O, e deve ripartire l'accesso alle periferiche ai vari programmi in maniera equa.

Per poter eseguire le operazioni sopra elencate bisogna:

- Rendere possibile al sistema operativo di inviare comandi alle periferiche
- Rendere possibile ai dispositivi di notificare la corretta esecuzione di un'operazione
- Consentire trasferimenti diretti tra periferica e memoria

Per poter comunicare con le periferiche bisogna mettere a disposizione alcune "parole di controllo", e lo si può fare in due modi:

- Scrivendo/leggendo in particolari locazioni di memoria (**memory mapped I/O**)
  In queste locazioni, non accessibili ai programmi utente ma solo al sistema operativo, possono essere scritti comandi che il controllore della I/O leggerà e eseguirà, e in altre locazioni la periferica potrà scrivere dei dati di ritorno o di stato che verranno poi letti dal sistema operativo.

- Facendo uso di alcune istruzioni speciali dedicate alla I/O

La modalità più semplice per questo trasferimento di dati è detta **polling**, ovvero attesa attiva, e consiste nel mandare un comando alla periferica e aspettare (facendo un ciclo di attesa sul bit di stato) fino a che l'operazione non è stata portata a termine dalla periferica

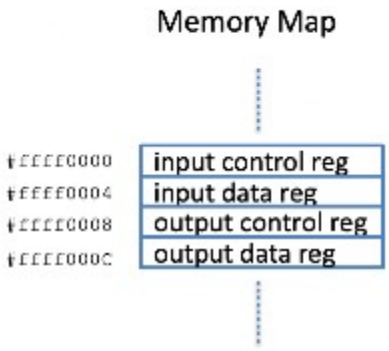
- Input: lettura dalla tastiera in x7

```
          lui  x5, 0xffff #ffff0000
Waitloop: lw   x6, 0(x5) #control
          andi x6, x6,0x0001
          beq  x6, x0, Waitloop
          lw   x7, 4(x5) #data
```

- Output: stampa del dato da x7

```
          lui  x5, 0xffff #ffff0000
Waitloop: lw   x6, 8(x5) #control
          andi x6,x6,0x0001
          beq  x6,x0, Waitloop
          sw   x7, 12(x5) #data
```

- Questo ciclo di attesa è chiamato *polling*



Costo del polling

- Consideriamo un processore a 500Mhz e supponiamo che occorrano 400 cicli di clock per un'operazione di polling. Qual è il costo percentuale?
  - Esempio 1: Mouse. Per non perdere movimenti da parte dell'utente occorre acquisire il dato 30 volte al secondo.
  - Esempio 2: Hard disk. I dati vengono trasferiti in blocchi di 16 byte a 8MB/s senza la possibilità di perdite.

Esempio 1 (Mouse)

- Cicli di clock al secondo spesi per il polling

  = 30 * 400 = 12000 clocks/sec

  - % Processor for polling:

    $12*10^3/500*10^6 = 0.002\%$

    $\Rightarrow$ Fare polling sul mouse «ruba» un utilizzo di processore trascurabile

Questo overhead viene pagato sempre, sia che ci sia il trasferimento, sia che non ci sia

Esempio 2 (Hard disk)

- Numero di volte/sec che occorre fare cicli di attesa per non perdere dati:

  = 8 MB/s /16B = 500K polls/sec

- Spesa in cicli di clock/sec

  = 500K * 400 = 200,000,000 clocks/sec

- % processore

  $200*10^6/500*10^6 = 40\%$

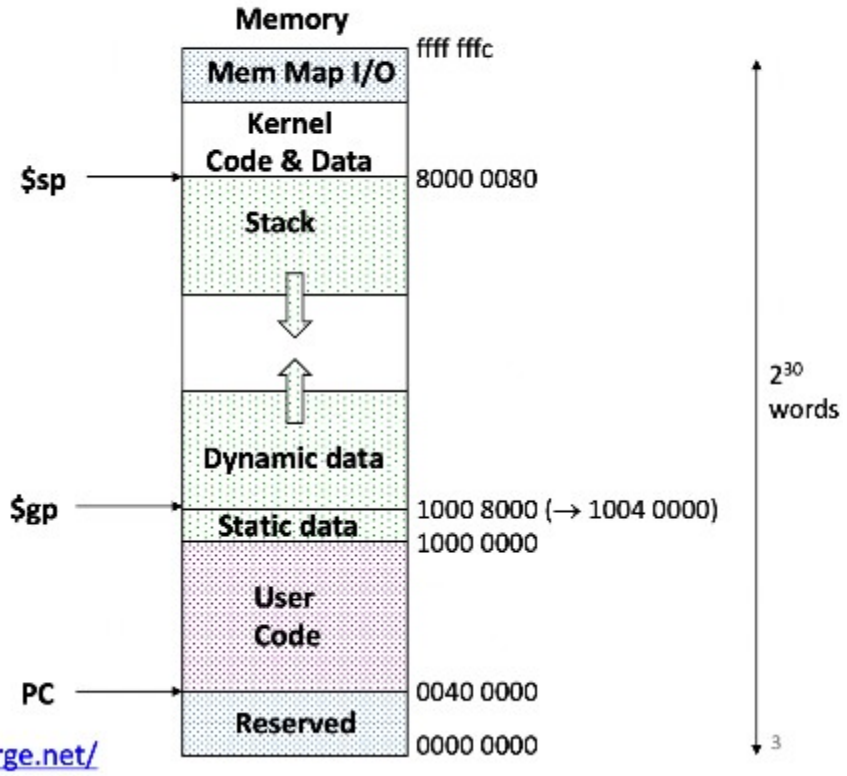
  $\Rightarrow$ Inaccettabile perché pagata sempre

Interrupt

Siccome il polling (attesa attiva) fa perdere molto tempo e risorse, viene utilizzato solo nelle operazioni di controllo, nelle quali ho una velocità di trasferimento predeterminata e ho pochi dati da dover passare (solo dei dati di controllo appunto).
In altri casi però questo è inaccettabile, perché lo spreco è troppo elevato, quindi è stato inventato l'I/O a **interruzione** del programma.

Organizzazione memoria SPIM (MIPS)

- Partiamo da una tipica organizzazione di memoria (ad es. quella dell'emulatore SPIM)



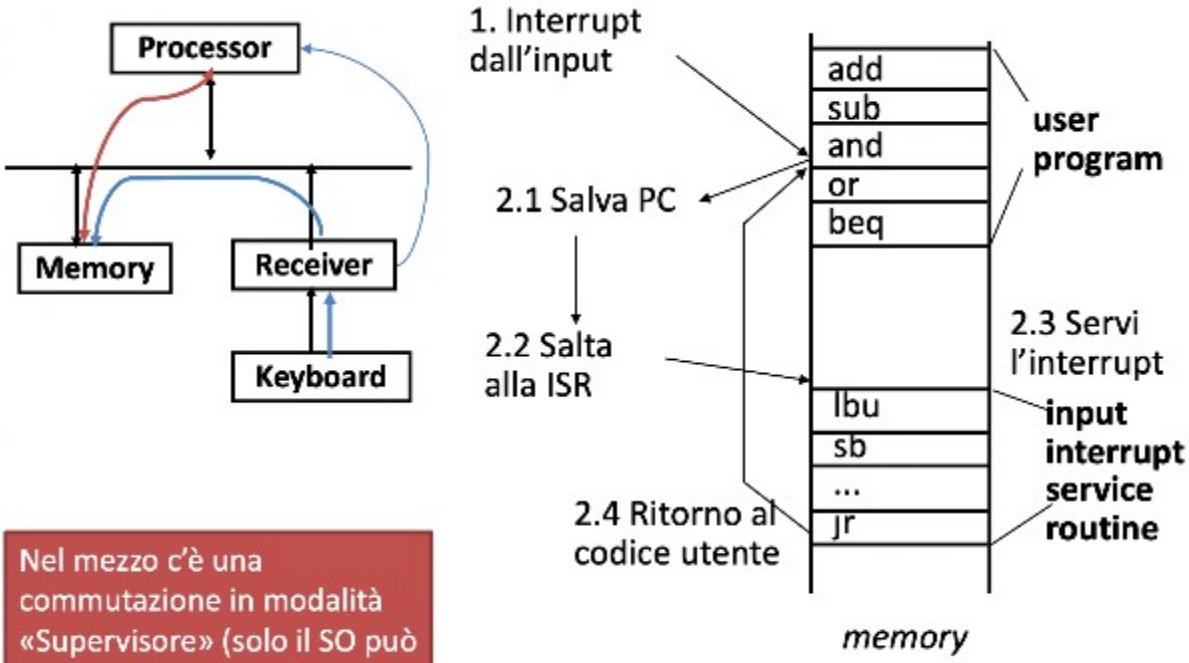
http://spimsimulator.sourceforge.net/

Controllo del terminale in SPIM (INPUT)

- Osserviamo da vicino le locazioni per il controllo del terminale (INPUT)



Un'interruzione quindi è un segnale che il processore “riceve” quando una periferica è pronta ad eseguire il trasferimento richiesto.
Si può però dare un certo livello di priorità a questi segnali e bisogna capire quale periferica ha richiesto l’interruzione.
Le interruzioni sono asincrone rispetto all’esecuzione delle istruzioni, nel senso che può avvenire in un qualsiasi momento e il processore deve essere in grado di gestirla senza “perdere” nulla degli altri programmi (anche se si possono creare delle sezioni non interrompibili nel codice del kernel).
I vantaggi sono che il programma viene interrotto solo quando effettivamente è pronto il dato.
Gli svantaggi sono che serve un hardware speciale per permettere alle periferiche di generare un'interruzione, per permettere al SO di rilevarla, salvare lo stato del processore (ISR, Interrupt Service Routine) per eseguirla e riprendere lo stato del processore precedente, dal punto dove si era interrotto



Nel mezzo c'è una commutazione in modalità «Supervisore» (solo il SO può gestire l'interruzione)

Esempio controllo terminale SPIM

1. La periferica indica con un'interruzione che ha un nuovo carattere dalla tastiera nell'opportuno registro di ricezione



- ✓ Contestualmente il bit pronto viene messo a uno nel registro di controllo



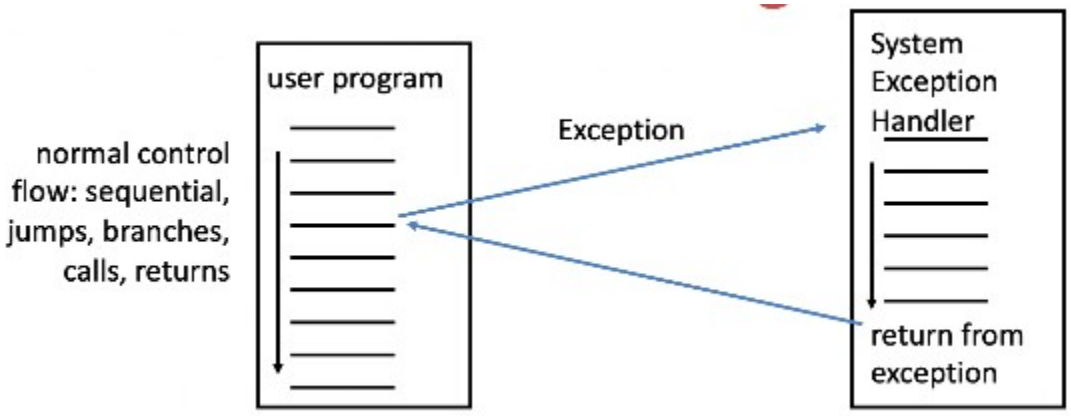
2. Il processo utente viene interrotto trasferendo il controllo a una ISR che copia il dato in memoria utente
   - ✓ All'atto della lettura il bit ready viene ri-azzerato
   - ✓ Notare che prima di effettuare il trasferimento il bit interrupt enable era stato posto a 1 per abilitare le interruzioni

- Ritorniamo all'esempio di prima dell'hard disk e supponiamo che un interrupt costi 500 cicli di clock (plausibile che costi di più del polling)
- Se le interruzioni vengono generate alla frequenza di polling
  - Disk Interrupts/sec = 8 MB/s /16B
    = 500K interrupts/sec
  - Disk Polling Clocks/sec = 500K * 500
    = 250,000,000 clocks/sec
  - % Processor: $250*10^6/500*10^6$= 50%
  - Sembrerebbe che non ci sia guadagno…anzi
- Tuttavia se l'hard disk è attivo solo per il 5% del tempo, gli interrupt generati saranno il 5% e la spesa di processore sarà:
  5% * 50%=2.5%

In questo caso, come si vede dall'esempio, anche se la "mole" di istruzioni, di uso del processore, cresce, in generale si guadagna molto, visto che quella percentuale della CPU verrà occupata solo quando ci sarà un interrupt, e non costantemente come nel polling, a prescindere se la periferica veniva usata oppure no. Quindi in media è molto più conveniente usare le interrupt per questo tipo di periferiche.

Eccezioni

Un'**eccezione** è un trasferimento non programmato del controllo del programma, e il sistema deve attuare delle azioni per poterla gestire, per poi tornare al punto del programma precedente.



Esistono tre tipi di eccezioni:

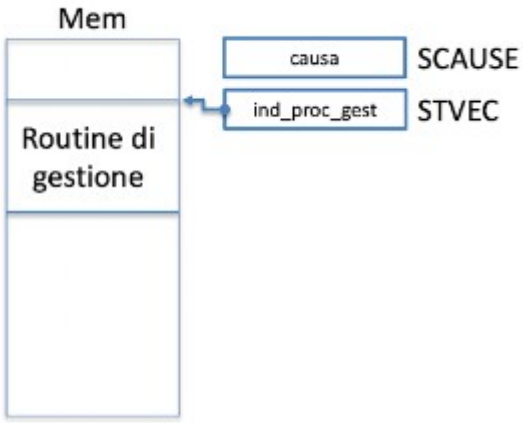
- Interrupts
  Causate da eventi delle periferiche esterne
  Asincrone
  Sospendono il programma e lo riprendono nel punto in cui è stato interrotto
  Sono gestibili in mezzo a due istruzioni
- Traps (eccezioni)
  Causate da eventi interni al programma (overflow, errori…)

Sincrone rispetto all'esecuzione del programma
Si può provare a rieseguire l'istruzione che l'ha causata oppure si può abortire il programma

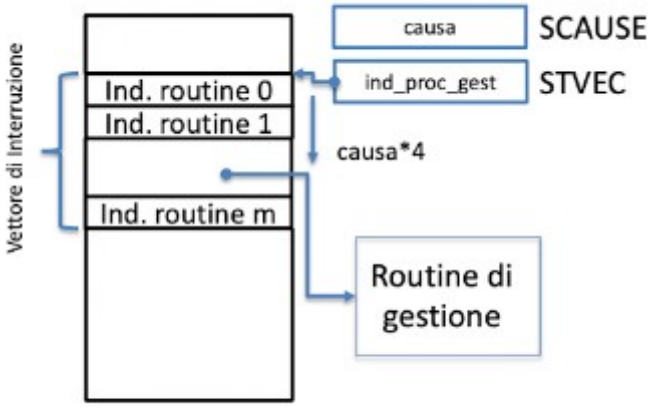
- Environment call / break
  Environment call (**ecall**): causata da una esplicita richiesta di un servizio di sistema
  Environment break (**ebreak**): causata da una esplicita chiamata per motivi diagnostici o di debug

Ogni volta che viene generata un'eccezione, lo stato della macchina viene salvato (salvando il contenuto dei registri etc); successivamente esistono due metodi principali per gestirla:

- Salto diretto all'indirizzo della routine di gestione
  In questa situazione, per ogni eccezione viene chiamata la stessa routine di gestione, la quale ne analizzerà la causa e agirà in base a questo.



  In questa situazione non dobbiamo fare nessun accesso in memoria per prelevare l'indirizzo di routine, siccome è statico ed è già presente nell'apposito registro $STVEC$.
  La causa dell'interruzione invece è contenuta nel registro $SCAUSE$
- Vettore di interruzione
  In questa situazione il registro $STVEC$ punterà alla prima cella di un array contenente vari indirizzi di varie routine. In base al tipo di eccezione contenuta nel registro $SCAUSE$, si scorrerà l'array e verrà scelta la routine adatta per quella causa (eseguendo $causa \cdot 4$)



Lo svantaggio è quello di dover accedere alla memoria per prelevare l'indirizzo della routine di gestione

Al verificarsi di un'eccezione esistono vari modi per salvare tutti i dati dello stato della macchina:

- Sullo stack
- In registri ausiliari
- In registri speciali

In RISC-V avviene sullo stack e su registri speciali ($SEPC$, $SCAUSE$, $SSTATUS$, $STVAL$, …)

- Il RISC-V ha vari registri a 64 bit
  - SEPC - contiene indirizzo dell'istruzione che ha generato l'eccezione
  - SSTATUS - contiene i bit di abilitazione globale degli interrupt
  - SCAUSE - I bit 63 e [3-0] codificano le possibili sorgenti di eccezione
    - ✓ Illegal instruction = {0, 2}
    - ✓ Breakpoint = {0, 3}
    - ✓ Time interrupt = {1, 5}
    - ✓ External interrupt = {1,9}
    - ✓ …
  - STVAL – Supervisor Trap Value – contiene l'indirizzo al quale si è verificato un riferimento errato alla memoria
  - SIE – Supervisor Interrupt Enable – specifica abilitazioni più fini degli interrupt in attesa
  - SIP – Supervisor Interrupt Pending – monitoraggio degli interrupt in attesa
  - STVEC – Supervisor Trap Vector – Indirizzo base della lista dei vettori di interrupt
  - SSCRATCH – Registro per salvataggi temporanei

Tutti questi registri si trovano in un banco interno al processore chiamato **Control Status Register** (CSR)

Per ogni eccezione, la parte di controllo della CPU modifica questi registri:

- Siccome il PC contiene l'indirizzo dell'istruzione successiva $PC+4$, bisogna memorizzarlo in $SEPC$ prima dell'incremento
- In questa situazione il PC viene direttamente sovrascritto mettendoci dentro il valore di $SEPC$



- Il livello di privilegio può essere S- per Supervisor o U- per User
  - Questi sono rispettivamente codificati con i valori 1 e 0
  - Chi si trova ad un certo livello non può sapere se esistono livelli di privilegio superiori (tale informazione non è accessibile)

- SIE (S- Interrupt Enable) abilita globalmente gli interrupt a quel dato livello – per non «entrare in loop» viene subito disabilitato (0)

- SPIE (S- Previous Interrupt Enable) indica lo stato precedente del bit SIE al momento in cui si verifica l'eccezione

- SPP (S- Previous Privilege mode) indica il livello di privilegio precedente al momento in cui si verifica l'eccezione

Controllo fine degli interrupt

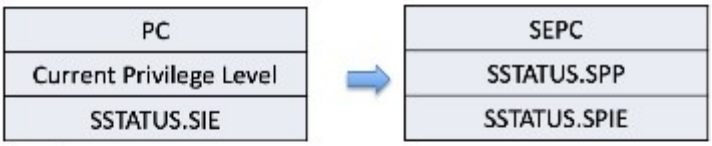


- SxIE sono bit di abilitazione più fine degli interrupt
- SxIP sono bit di indicazione di interrupt pendenti (in attesa)

- x si riferisce alla possibile sorgente dell'eccezione
  - x = E → interrupt esterno
  - x = T → interrupt dal timer
  - x = S → interrupt software (ovvero eccezione)

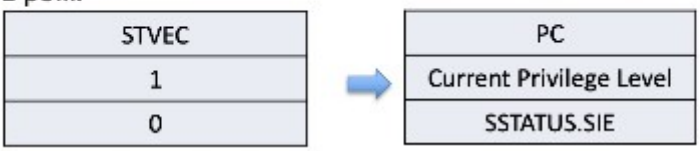
Ingresso ed Uscita in modalità Supervisor

- Al verificarsi di una eccezione



Salvataggio dello stato della macchina: valori correnti nei rispettivi registri dei valori precedenti

E poi...

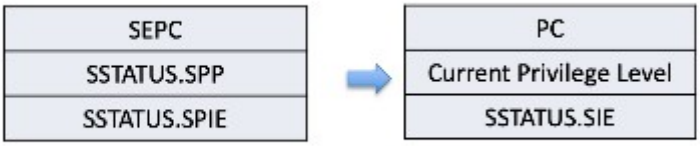


Salto alla routine di gestione

Modalità supervisor

Interrupt disabilitati

- All'esecuzione della SRET[1]



Ripristino dello stato della macchine: valori precedenti nei rispettivi registri dei valori correnti

[1]Tipicamente si usa SRET per ritornare da una routine di gestione interrupt o eccezione

SCAUSE Register



- Int (1 bit) se vale 1, la sorgente è un interrupt, se vale 0 la sorgente è una eccezione
- Code (4 bits) codifica la ragione dell'eccezione
  - 0 - Instruction address misaligned
  - 2 - Illegal instruction
  - 3 - Breakpoint
  - 4 - Load address misaligned
  - 5 - Load address fault
  - 6 - Store address misaligned
  - 7 - Store address fault
  - 8 - Environment call from U-mode
  - 9 - Environment call from S-mode
  - C - Instruction page fault
  - D - Load page fault
  - ...

Lettura/Scrittura di SEPC, SCAUSE, STVEC, STVAL, ...

- Per modificare i contenuti dei registri speciali CSR:
  - CSR Atomic Read & Write (/Immediate): CSRRW rd, csr, rs — CSRRWI rd, csr, imm
  - CSR Atomic Read & Set bits (/Immediate): CSRRS rd, csr, rs — CSRRSI rd, csr, imm
  - CSR Atomic Read & Clear bits (/Immediate): CSRRC rd, csr, rs — CSRRCI rd, csr, imm

- Esempi

```
CSRRW t0,stvec,t1      # t0 ← stvec, stvec ← t1
CSRRW x0,sscratch,t1   # sscratch ← t1          (in questo caso rd = x0: CSR non letto)

CSRRS t0,sstatus,t1    # t0 ← sstatus, sstatus ← sstatus OR t1
CSRRS t0,sstatus,x0    # t0 ← sstatus           (in questo caso rs = x0: CSR non scritto)

CSRRSI t0,sie,32       # t0 ← sie e mette a 1 il bit-5 (2^5) di sie (interrupt timer)
CSRRSI t0,sie,512      # NON possibile (l'immediato deve stare su 5 bit)

CSRRC t0,sie,t1        # t0 ← sstatus, sstatus ← sstatus OR NOT(t1)
                       # es. se t1=2^9+2^5=544, azzera i bit 9 e 5 di sie
                       # (disabilita interrupt esterni e interrupt timer)
```

- Nota
  - Se il vecchio valore di CSR non serve, CSRRS/CSRRSI e CSRRC/CSRRCI vengono chiamate con rd=x0

20

In RISC-V le uniche eccezioni che possono essere generate sono:

- Esecuzione di una istruzione non valida
- Malfunzionamenti hardware

- In caso di eccezione il processore deve:
  - Salvare l'indirizzo dell'istruzione che ha generato l'eccezione nel *registro program counter dell'eccezione* (*SEPC - supervision program counter* register)
  - Salvare la causa dell'eccezione nel *registro causa dell'eccezione* (*SCAUSE – supervisor cause exception* register)
  - Trasferire il controllo ad un indirizzo specifico del sistema operativo per gestire l'eccezione
  - Terminata la gestione ripristinare lo stato del processore e ritornare all'esecuzione precedente (se possibile)

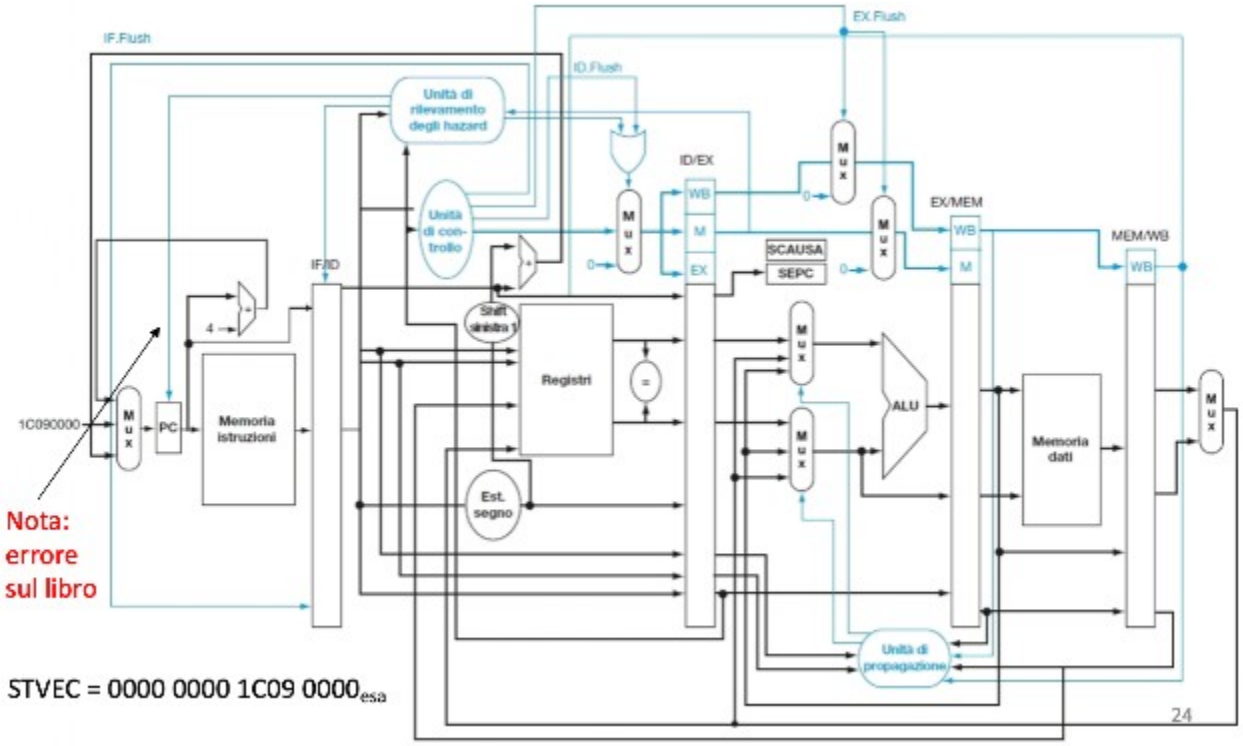
Le eccezioni in RISC-V vengono trattate come se fossero degli hazard sul controllo

Supporto del RISC-V per la gestione delle eccezioni (cont.)

- Per la gestione di una eccezione in IF si usa stesso meccanismo di gestione degli errori di predizione per i salti (convertendo l'istruzione in una nop)
- Introduciamo un nuovo segnale di controllo ID.Flush, messo in OR con il segnale di stallo che proviene dall'unità rilevamento hazard, in modo da eliminare l'istruzione dallo stadio ID e realizzare così uno stallo
- Introduciamo un nuovo segnale EX.Flush, che pilota un nuovo multiplexer per mettere a 0 i segnali di controllo di questo stadio
- Assumendo che l'indirizzo della prima istruzione del codice di gestione delle eccezioni sia $0000\ 0000\ 1C09\ 0000_{esa}$, per saltare al codice di gestione dell'eccezione occorre aggiungere una linea che porta il valore $0000\ 0000\ 1C09\ 0000_{esa}$ al multiplexer che seleziona il nuovo valore del PC
- Salviamo infine l'indirizzo dell'istruzione che ha causato l'eccezione nel registro SEPC

Supporto del RISC-V per la gestione delle eccezioni (cont.)

- Problema:
  - Se non si interrompe l'istruzione prima della fine della sua esecuzione non sarà più possibile vedere il valore originale del registro di destinazione (ad es. x1, che potrebbe essere «sporcato» dalla destinazione dell'istruzione che ha generato eccezione)
    - ✓ Se supponiamo che eccezione venga riconosciuta nello stadio EX si può utilizzare EX.Flush per prevenire che l'istruzione scriva il registro destinazione nello stadio WB
- Nota:
  - Molte eccezioni richiedono di completare normalmente l'esecuzione dell'istruzione che ha causato l'eccezione
    - ✓ Il modo più semplice per ottenere ciò consiste nell'eliminare l'istruzione e farla eventualmente ripartire dall'inizio dopo che l'eccezione è stata gestita



STVEC = 0000 0000 1C09 0000_{esa}